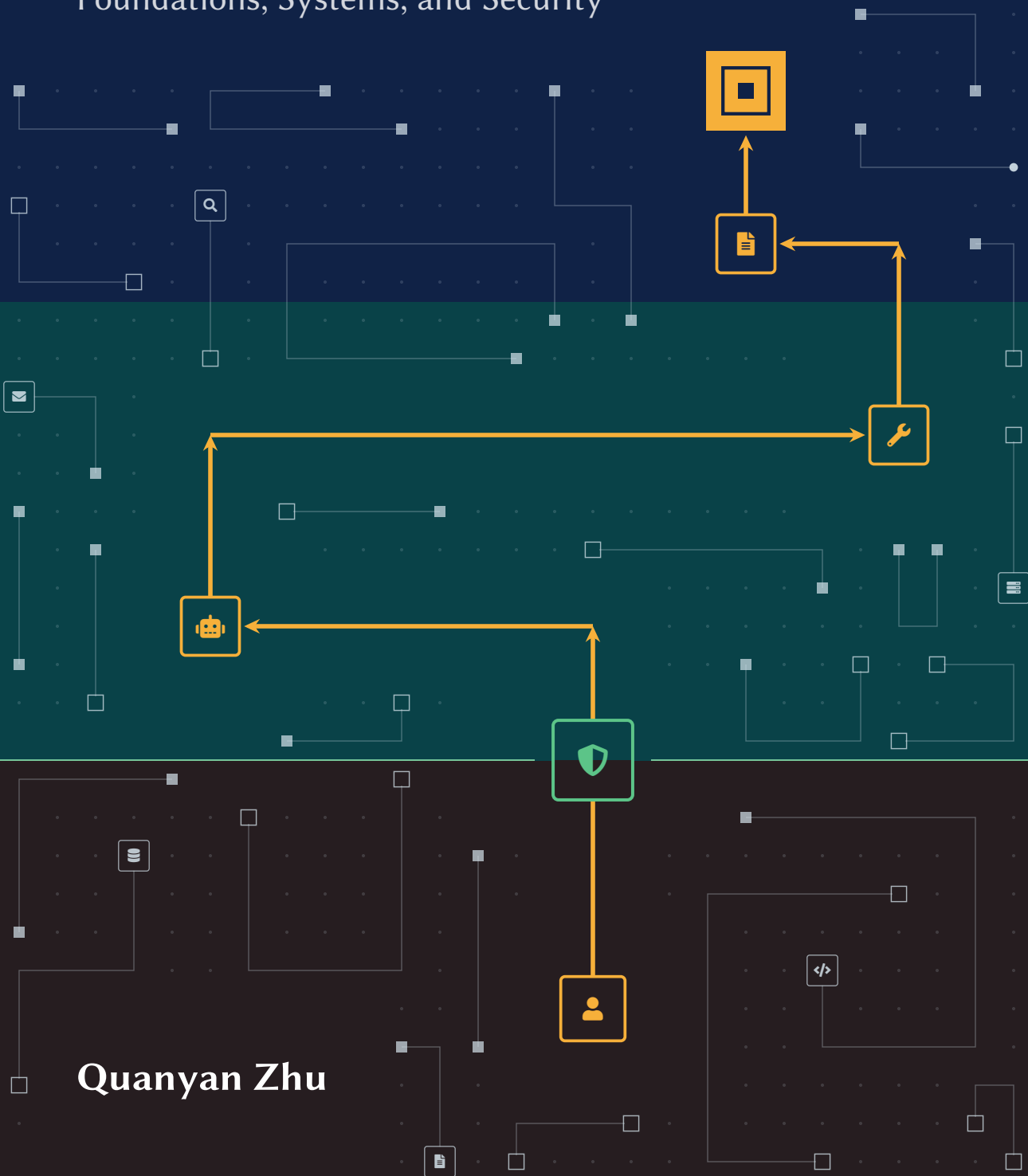


# Agentic AI

Foundations, Systems, and Security



Quanyan Zhu

# Agentic AI

*Foundations, Systems, and Security*

---

QUANYAN ZHU

*Agentic AI: Foundations, Systems, and Security*

Copyright © 2026 Quanyan Zhu. All rights reserved.

No part of this book may be reproduced in any form or by any electronic or mechanical means, including information storage and retrieval systems, without permission in writing from the author, except for brief quotations in reviews and scholarly works.

The author has made every effort to ensure the accuracy of the material in this book. Descriptions of products, protocols, benchmarks, and regulations are current as of the dates given in the text and may have changed since.

Version of September 22, 2026.

Typeset by the author in Libertinus Serif using  $\text{\LaTeX}$ . Icons in figures are from Font Awesome Free (fontawesome.com), used under the Creative Commons Attribution 4.0 License.

---

# Preface

Agentic AI has grown quickly in the past few years. Artificial intelligence is moving from systems that answer to systems that act. An agentic AI system receives a goal, breaks it into steps, calls tools, reads what comes back, and keeps going, often for many steps and with real authority delegated by a person or an organization. Today’s language agents can plan multistep tasks, search and read the web, write and run code, query databases, operate graphical computer interfaces, and delegate work to other agents. Their reach is lengthening fast. METR tracks the length of task, measured by the time a skilled human needs, that frontier agents complete with even odds. Between 2019 and early 2025 that length doubled roughly every seven months (Kwa et al., 2025). Products followed the research. Computer-use agents appeared between October 2024 and October 2025 (Anthropic, 2024; OpenAI, 2025a), and in May 2025 cloud coding agents began taking assigned tasks and returning proposed changes for human review (OpenAI, 2025b; GitHub, 2025).

Forecasters expect this growth to continue. In August 2025 Gartner predicted that 40% of enterprise applications would integrate task-specific AI agents by the end of 2026, up from less than 5% in 2025 (Gartner, 2025b). Two months earlier, the same firm predicted that over 40% of agentic AI projects would be canceled by the end of 2027 because of escalating costs, unclear business value, or inadequate risk controls (Gartner, 2025a). The two forecasts belong together. Adoption is outrunning understanding, and the gap is paid for in failed projects, incidents, and lost trust.

Agents are already at work across many sectors and applications. They write and review software, generate and test scientific hypotheses, triage security alerts and look for vulnerabilities, answer customers and run enterprise workflows, draft clinical documentation, initiate payments, and plan for robots and industrial systems (Chapter 12). They are deployed by technology companies and startups, by enterprises of every size, by research laboratories, and by public agencies, and they are used every day by individual developers, analysts, and scientists. Agents are also beginning to deal with one another. They discover each other’s capabilities, delegate subtasks, and negotiate across organizational boundaries, which opens the way to networks of interacting agents

(Zhu, 2026f).

## Why Foundations

Understanding the fundamental principles of agentic AI is key, because deploying an agent responsibly raises questions that no benchmark or product release settles. The first concerns resources. How should an agent spend tokens, compute, time, and human attention, and when does more reasoning, more sampling, or another round of verification pay for itself (Yang and Zhu, 2026c; Zhu, 2026c)? The second concerns security. An agent reads untrusted content while holding real permissions, and attackers write part of its input and adapt to its defenses. The third concerns interoperability. Agents from different vendors now connect to shared tools and to one another through protocols such as the Model Context Protocol (Model Context Protocol Contributors, 2026) and Agent2Agent (Agent2Agent Protocol Contributors, 2026). What must those protocols guarantee, and who bears the risk when a composition fails? The fourth concerns governance: how trust, accountability, and liability are allocated along the chain from model developer to deployer, user, and agent. The fifth concerns emergent risks, the failures that appear only at scale, such as correlated errors across copies of one model, cascades through multi-agent pipelines, and tacit collusion among pricing agents (Fish et al., 2026). The last concerns agent networks. What collective behavior emerges when many self-interested agents, acting for different principals, coordinate, compete, and form coalitions?

To answer such questions we need foundations. Without them we proceed by trial and error, which is expensive in any engineering discipline and dangerous in an adversarial one. When failures are found by trial, the first to find them may be an attacker.

This book aims to build system-scientific foundations for agentic AI. It treats an agent as a feedback system embedded in an environment that contains principals, adversaries, other agents, and institutions. It draws on decision theory, control, learning, game theory, and information theory, the disciplines that have long studied such systems, to develop a theory of agentic AI, and it connects that theory to the engineering of real agents and to their security, safety, and governance. The book approaches these questions from three complementary perspectives, one per part. Part I, Foundations, treats agents as sequential decision-makers and develops decision theory, Markov decision processes and partial observability, learning, planning and control, game theory, and strategic information. Part II, Systems, turns to how agents are built: architectures, tools and environments, memory and context, multi-agent orchestration, evaluation, and case studies. Part III, Security, Safety, and Governance, covers threat models, adversarial games and strategic defense, resilience and runtime assurance, alignment and human oversight, and trust, accountability, and regulation.

The central thesis is that these perspectives need each other. Engineering patterns without theory give no guarantees. Theory without systems misses the ways real agents

fail. And neither is enough without a strategic account of adversaries and institutions. The final chapter collects the open problems where the three meet ([Chapter 18](#)).

Volatile facts, including products, protocols, prices, regulations, and deployment figures, are dated throughout the book and collected in “In Practice” boxes, so that readers can tell lasting principles from the state of the field as of the time of writing (September 2026).

## Audience and Prerequisites

The book is written for graduate students, advanced undergraduates, and researchers and practitioners in computer science, electrical engineering, operations research, and related fields. Readers should be comfortable with probability, linear algebra, and basic optimization, and should have some programming experience. Appendices A–C review the required background in probability and optimization, game theory, and reinforcement learning and control.

## Features

Every chapter opens with learning objectives and closes with key takeaways, notes on further reading, and exercises that range from proofs to programming. Three running examples recur throughout: a research assistant agent (R1), a software engineering agent (R2), and a security operations agent (R3). Appendix D gives laboratory exercises that build a working agent step by step.

## Using This Book in a Course

The book supports several courses. A theory course on the foundations of agentic AI can cover Chapters 1–6, 14, and 16, with Appendices A–C as background. A systems course on building agents can cover Chapter 1, Sections 2.5–2.6 and 4.4, Chapters 7–13 and 15, and the laboratory exercises of Appendix D. A course on secure and trustworthy agents can cover Chapters 1, 5, 7, 8, and 13–17.

## Acknowledgments

I would like to thank my coauthors and students, who have shared their insights and visions with me through many discussions.

Quanyan Zhu  
September 2026

---

# Contents

|          |     |
|----------|-----|
| Preface  | iii |
| Notation | xii |

## I FOUNDATIONS: AGENTS AS DECISION-MAKERS

|     |                                                                   |    |
|-----|-------------------------------------------------------------------|----|
| 1   | Agentic AI: Concepts, History, and Scope                          | 2  |
| 1.1 | What Makes a System Agentic? . . . . .                            | 3  |
| 1.2 | A Brief History of Artificial Agents . . . . .                    | 10 |
| 1.3 | The Anatomy of an Agentic System . . . . .                        | 16 |
| 1.4 | A Taxonomy of Agentic Systems . . . . .                           | 20 |
| 1.5 | Why Agentic AI Raises New Questions . . . . .                     | 24 |
| 1.6 | How to Read This Book . . . . .                                   | 28 |
|     | Notes and Further Reading . . . . .                               | 31 |
|     | Exercises . . . . .                                               | 33 |
| 2   | Agents as Sequential Decision-Makers                              | 36 |
| 2.1 | Decision Making under Uncertainty . . . . .                       | 37 |
| 2.2 | Markov Decision Processes . . . . .                               | 39 |
| 2.3 | Partial Observability . . . . .                                   | 43 |
| 2.4 | Goals, Rewards, and Specifications . . . . .                      | 48 |
| 2.5 | Long-Horizon Behavior and Error Compounding . . . . .             | 51 |
| 2.6 | Agents with Language Action Spaces . . . . .                      | 57 |
|     | Notes and Further Reading . . . . .                               | 61 |
|     | Exercises . . . . .                                               | 62 |
| 3   | Learning Agents: From Reinforcement Learning to Foundation Models | 64 |
| 3.1 | Reinforcement Learning Fundamentals. . . . .                      | 65 |
| 3.2 | Lessons from Deep Reinforcement Learning . . . . .                | 70 |
| 3.3 | Foundation Models as Priors for Agents . . . . .                  | 72 |
| 3.4 | Post-Training for Agency. . . . .                                 | 74 |
| 3.5 | Learning to Use Tools and Act . . . . .                           | 82 |

|     |                                                    |     |
|-----|----------------------------------------------------|-----|
| 3.6 | Continual and Open-Ended Learning . . . . .        | 87  |
|     | Notes and Further Reading . . . . .                | 90  |
|     | Exercises . . . . .                                | 92  |
| 4   | Reasoning, Planning, and Control . . . . .         | 96  |
| 4.1 | Classical Planning and Search. . . . .             | 97  |
| 4.2 | Planning under Uncertainty . . . . .               | 101 |
| 4.3 | Reasoning in Language Models . . . . .             | 106 |
| 4.4 | Interleaving Reasoning and Acting . . . . .        | 112 |
| 4.5 | A Control-Theoretic View of Agent Loops . . . . .  | 115 |
| 4.6 | Limits of LLM Planning . . . . .                   | 122 |
|     | Notes and Further Reading . . . . .                | 125 |
|     | Exercises . . . . .                                | 126 |
| 5   | Multi-Agent Systems and Game Theory . . . . .      | 129 |
| 5.1 | Strategic Interaction . . . . .                    | 130 |
| 5.2 | Dynamic and Stochastic Games . . . . .             | 136 |
| 5.3 | Incomplete Information and Incentives. . . . .     | 140 |
| 5.4 | Learning in Games. . . . .                         | 146 |
| 5.5 | Cooperation, Coordination, and Collusion . . . . . | 150 |
| 5.6 | Large Populations of Agents . . . . .              | 154 |
|     | Notes and Further Reading . . . . .                | 159 |
|     | Exercises . . . . .                                | 161 |
| 6   | Information, Beliefs, and Communication . . . . .  | 164 |
| 6.1 | Information Structures . . . . .                   | 165 |
| 6.2 | Strategic Communication. . . . .                   | 169 |
| 6.3 | Modeling Other Minds . . . . .                     | 177 |
| 6.4 | Language as a Coordination Protocol . . . . .      | 183 |
| 6.5 | Deception and Manipulation . . . . .               | 186 |
|     | Notes and Further Reading . . . . .                | 190 |
|     | Exercises . . . . .                                | 192 |

## II SYSTEMS: ENGINEERING AGENTIC AI

|     |                                                   |     |
|-----|---------------------------------------------------|-----|
| 7   | Architectures of LLM-Based Agents . . . . .       | 196 |
| 7.1 | The Agent Loop . . . . .                          | 197 |
| 7.2 | Cognitive Architectures Revisited . . . . .       | 204 |
| 7.3 | Design Patterns . . . . .                         | 209 |
| 7.4 | Structured Outputs and Control Flow . . . . .     | 216 |
| 7.5 | Case Study: A Software Engineering Agent. . . . . | 224 |
| 7.6 | Cost, Latency, and Reliability . . . . .          | 228 |
|     | Notes and Further Reading . . . . .               | 237 |
|     | Exercises . . . . .                               | 239 |

|      |                                                       |     |
|------|-------------------------------------------------------|-----|
| 8    | Tools, Actions, and Environments                      | 242 |
| 8.1  | Action Spaces for Digital Agents . . . . .            | 243 |
| 8.2  | Tool Specification and Discovery . . . . .            | 250 |
| 8.3  | Execution Environments . . . . .                      | 258 |
| 8.4  | Embodied and Cyber-Physical Action . . . . .          | 266 |
| 8.5  | Designing Agent–Computer Interfaces. . . . .          | 270 |
|      | Notes and Further Reading . . . . .                   | 274 |
|      | Exercises . . . . .                                   | 276 |
| 9    | Memory, Knowledge, and Context                        | 280 |
| 9.1  | A Taxonomy of Agent Memory . . . . .                  | 281 |
| 9.2  | Retrieval-Augmented Generation . . . . .              | 284 |
| 9.3  | Long Context and Context Engineering . . . . .        | 291 |
| 9.4  | Persistent Memory Systems. . . . .                    | 298 |
| 9.5  | Structured Knowledge . . . . .                        | 303 |
| 9.6  | Memory as a Decision Problem . . . . .                | 305 |
|      | Notes and Further Reading . . . . .                   | 313 |
|      | Exercises . . . . .                                   | 315 |
| 10   | Multi-Agent Orchestration                             | 318 |
| 10.1 | Why Multiple Agents? . . . . .                        | 319 |
| 10.2 | Orchestration Topologies. . . . .                     | 325 |
| 10.3 | Collaboration Mechanisms . . . . .                    | 327 |
| 10.4 | Interoperability Protocols. . . . .                   | 335 |
| 10.5 | Coordination Theory for LLM Agents . . . . .          | 341 |
| 10.6 | Failure Modes of Multi-Agent Systems . . . . .        | 348 |
|      | Notes and Further Reading . . . . .                   | 354 |
|      | Exercises . . . . .                                   | 355 |
| 11   | Evaluation and Benchmarking                           | 358 |
| 11.1 | What to Measure . . . . .                             | 359 |
| 11.2 | The Benchmark Landscape . . . . .                     | 363 |
| 11.3 | Evaluation Methodology . . . . .                      | 368 |
| 11.4 | Long-Horizon Capability . . . . .                     | 377 |
| 11.5 | Human Evaluation and Field Studies. . . . .           | 380 |
| 11.6 | Evaluating Dangerous Capabilities . . . . .           | 381 |
|      | Notes and Further Reading . . . . .                   | 384 |
|      | Exercises . . . . .                                   | 385 |
| 12   | Agentic AI in Practice: Case Studies                  | 389 |
| 12.1 | A Template for Case Studies . . . . .                 | 390 |
| 12.2 | Software Engineering Agents . . . . .                 | 392 |
| 12.3 | Agents for Scientific Discovery . . . . .             | 396 |
| 12.4 | Agents in Cyber Operations . . . . .                  | 399 |
| 12.5 | Enterprise and Customer Operations . . . . .          | 403 |
| 12.6 | High-Stakes Domains: Healthcare and Finance . . . . . | 406 |
| 12.7 | Robotics and Cyber-Physical Systems . . . . .         | 411 |

|      |                                     |     |
|------|-------------------------------------|-----|
| 12.8 | Lessons across Domains . . . . .    | 414 |
|      | Notes and Further Reading . . . . . | 420 |
|      | Exercises . . . . .                 | 422 |

### III SECURITY, SAFETY, AND GOVERNANCE

|      |                                                                |     |
|------|----------------------------------------------------------------|-----|
| 13   | Threat Models for Agentic Systems . . . . .                    | 426 |
| 13.1 | Security Principles Revisited . . . . .                        | 427 |
| 13.2 | The Agentic Attack Surface . . . . .                           | 430 |
| 13.3 | Prompt Injection . . . . .                                     | 432 |
| 13.4 | Tool and Privilege Misuse . . . . .                            | 440 |
| 13.5 | Poisoning and Backdoors . . . . .                              | 444 |
| 13.6 | Threats in Multi-Agent Ecosystems . . . . .                    | 448 |
| 13.7 | Misuse of Agents by Attackers . . . . .                        | 451 |
|      | Notes and Further Reading . . . . .                            | 453 |
|      | Exercises . . . . .                                            | 454 |
| 14   | Adversarial Games and Strategic Defense . . . . .              | 457 |
| 14.1 | Security as a Game. . . . .                                    | 457 |
| 14.2 | Incomplete Information, Deception, and Moving Targets. . . . . | 462 |
| 14.3 | Dynamic Games of Compromise . . . . .                          | 467 |
| 14.4 | Robustness as a Minimax Problem. . . . .                       | 472 |
| 14.5 | Automated Red Teaming . . . . .                                | 475 |
| 14.6 | Case Study: Stackelberg Prompt-Injection Defense . . . . .     | 479 |
|      | Notes and Further Reading . . . . .                            | 484 |
|      | Exercises . . . . .                                            | 485 |
| 15   | Resilience, Guardrails, and Runtime Assurance . . . . .        | 488 |
| 15.1 | From Robustness to Resilience . . . . .                        | 489 |
| 15.2 | Defense in Depth for Agents . . . . .                          | 492 |
| 15.3 | Sandboxing and Containment. . . . .                            | 501 |
| 15.4 | Runtime Monitoring . . . . .                                   | 504 |
| 15.5 | Formal Methods and Runtime Assurance . . . . .                 | 509 |
| 15.6 | Incident Response and Recovery . . . . .                       | 512 |
|      | Notes and Further Reading . . . . .                            | 515 |
|      | Exercises . . . . .                                            | 516 |
| 16   | Alignment and Human Oversight . . . . .                        | 519 |
| 16.1 | The Alignment Problem for Agents . . . . .                     | 520 |
| 16.2 | Specification Gaming and Reward Hacking. . . . .               | 522 |
| 16.3 | Learning Human Preferences and Values. . . . .                 | 526 |
| 16.4 | Scalable Oversight . . . . .                                   | 529 |
| 16.5 | Corrigibility and Control . . . . .                            | 532 |
| 16.6 | Transparency and Interpretability . . . . .                    | 536 |
| 16.7 | Designing Human Oversight . . . . .                            | 540 |
|      | Notes and Further Reading . . . . .                            | 547 |

|    |                                                            |     |
|----|------------------------------------------------------------|-----|
|    | Exercises . . . . .                                        | 548 |
| 17 | Trust, Accountability, and Governance                      | 552 |
|    | 17.1 Trust in Agentic Systems . . . . .                    | 553 |
|    | 17.2 Accountability and Liability. . . . .                 | 556 |
|    | 17.3 Risk Management Frameworks . . . . .                  | 562 |
|    | 17.4 Regulation . . . . .                                  | 566 |
|    | 17.5 Mechanism Design for Governance . . . . .             | 571 |
|    | 17.6 Societal and Economic Impacts . . . . .               | 578 |
|    | Notes and Further Reading . . . . .                        | 581 |
|    | Exercises . . . . .                                        | 581 |
| 18 | Open Problems and the Road Ahead                           | 584 |
|    | 18.1 Open Problems in Foundations . . . . .                | 585 |
|    | 18.2 Open Problems in Systems . . . . .                    | 589 |
|    | 18.3 Open Problems in Security and Safety . . . . .        | 591 |
|    | 18.4 Open Problems in Governance . . . . .                 | 594 |
|    | 18.5 Toward a Science of Agentic AI . . . . .              | 597 |
|    | Notes and Further Reading . . . . .                        | 600 |
|    | Research Projects . . . . .                                | 600 |
| A  | Probability, Optimization, and Information Theory          | 603 |
|    | A.1 Probability and Conditional Expectation . . . . .      | 603 |
|    | A.2 Concentration Inequalities . . . . .                   | 606 |
|    | A.3 Convex Optimization and Duality . . . . .              | 609 |
|    | A.4 Information Theory Essentials . . . . .                | 613 |
|    | Notes and Further Reading . . . . .                        | 615 |
| B  | Game Theory Primer                                         | 616 |
|    | B.1 Fixed-Point Theorems . . . . .                         | 616 |
|    | B.2 Existence of Equilibria . . . . .                      | 617 |
|    | B.3 Zero-Sum Games and Linear Programming. . . . .         | 620 |
|    | B.4 Computing Equilibria . . . . .                         | 623 |
|    | Notes and Further Reading . . . . .                        | 625 |
| C  | Reinforcement Learning and Control Primer                  | 626 |
|    | C.1 Contraction Mappings and Dynamic Programming . . . . . | 626 |
|    | C.2 Convergence of Value and Policy Iteration . . . . .    | 628 |
|    | C.3 Stochastic Approximation . . . . .                     | 630 |
|    | C.4 Linear-Quadratic Control . . . . .                     | 633 |
|    | Notes and Further Reading . . . . .                        | 637 |
| D  | Laboratory Exercises                                       | 639 |
|    | D.1 Setup and Conventions. . . . .                         | 639 |
|    | D.2 Lab 1: A Minimal Agent Loop. . . . .                   | 644 |
|    | D.3 Lab 2: Tools and the Model Context Protocol . . . . .  | 647 |
|    | D.4 Lab 3: Retrieval and Memory . . . . .                  | 649 |

|              |                                                            |     |
|--------------|------------------------------------------------------------|-----|
| D.5          | Lab 4: Multi-Agent Debate . . . . .                        | 650 |
| D.6          | Lab 5: Prompt-Injection Red Teaming in a Sandbox . . . . . | 652 |
| D.7          | Lab 6: Building an Evaluation Harness . . . . .            | 656 |
| Bibliography |                                                            | 95  |

---

# Notation

Sets and spaces are calligraphic ( $\mathcal{S}, \mathcal{A}$ ), and discrete labels such as states of an error chain or action classes are sans serif (G, I). The logarithm  $\log$  is natural unless a base is shown,  $x^\top$  is the transpose, and  $V^*, Q^*, \pi^*$  denote optimal quantities. Subscripts and superscripts on  $\mathbb{E}$  and  $\mathbb{P}$  name the distribution or policy that generates the randomness, as in  $\mathbb{E}_\nu$  or  $\mathbb{E}^\pi$ .

A book that spans decision theory, learning, games, and security needs more symbols than two alphabets supply. The tables below list notation used across chapters; a symbol listed here keeps its meaning unless a chapter explicitly redefines it. Symbols local to a single section are defined where they appear. The last table collects the letters that carry different meanings in different chapters.

---

## General

---

|                                                           |                                                             |
|-----------------------------------------------------------|-------------------------------------------------------------|
| $\mathbb{R}$                                              | Real numbers                                                |
| $\mathbb{E}[\cdot], \mathbb{P}(\cdot), \text{Var}(\cdot)$ | Expectation, probability, variance                          |
| $\mathbb{1}\{\cdot\}, \mathbf{1}$                         | Indicator function; all-ones vector                         |
| $\Delta(X)$                                               | Set of probability distributions over a finite set $X$      |
| $\mathcal{N}(\mu, \sigma^2)$                              | Normal distribution with mean $\mu$ and variance $\sigma^2$ |
| $D_{\text{KL}}(p \parallel q)$                            | Kullback–Leibler divergence                                 |
| $d_{\text{TV}}(\nu, \nu')$                                | Total variation distance (Theorem A.3)                      |
| $H(X), I(X; Y)$                                           | Entropy and mutual information (Theorem A.24)               |
| $\arg \max, \arg \min$                                    | Set of maximizers, minimizers                               |
| $\ \cdot\ _\infty$                                        | Supremum norm                                               |
| $[z]^+$                                                   | Positive part, $\max(z, 0)$                                 |

---

---

**Decision processes and learning (Part I)**


---

|                                                           |                                                                                                 |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| $s \in \mathcal{S}, a \in \mathcal{A}, o \in \mathcal{O}$ | State, action, observation                                                                      |
| $P(s'   s, a)$                                            | Transition kernel                                                                               |
| $r(s, a), \gamma$                                         | Reward function, discount factor                                                                |
| $R_t, R_{\max}$                                           | Reward received at step $t$ ; bound on rewards                                                  |
| $\Omega(o   s', a), b_0$                                  | Observation kernel; initial state distribution (Theorem 2.6)                                    |
| $h_t \in \mathcal{H}$                                     | History $(o_0, a_0, \dots, o_t)$ (Theorem 2.6)                                                  |
| $b \in \Delta(\mathcal{S}) = \Delta(S), \beta(b, a, o)$   | Belief state; Bayesian belief update (Eq. (2.3))                                                |
| $\pi(a   s), \pi(a   c)$                                  | Policy on states or on contexts                                                                 |
| $V^\pi, Q^\pi, V^*, Q^*$                                  | Value, action-value, and optimal value functions                                                |
| $\mathcal{B}, \mathcal{B}^\pi$                            | Bellman optimality and policy-evaluation operators (Eq. (2.2))                                  |
| $T$                                                       | Horizon; number of steps in an episode                                                          |
| $\mu : \mathcal{H} \rightarrow \mathcal{Z}$               | Compression map from histories to information states (Theorem 2.8)                              |
| $\hat{r}, \hat{P}, (\delta_r, \delta_p)$                  | Reward and transition models of an approximate information state and their errors (Theorem 2.9) |
| $\pi_\theta, J(\theta)$                                   | Parameterized policy and its expected return (Chapter 3)                                        |
| $r_\phi, \mathcal{D}_{\text{pref}}$                       | Learned reward model; preference dataset (Eq. (3.9))                                            |
| $\pi_{\text{ref}}, \beta$                                 | Reference policy and KL weight in RLHF (Eq. (3.10))                                             |

---

**Language-agent interface (introduced in Chapter 2, used throughout Part II)**


---

|                                         |                                                                                                                                                                                                                 |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\Sigma^*$                              | Strings over the model's token vocabulary $\Sigma$ , also written $\mathcal{V}$                                                                                                                                 |
| $p_\theta(y   x)$                       | Language model with weights $\theta$ : probability of output $y$ given input $x$                                                                                                                                |
| $c_t \in \mathcal{C}$                   | Context at step $t$                                                                                                                                                                                             |
| $U$                                     | Context update, $c_{t+1} = U(c_t, a_t, o_{t+1})$ (Theorem 2.15)                                                                                                                                                 |
| render, parse, $\perp$                  | Renderer from contexts to model input; parser from model output to $\mathcal{A} \cup \{\perp\}$ ; parse failure (Eqs. (2.9) and (8.1))                                                                          |
| $\tau \in \mathcal{T}, X_\tau, f_\tau$  | Tool, its admissible arguments, and its executor (Theorem 8.1)                                                                                                                                                  |
| $(\tau, x), \text{msg}(m), \text{stop}$ | Tool call, message to the user, termination (Theorem 8.2)                                                                                                                                                       |
| $\mathcal{M}$                           | Memory store                                                                                                                                                                                                    |
| $T, B_{\text{tok}}, B_{\text{cost}}, N$ | Step, token, and cost budgets; step at which the episode ends (Theorem 2.15)                                                                                                                                    |
| $B, C_B$                                | Context budget in tokens; contexts within the budget (Theorem 9.7)                                                                                                                                              |
| $\mu : \mathcal{H} \rightarrow C_B$     | Memory policy (Theorem 9.7)                                                                                                                                                                                     |
| $\mathcal{D}, \kappa$                   | Deployment boundary; restoration budget (Theorem 8.7)                                                                                                                                                           |
| $\text{cls}(a, s)$                      | Class of action $a$ in state $s$ , in $\{\mathbf{R}, \mathbf{V}, \mathbf{I}\} \times \{\mathbf{C}, \mathbf{X}\}$ : read-only, reversible, or irreversible effect; contained or externally visible (Theorem 8.7) |

---

---

**Reliability and verification**


---

|                                |                                                                                                              |
|--------------------------------|--------------------------------------------------------------------------------------------------------------|
| $G, E, F$                      | On track, off track, failed (Eq. (2.6))                                                                      |
| $\epsilon, \rho$               | Per-step probability of a recoverable error; recovery probability (Theorem 2.13); per stage in Theorem 10.12 |
| $\phi, \psi$                   | Per-step probability of irrecoverable failure when on track and off track (Theorem 2.13)                     |
| $g_T$                          | Probability of success after $T$ steps (Eq. (2.7))                                                           |
| $d, f$                         | Verifier detection rate and false-alarm rate (Theorem 10.12)                                                 |
| $\text{pass}@k, \text{pass}^k$ | Probability that at least one, or all, of $k$ attempts succeed (Theorem 11.2)                                |

---

**Games and information (Part I)**


---

|                                       |                                                       |
|---------------------------------------|-------------------------------------------------------|
| $i \in \mathcal{N} = \{1, \dots, n\}$ | Player (agent) index                                  |
| $A_i, A = \prod_i A_i$                | Actions of player $i$ ; action profiles (Theorem 5.1) |
| $a_{-i}, \sigma_{-i}$                 | Actions or strategies of all players other than $i$   |
| $u_i$                                 | Utility of player $i$                                 |
| $\sigma_i \in \Delta(A_i), \sigma$    | Mixed strategy of player $i$ ; strategy profile       |
| $\text{BR}_i$                         | Best-response correspondence of player $i$            |
| $\theta_i \in \Theta_i$               | Private type of player $i$ (Theorem 5.14)             |
| $R_i^T$                               | Regret of player $i$ after $T$ rounds (Theorem 5.21)  |

---

---

**Security, safety, and governance (Part III)**


---

|                                           |                                                                                                                                 |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| $\Lambda = \Lambda^T \sqcup \Lambda^U$    | Trusted and untrusted channels ( <a href="#">Chapter 13</a> )                                                                   |
| $\Lambda_{\text{adv}}, m$                 | Channels the adversary can write to; number of attempts ( <a href="#">Theorem 13.1</a> )                                        |
| $G \subseteq \mathcal{A}, \mathcal{V}(c)$ | Attacker goal; adversarial variants of context $c$ ( <a href="#">Theorem 13.3</a> )                                             |
| $\text{Adv}_G(c, c')$                     | Injection advantage ( <a href="#">Eq. (13.1)</a> )                                                                              |
| $H \subseteq \mathcal{A}$                 | High-impact actions ( <a href="#">Theorem 13.5</a> )                                                                            |
| $\beta, q, L$                             | Probability that injected content passes the detector; susceptibility; loss per compromise ( <a href="#">Theorem 13.7</a> )     |
| $k, D, r$                                 | Branching factor, depth, and per-edge transmission probability of a delegation tree ( <a href="#">Theorem 13.10</a> )           |
| $x, U_d(x, i), U_a(x, i)$                 | Coverage vector; defender and attacker expected payoffs when target $i$ is attacked ( <a href="#">Chapter 14</a> )              |
| $\ell(\pi, w), R_D^T, R_A^T$              | Success probability of attack $w$ against configuration $\pi$ ; defender and attacker regret ( <a href="#">Theorem 14.12</a> )  |
| $U(k)$                                    | Upper confidence bound on per-attempt success after $k$ successful red-team trials ( <a href="#">Theorem 14.13</a> )            |
| $Q(t), Q_0(t)$                            | Task performance and its nominal level                                                                                          |
| $\mathcal{R}, \text{TTD}, \text{TTR}$     | Resilience over an incident window; time to detect; time to recover ( <a href="#">Theorem 15.1</a> )                            |
| $\mathcal{K}, G(k, z)$                    | Lattice of action classes; approval policy ( <a href="#">Theorem 15.5</a> )                                                     |
| $d(\lambda), C$                           | Reviewer detection rate at review load $\lambda$ ; review capacity ( <a href="#">Theorem 16.8</a> )                             |
| $h_i, h^*$                                | Expected harm of action class $i$ ; review threshold ( <a href="#">Eq. (16.2)</a> )                                             |
| $r(z), \ell(z)$                           | Conditional reliability and reliance rate given signal $z$ ( <a href="#">Theorem 17.1</a> )                                     |
| $q, F, s$                                 | Audit probability, penalty, and savings from shirking; compliance holds iff $q(d - f)F \geq s$ ( <a href="#">Theorem 17.6</a> ) |

---

---

**Letters with chapter-specific meanings**


---

|                                         |                                                                                                                                                |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| $\theta$                                | Model weights ( $p_\theta, \pi_\theta$ ); private type (Chapters 5 and 14); decision thresholds (Chapters 7 and 15)                            |
| $\mu$                                   | Compression map or memory policy; correlated-equilibrium distribution and common prior (Chapters 5 and 6); mean                                |
| $\beta$                                 | Belief update (Chapter 2); KL weight (Chapter 3); detector pass probability (Chapter 13)                                                       |
| $\delta$                                | Errors $\delta_r, \delta_p$ ; temporal-difference error (Chapter 3); confidence level (Chapters A and 13); discount factor (Chapters 5 and 17) |
| $\sigma$                                | Strategy; logistic function (Chapter 3); standard deviation                                                                                    |
| $\tau$                                  | Tool; trajectory (Chapter 3)                                                                                                                   |
| $\epsilon, \varepsilon$                 | Error probabilities and accuracy levels; exploration rate and clipping range (Chapter 3)                                                       |
| $T$                                     | Horizon; contraction map (Chapter C); trusted model (Chapter 15)                                                                               |
| $d, f$                                  | Detection and false-alarm rates; distortion $d$ (Chapter 9); tool executor $f_\tau$                                                            |
| $G$                                     | Attacker goal (Chapter 13); approval policy (Chapter 15)                                                                                       |
| $\mathcal{N}, \mathcal{M}, \mathcal{T}$ | Players, memory, tools; also normal distribution, meet of partitions (Chapter 6), Shapley operator (Chapter 5)                                 |

---

PART I

# Foundations: Agents as Decision-Makers

---

*The mathematical core: what an agent is, how it decides, learns,  
plans, and interacts with other agents.*

---

# Agentic AI: Concepts, History, and Scope

*Agentic AI systems pursue goals on a principal's behalf through extended loops of perception, reasoning, and action. This chapter gives a working definition of an agentic system and follows one short episode of such a system action by action. It treats agency and autonomy as matters of degree, pictures six levels of autonomy, and separates agents from chatbots and fixed workflows by asking who chooses the next step and who acts. It places foundation-model agents in a lineage that runs from cybernetics through symbolic planning, belief–desire–intention architectures, and reinforcement learning, and describes how, between 2024 and 2026, coding, research, and computer-use agents moved from research prototypes to deployed products. It then introduces a reference architecture, three running examples used throughout the book, and a taxonomy. It explains why long horizons, open-ended action spaces, adversaries, and delegation raise new questions, with a calculation that links error recovery to the task horizons now used to track progress. It closes with a guide to the book's three perspectives: theory, systems, and security.*

---

## LEARNING OBJECTIVES

- Define agent, agency, and autonomy, and distinguish agentic systems from chatbots, generative models, and fixed workflows by who chooses the next step, who acts, and with what authority.
- Trace a single episode of the agent loop step by step, and identify which steps exercise goal-directedness, interaction, and delegated authority.
- Assign levels of autonomy per action class, using levels of automation from the human factors literature.
- Trace the intellectual history of artificial agents from feedback control to foundation-model agents, identify which ideas each era contributed, and describe the kinds of agents deployed as of 2026.
- Describe the reference architecture of an agentic system and the three running examples used in later chapters.

- Classify agentic systems by degree of autonomy, environment, number of agents, and learning mode.
  - Explain why long horizons, open-ended action spaces, adversaries, and delegation create new technical and societal questions; compute how per-step failure and recovery determine the horizon at which an agent succeeds half the time; and locate the chapters that address each question.
- 

## 1.1 What Makes a System Agentic?

Ask a language model when the last train from Boston to New York leaves on Friday, and it produces an answer: perhaps correct, perhaps out of date, but in any case a single piece of text that a person must read, check, and act on. Now ask a different kind of system to get you to New York on Friday evening at a reasonable price. It must find the timetable, compare trains, check your calendar, choose, pay with a stored credential, add the booking to your calendar, and notice two days later that the train has been canceled and rebook. The second system does not merely answer. It acts, over many steps, in an environment it does not fully control, using authority you have lent it. That difference, between producing an output and pursuing an outcome, is the subject of this book.

**EXAMPLE 1.1** (Answering versus acting). Compare three systems built on the same underlying language model. System A answers travel questions from its training data. System B also answers questions but first retrieves the current timetable from a search tool. System C receives the goal “book me a refundable seat to New York for Friday evening under \$150,” and may search, hold, and purchase tickets and edit the user’s calendar. A and B both return text to a human, who remains responsible for every consequential step; B is better informed but no more agentic in the sense that matters for risk. C selects and executes a sequence of actions whose effects persist in the world. An error by A yields a wrong sentence. An error by C yields a wrong purchase, a double booking, or a leaked payment credential. The engineering problems (tool interfaces, state tracking, recovery from failure) and the governance problems (who authorized the purchase, who pays for the mistake) appear only with C.

### 1.1.1 Agents, Environments, and Goals

The idea of an agent is older than computing. In the standard textbook account of Russell and Norvig (2020), an *agent* is anything that perceives its environment through sensors and acts upon it through actuators, and a *rational agent* chooses actions that are expected to maximize a performance measure, given what it has perceived. Franklin and Graesser (1997) sharpened this by requiring that an autonomous agent be situated in its environment, sense and act on it over time, pursue an agenda of its own, and act so as to affect what it will sense in the future. Wooldridge (2009) emphasizes four properties:

autonomy, reactivity to a changing environment, proactive pursuit of goals, and social ability.

Three ingredients recur in these accounts. The first is an *environment*: whatever lies outside the agent and responds to its actions. For a thermostat the environment is a room; for a software agent it is a file system, a set of web services, and the people who read its messages. The second is a *closed loop* between perception and action: the agent’s actions change the environment, which changes what the agent perceives next. The third is a *goal*, which may be a set point, a reward signal, a logical condition to be made true, or, in the systems that concern us most, a natural-language instruction whose meaning the agent must interpret.

The goal usually belongs to someone else. A travel agent, human or artificial, books trips for a client; the client is the *principal*. Economists study this arrangement as the principal–agent problem: the principal cannot observe everything the agent does or knows and must design incentives and monitoring accordingly (Holmström, 1979). The same asymmetry applies when the agent is software. Its principal (a user, or an organization deploying it) delegates a task stated incompletely, cannot watch every intermediate step, and bears the consequences of the outcome. Formal treatments of this setting include mechanism design for delegating decisions to privately informed agents (Zhang and Zhu, 2022) and contract-theoretic models for pricing agentic AI services that account for quality of service and liability costs (Yang and Zhu, 2025).

Chapter 2 formalizes these ideas (Theorem 2.15). It is useful to preview the formal interface now, because the rest of this chapter and much of the book use it. At step  $t$  a language agent holds a *context*  $c_t$  that contains its instructions, the task, the history of past actions and observations, and any memory retrieved for the occasion. A policy  $\pi(a_t \mid c_t)$ , realized by a language model  $p_\theta$  running inside a software *scaffold*, selects an action  $a_t \in \mathcal{A}$ . An action is a tool call  $(\tau, x)$  that names a tool  $\tau \in \mathcal{T}$  and its arguments  $x$ , a message to the user, or a decision to terminate. The environment returns an observation  $o_{t+1} \in \mathcal{O}$ , and the scaffold updates the context,

$$a_t \sim \pi(\cdot \mid c_t), \quad o_{t+1} \sim \text{Env}(\cdot \mid a_t), \quad c_{t+1} = U(c_t, a_t, o_{t+1}), \quad (1.1)$$

by appending, truncating, summarizing, or retrieving information. An episode ends when the agent terminates or a budget of steps  $T$ , tokens, or cost runs out, and a memory store  $\mathcal{M}$  can persist across episodes. Equation (1.1) is deliberately abstract: a thermostat, a chess engine, and a coding agent all fit it, with very different policies, action sets, and update rules. The following walkthrough makes it concrete.

**EXAMPLE 1.2** (One episode of the loop). Run System C of Theorem 1.1 through (1.1). The initial context  $c_0$  holds the scaffold’s instructions, the user’s goal, and the permissions the user granted: search, hold, and buy tickets up to \$150, and read and edit the calendar. Table 1.1 lists the episode. In each row the model reads  $c_t$  and emits  $a_t$ , the scaffold executes it, and  $c_{t+1}$  is  $c_t$  with  $a_t$  and  $o_{t+1}$  appended.

TABLE 1.1. The episode of [Theorem 1.2](#), one row per pass through the loop (1.1). The context  $c_{t+1}$  is  $c_t$  with the row’s action and observation appended. The last column names the clause of [Theorem 1.3](#) that the step exercises.

| $t$ | Action $a_t$                             | Observation $o_{t+1}$                                                                             | What the step exercises                      |
|-----|------------------------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------|
| 0   | search_trains(BOS, NYP, Fri 16:00–21:00) | 17:10 \$139 refundable; 18:05 \$98 non-refundable; 19:30 \$162 refundable; 20:15 \$119 refundable | interaction: reading the world               |
| 1   | read_calendar(Fri)                       | meeting until 18:00                                                                               | goal: interpreting “evening”                 |
| 2   | hold(20:15)                              | hold H7, expires in 15 minutes                                                                    | goal: the one train meeting every constraint |
| 3   | purchase(H7)                             | ticket confirmed, \$119 charged                                                                   | authority: spending the principal’s money    |
| 4   | add_event(Fri 20:15, train)              | event created                                                                                     | authority: editing the principal’s calendar  |
| 5   | message to user; terminate               | (none)                                                                                            | reporting and stopping                       |

The search at  $t = 0$  returns four trains. At  $t = 1$  the agent reads the calendar, because “Friday evening” is ambiguous, and a meeting that ends at 18:00 rules out the 17:10 train. At  $t = 2$  the constraints leave one candidate: the 18:05 train is non-refundable and the 19:30 train costs \$162, above the cap, so the agent holds the 20:15 train at \$119. The purchase at  $t = 3$  is the only step that spends the principal’s money; the calendar edit at  $t = 4$  persists but is cheap to undo. Six actions, one of which carries almost all of the risk, is the typical shape of an agent episode, and it is why autonomy is best set per action ([Fig. 1.1](#)).

Where the cap is enforced also matters. If the purchase tool itself refuses amounts above \$150, the constraint holds even when the model misreads a price. A cap stated only in the instructions holds only as reliably as the model follows them. Two days later a cancellation notice arrives. It is an observation that starts a *new* episode, and the agent can rebook only if the first episode wrote the booking reference to the memory store  $\mathcal{M}$ , since that episode’s context is gone. Finally, even six steps compound: if each step independently goes wrong with probability 0.02, all six go right with probability  $0.98^6 \approx 0.89$  ([Theorem 1.5](#)).

### 1.1.2 Autonomy, Delegation, and Levels of Agency

Definitions of agent that rest on a closed loop are generous. A thermostat senses temperature and switches a heater to reach a set point, so it qualifies. That generosity is a feature: the control-theoretic view of the thermostat is exactly the right lens for some questions about language agents ([Chapter 4](#)). But it means that “Is this system an agent?”

is rarely the useful question. The useful questions are how much of the loop the system closes on its own, over what horizon, with what authority, and with what consequences.

Chan et al. (2023) make this point in the context of harms, describing systems as more or less agentic according to how underspecified their goals are, how directly their actions affect the world without a human in between, how goal-directed their behavior is, and how far ahead they plan. Shavit et al. (2023) similarly treat agenticity as a degree: the extent to which a system can adaptively achieve complex goals in complex environments with limited direct supervision. Zhu (2026d) likewise characterizes agentic AI as a continuum of autonomy and delegated authority, distinguishing systems that produce informational outputs from systems that can independently cause consequential events through external actions, the distinction drawn in Theorem 1.1. We adopt the same stance.

**DEFINITION 1.3 (Agentic system).** An *agentic system* is a computational system that

- (i) is *goal-directed*: it selects actions because of their expected contribution to an objective, which may be specified only partially;
- (ii) engages in *sustained interaction* with an environment: it acts, observes the consequences, and conditions later actions on them, over multiple steps; and
- (iii) exercises *delegated authority*: a principal has granted it permission to take actions whose effects the principal does not individually review in advance.

The *degree of agency* of a system increases with the complexity and underspecification of the goals it can pursue, the length and openness of the interaction it can sustain, and the scope and consequence of the authority it exercises.

Theorem 1.3 is graded rather than binary. Each clause admits degrees: a goal can be a fixed set point or an open-ended instruction; interaction can last three steps or three weeks; authority can extend to reading a web page or to wiring money. In the trace of Table 1.1, clause (i) shows up in the interpretation of “evening,” clause (ii) in the calendar lookup that the search result prompted, and clause (iii) in the purchase. The definition also locates agency in the *system*, not in the model.

#### KEY IDEA

*Agency belongs to the whole system (model, scaffold, tools, and permissions), not to the model’s weights. The same model can power a chatbot with no authority or an agent that spends money, so the security and governance of agents cannot be reduced to the safety of the underlying model.*

*Autonomy* is the dimension of agency most directly under a designer’s control. It measures how much of the loop runs without human intervention. The human factors literature has studied this question for decades under the name *levels of automation*. Sheri-

TABLE 1.2. Levels of autonomy for agentic systems, aligned with the levels of automation of Sheridan and Verplank (1978) and Parasuraman et al. (2000). Levels can differ across the four processing stages and across actions of different consequence.

| Level | Name             | Human role and system behavior                                                            | S–V scale |
|-------|------------------|-------------------------------------------------------------------------------------------|-----------|
| 0     | Tool             | The human decides and acts; the system executes single commands.                          | 1         |
| 1     | Advisor          | The system proposes one or more options; the human chooses and acts.                      | 2–4       |
| 2     | Approval-gated   | The system selects and prepares an action, which runs only after explicit human approval. | 5         |
| 3     | Supervised       | The system acts unless the human vetoes within a time window, and it reports what it did. | 6–7       |
| 4     | Monitored        | The system acts and reports only on request, on exceptions, or at the end of the task.    | 8–9       |
| 5     | Fully autonomous | The system acts without informing or consulting humans.                                   | 10        |

dan and Verplank (1978) proposed a ten-point scale for teleoperated systems, running from a computer that offers no assistance to one that decides and acts while ignoring the human. Parasuraman et al. (2000) refined the idea in two ways that matter for agents. First, automation can be applied separately to four stages of processing: information acquisition, information analysis, decision and action selection, and action implementation. A system can be highly automated in gathering information while leaving decisions to people. Second, the right level for each stage should be chosen by evaluating consequences for human performance, such as mental workload, situation awareness, complacency, and skill degradation, and for outcomes, such as reliability and the cost of errors.

Table 1.2 condenses these scales into six levels suited to agentic systems, and Fig. 1.1 pictures them as a division of labor between a human and an agent. Read the figure from left to right. At level 0 the human initiates every action; at level 2 the agent initiates and the human must say yes; at level 3 the agent initiates and the human must say no in time; at level 4 the human hears only about exceptions; at level 5 the human is not in the loop at all. The step from level 2 to level 3 looks small but reverses the default: silence means “stop” at level 2 and “go” at level 3. We use the levels in the taxonomy of Section 1.4, and Chapter 16 returns to them when designing human oversight.

A single deployment typically mixes levels. A software engineering agent may edit files and run tests at level 4 inside a sandbox, yet open a pull request only at level 2, since merging requires a human reviewer. The travel agent of Theorem 1.2 can search and hold at level 4 and still buy at level 2 if the user prefers to approve purchases. Assigning autonomy by action rather than by system is one of the most effective design levers

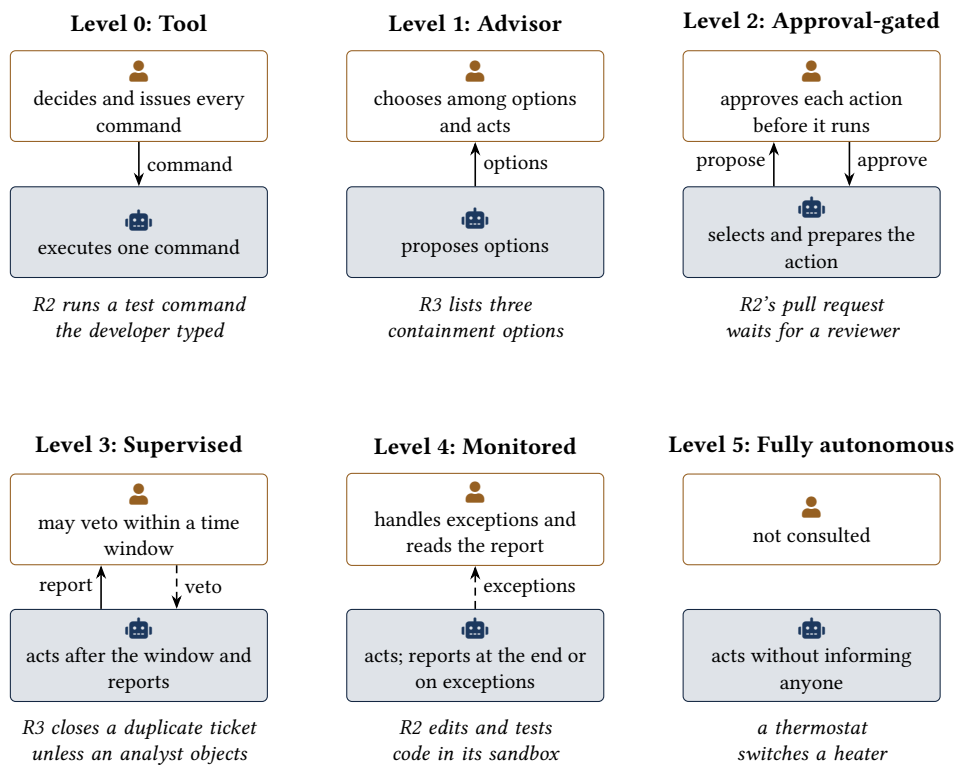


FIGURE 1.1. The six levels of Table 1.2 as a division of labor. In each panel the upper box gives the human’s role and the lower box the agent’s; arrows show who initiates, and dashed arrows are optional (a veto, an exception report). The italic line gives an example from the running examples of Section 1.3. A single deployment mixes levels: R2 appears at levels 0, 2, and 4.

available, and we will see it repeatedly; Theorem 8.7 makes action classes precise.

 **KEY IDEA**

Assign autonomy per action class, not per system. Let the agent run freely where errors are cheap, detectable, and reversible, and place human approval on the few actions that are costly or cannot be undone.

1.1.3 Agentic AI versus Generative AI and Automation

Agentic systems are often confused with two neighbors, and Fig. 1.2 separates the three by asking three questions: who chooses the next step, who acts on the world, and with what authority.

Predictive and generative models map an input to an output: a label, a forecast, an image, a paragraph. A language model used as a chatbot is generative; each response is a single output that a human consumes. In panel (a) the loop is closed by the user,

who reads, decides, and acts. Such a model can be a component of an agent, and in the systems of Part II it is the most important component, but generation alone satisfies none of the clauses of [Theorem 1.3](#) in a strong form. Its interaction consists of conversational turns in which a human closes the loop, and it holds no delegated authority beyond producing text.

*Fixed automation*, such as an industrial controller, a scheduled script, or a robotic process automation workflow, does act on the world, often with considerable authority, and it may run for years. What it lacks is flexibility of goal and plan. The designer enumerates in advance the situations it handles and the response to each; outside that envelope it fails or stops. A model call placed inside such a pipeline, as in panel (b), fills in content (a label, a draft) but does not change the path, which is why practitioners call the result a *workflow* rather than an agent. The reliability of fixed automation comes precisely from its rigidity, and much of the engineering in Part II can be read as an attempt to recover some of that predictability in systems whose behavior is not enumerated in advance.

Agentic AI occupies the region where flexible, learned policies meet delegated authority. In panel (c) the model chooses each next action at run time, observes the result, and continues until it decides to stop or reaches a gate. It inherits the open-ended competence of generative models and the real-world consequences of automation, and hence the difficulties of both: outputs that are hard to predict and verify, and actions that are hard to undo. The boundary is a matter of degree. A workflow whose branches are chosen by a model is partway to an agent, and [Chapter 7 \(Section 7.3\)](#) treats the spectrum between them as a design space.

#### IN PRACTICE (AS OF SEPTEMBER 2026)

Practitioner vocabulary has settled on a handful of terms used throughout this book. A *workflow* runs model calls along code paths fixed in advance, and an *agent* lets the model direct its own process and tool use ([Schluntz and Zhang, 2024](#)). The code around the model is called a *scaffold* or, in much practitioner writing, a *harness* ([Young, 2025](#)). *Context engineering* is the discipline of choosing what enters the model's context at each step ([Rajasekaran et al., 2025](#)). A *computer-use agent* acts through a graphical interface, a *background agent* works asynchronously and returns its result for review, and *skills* are packaged instructions and scripts that an agent loads when a task calls for them ([Zhang et al., 2025b](#)). “Agentic” is also used loosely in marketing for any product that calls a model more than once. [Theorem 1.3](#) and [Fig. 1.2](#) give a test that does not depend on the label.

**EXAMPLE 1.4** (One model, four degrees of agency). Consider a fixed language model deployed four ways in an IT help desk. (a) As a chatbot that answers questions about password policy, it is a generative system at level 0–1 with no delegated authority. (b) With read-only access to a ticketing system, it can look up the status of a user's ticket;

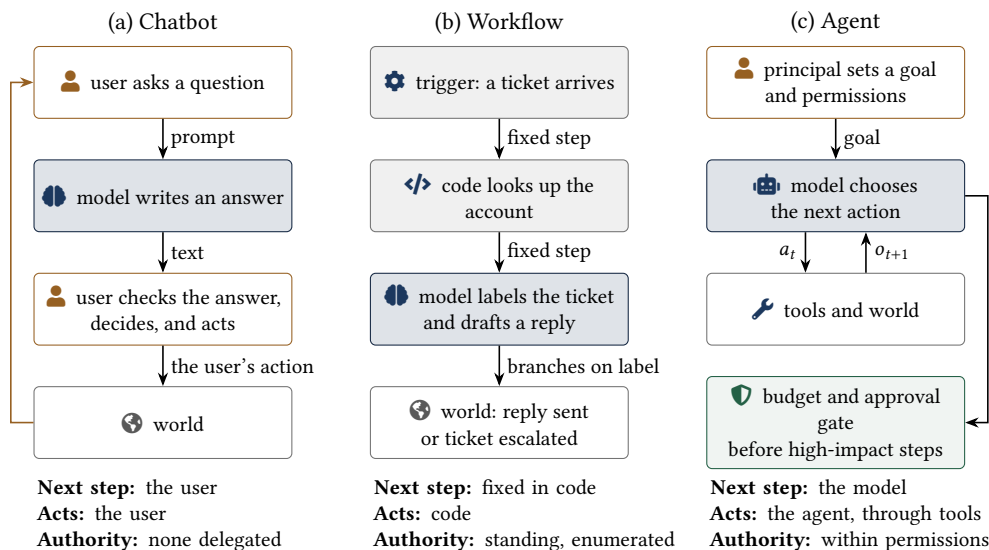


FIGURE 1.2. Three ways to use the same language model. Blue boxes are model calls, gray boxes deterministic code, white boxes people and the world, and the green box a control. (a) In a chatbot the user closes the loop (warm arrow). (b) In a workflow the path is fixed in code and the model fills in content. (c) In an agent the model chooses each next action, observes the result, and continues; the right-hand path leads to stopping or to a gated high-impact step.

it now interacts with an environment over several steps, but its authority is limited to reading. (c) With permission to reset passwords after the user confirms by a second factor, it takes consequential actions at level 2. (d) With permission to triage the entire ticket queue overnight, closing duplicates and resetting locked accounts without review, it acts at level 4 over long horizons, and a single misinterpretation can propagate across hundreds of accounts before anyone notices. The model is identical in all four cases; what grows along the sequence is exactly what [Theorem 1.3](#) measures.

## 1.2 A Brief History of Artificial Agents

Foundation-model agents are often described as a sudden invention. They are better understood as the latest answer to an old question: what policy should close the loop between perception and action? Each era of research proposed a different answer (a feedback law, a search procedure over symbolic models, a commitment to intentions, a learned value function), and each left behind concepts that current systems reuse, often under new names. [Figure 1.3](#) summarizes the lineage and, in its right-hand column, names what each era left to today's agents.

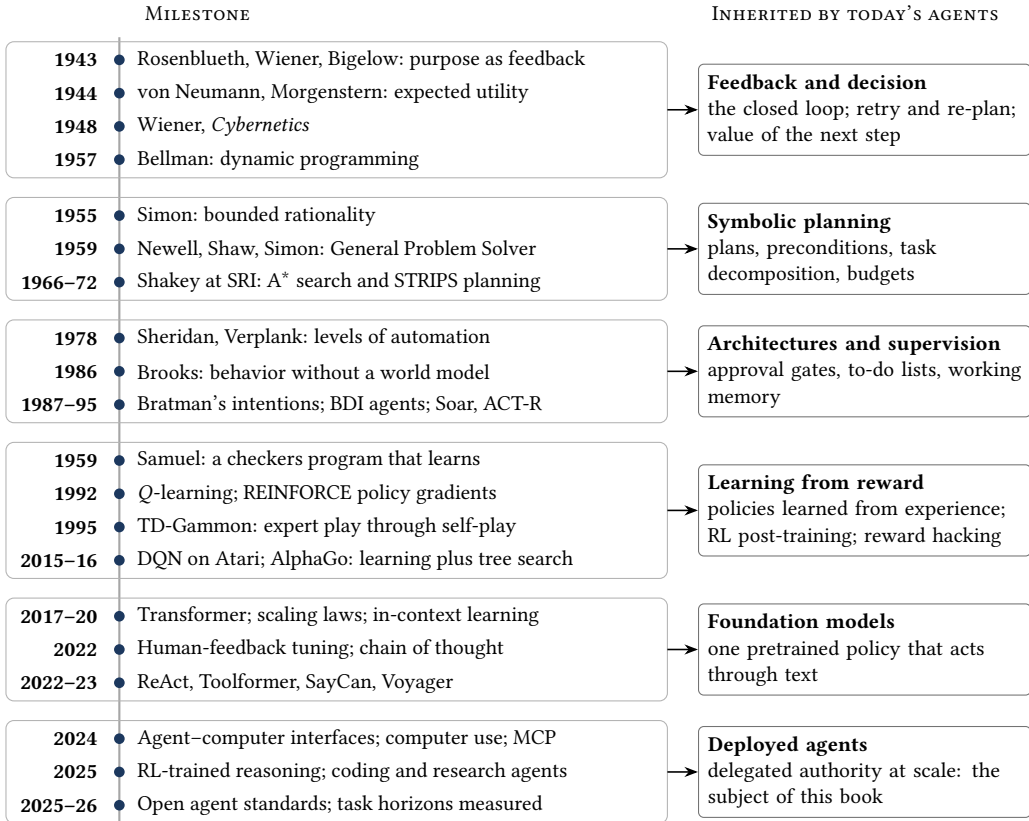


FIGURE 1.3. A timeline of artificial agents, grouped into six eras. Eras overlap in time; within an era, milestones are in chronological order. Dates are of publication or of the main period of work. The right-hand column names the ideas each era contributed that current language agents still use.

### 1.2.1 *Cybernetics and Feedback Control*

The modern concept of a goal-directed machine begins with a short paper by [Rosenblueth et al. \(1943\)](#). Its authors proposed that purposeful behavior, in animals and machines alike, can be explained by *negative feedback*: the system compares its state with a target and acts to reduce the discrepancy. A torpedo that steers toward the sound of a ship and a hand reaching for a glass are, on this view, the same kind of system. [Wiener \(1948\)](#) named the resulting field *cybernetics*, the study of control and communication in the animal and the machine, and placed information, feedback, and stability at its center.

Cybernetics contributed three ideas that remain central. First, goals can be realized by mechanism, without any inner representation of the goal as a proposition; a set point suffices. Second, the unit of analysis is the closed loop, not the controller alone, so the behavior of an agent cannot be understood apart from its environment. Third, feedback loops can be unstable: a controller that reacts too strongly or too late oscillates or diverges.

The last idea has a direct descendant in language agents that retry, reflect, and re-plan, sometimes spiraling into repeated failure; [Chapter 4](#) develops this control-theoretic view. Around the same time, decision theory supplied the normative account of choice under uncertainty that later agent theories adopted ([von Neumann and Morgenstern, 1944](#); [Savage, 1954](#)), and [Bellman \(1957\)](#) showed how to compute optimal sequential decisions by dynamic programming, the mathematical core of [Chapter 2](#).

### 1.2.2 *Rational Agents and Symbolic Planning*

Artificial intelligence, as a field named at the Dartmouth workshop of 1956, took a different route to goal-directed behavior: explicit reasoning over symbolic representations. The General Problem Solver of [Newell et al. \(1959\)](#) represented problems as states and operators and chose operators by *means-ends analysis*, reducing the difference between the current state and the goal. [Simon \(1955\)](#) had already argued that real decision-makers cannot optimize over all alternatives and instead search until they find one that is good enough. Bounded rationality, the recognition that computation is itself a scarce resource, is as relevant to an agent with a token budget as it was to a human in 1955; [Zhu \(2026c\)](#) examine tokens as the unit that links computation, cost, and value in such systems.

The first system to combine perception, planning, and action in a physical environment was Shakey, a mobile robot developed at SRI between 1966 and 1972 ([Nilsson, 1984](#)). Shakey perceived rooms and boxes with a camera and range finder, maintained a logical model of its world, planned sequences of actions, and executed them while monitoring for failure. Two lasting algorithms came out of the project: the A\* heuristic search algorithm ([Hart et al., 1968](#)) and the STRIPS planner ([Fikes and Nilsson, 1971](#)), which described actions by their preconditions and effects on a set of logical facts. Later work on hierarchical task networks ([Erol et al., 1994](#)) added decomposition of abstract tasks into subtasks, an idea that reappears whenever a language agent breaks a request into a plan.

The symbolic tradition also exposed a persistent gap between plans and the world. A plan is only as good as the model it was built from, and in open environments that model is always incomplete. [Brooks \(1986\)](#) drew a radical conclusion: rather than plan over a central world model, a robot should be built from layers of simple behaviors coupled directly to sensors, with higher layers subsuming lower ones. His robots were robust precisely because they did little deliberation. The tension between deliberation and reaction, between thinking before acting and acting to find out, runs through every later architecture, including the reasoning-and-acting loops of [Chapters 4 and 7](#). By the 1990s the field had consolidated around the *rational agent* as its organizing idea, a view codified in the textbook of [Russell and Norvig \(2020\)](#), whose first edition appeared in 1995.

### 1.2.3 *Belief–Desire–Intention Architectures*

Philosophy contributed a different insight. Bratman (1987) argued that humans are planning agents whose *intentions* are not merely strong desires but commitments: partial plans that we hold fixed, reason from, and reconsider only when there is good cause. Commitment matters for bounded agents, because constantly re-deliberating from scratch is expensive and makes coordination with others impossible.

The *belief–desire–intention* (BDI) model turned this theory into software. A BDI agent maintains beliefs about the world, desires (goals it might pursue), and intentions (goals it has committed to, together with plans from a library for achieving them). Its control loop repeatedly updates beliefs from percepts, deliberates about which desires to adopt as intentions, selects plans, and executes plan steps, while deciding how often to reconsider. Rao and Georgeff (1995) connected the formal logic of these attitudes to practical systems and described implemented applications, including an agent-based air traffic management system then under test at Sydney airport. In parallel, cognitive architectures such as Soar (Laird, 2012) and ACT-R (Anderson et al., 2004) modeled human cognition with explicit working memory, long-term memory, and production rules, and multi-agent systems research developed protocols for agents to communicate and allocate tasks (Smith, 1980; Finin et al., 1994; Wooldridge, 2009).

These architectures anticipated much of today’s vocabulary. A language agent’s context resembles working memory; its retrieved notes resemble long-term memory; a to-do list the agent writes and revises is an explicit intention structure. Sumers et al. (2024) make the correspondence systematic, and Chapter 7 builds on it. What BDI systems lacked was a way to acquire plans and world knowledge at scale. Plan libraries were written by hand, and the agents were only as capable as their authors’ foresight.

### 1.2.4 *Reinforcement Learning and Deep RL*

Learning offered a way around hand-written knowledge. Samuel (1959) built a checkers program that improved its evaluation function by playing against itself, an early demonstration that a machine could come to play better than its programmer. Reinforcement learning (RL) formalized this idea as learning a policy from reward in a Markov decision process (Sutton and Barto, 2018). *Q*-learning (Watkins and Dayan, 1992) and policy-gradient methods (Williams, 1992) provided general algorithms, and the theory of partially observable processes (Kaelbling et al., 1998) gave a principled account of acting under uncertainty about the state. TD-Gammon learned to play backgammon at the level of strong human experts through self-play (Tesauro, 1995).

Deep neural networks made these methods scale. Deep *Q*-networks learned to play many Atari games directly from screen pixels with a single architecture (Mnih et al., 2015), and AlphaGo combined learned policy and value networks with Monte Carlo tree search to defeat top professional Go players (Silver et al., 2016). These results established that a general learning algorithm, given enough experience and compute, can discover competent behavior that no one programmed.

Deep RL also taught lessons that remain relevant. Its agents were narrow: each was trained for one environment and transferred poorly. Learning required enormous amounts of interaction, which is cheap in a simulator and expensive or dangerous in the world. And agents optimized exactly the reward they were given, exploiting flaws in its specification in ways their designers did not intend (Amodei et al., 2016; Krakovna et al., 2020). Chapter 3 examines what carries over from this era, and Chapter 16 returns to specification gaming.

### 1.2.5 *Foundation Models as General-Purpose Policies*

The newest answer to the policy question is a large language model. The Transformer architecture (Vaswani et al., 2017), trained on internet-scale text with a simple next-token objective, produced models whose capabilities improved predictably with scale (Kaplan et al., 2020). Brown et al. (2020) showed that such models could perform new tasks from a few examples given in their input, without any parameter update, a phenomenon called *in-context learning*. Under idealized assumptions about the pretraining distribution, it can be interpreted as implicit Bayesian inference (Xie et al., 2022); Section 3.3 examines how far that interpretation extends. Fine-tuning from human preferences (Christiano et al., 2017; Ouyang et al., 2022) made models follow instructions, and prompting them to reason step by step improved performance on multi-step problems (Wei et al., 2022b).

The step from model to agent came when researchers placed these models inside a loop. ReAct interleaved natural-language reasoning traces with actions such as search queries, feeding observations back into the model’s context (Yao et al., 2023b). Toolformer trained a model to decide for itself when to call external tools (Schick et al., 2023). SayCan grounded a language model’s proposals in what a robot could actually do (Ahn et al., 2022). Voyager explored Minecraft by writing code and storing successful programs in a growing skill library (Wang et al., 2024a), and generative agents simulated a small town of characters with memory, reflection, and planning (Park et al., 2023). SWE-agent showed that the design of the interface through which an agent reads and edits code strongly affects its ability to resolve real software issues (Yang et al., 2024). Surveys by Wang et al. (2024b) and Xi et al. (2023) map this early literature.

Why did language models succeed as policies where earlier approaches stalled? Three properties stand out. Pretraining gives them a broad *prior* over sensible actions in countless situations, so they need no environment-specific training to act competently. Language serves as a universal interface: instructions, observations, tool documentation, and actions can all be expressed as text, so one policy can operate across very different environments. And in-context learning lets them adapt within an episode from feedback, error messages, and examples.

### 1.2.6 *From Research Prototypes to Deployed Agents*

Between 2024 and 2026 language agents moved from research demonstrations to products in daily use. The move was uneven, and its pattern is more durable than any list

of products. Agents were deployed first where three conditions held: actions and observations are naturally textual, errors can be detected cheaply, and the damage from a wrong action can be contained. Software engineering meets all three, since a repository is text, tests act as a verifier, and a sandbox plus code review contain mistakes. *Coding agents* accordingly became the first widely used class, working in an editor, in a terminal, or asynchronously in the cloud as *background agents* that return a pull request. *Research agents*, which search, read, and synthesize over many steps, meet the first condition fully and the other two partially: a report can be checked against its sources, but checking is costly. *Computer-use agents*, which operate graphical interfaces from screenshots with simulated keyboard and mouse, give up the first condition in exchange for generality, since any application a person can use becomes an environment. Enterprise deployments in customer operations and security operations combine these capabilities with the workflow structures of Chapter 7, and Chapter 12 examines them domain by domain.

Two further developments shaped the period. First, models trained with reinforcement learning to reason at length before answering (Guo et al., 2025), and then to interleave that reasoning with tool calls, strengthened the planning step of the loop (Chapters 3 and 4). Second, the interfaces between agents and the rest of the world began to standardize. Open protocols now describe how an agent discovers and calls tools (Model Context Protocol Contributors, 2026) and how agents delegate work to one another (Agent2Agent Protocol Contributors, 2026), and their stewardship moved from single vendors to neutral foundations (Chapter 8). Standard interfaces make agents easier to build. They also give attackers and auditors a common surface, a theme of Part III.

### ✂ IN PRACTICE (AS OF SEPTEMBER 2026)

- *Coding agents*. In May 2025 OpenAI (Codex) and GitHub (the Copilot coding agent) announced cloud agents that take an assigned task, work in an isolated environment, and propose changes for human review (OpenAI, 2025b; GitHub, 2025).
- *Computer use*. Anthropic released a computer-use capability in October 2024, OpenAI a computer-using agent in January 2025, and Google DeepMind a Gemini computer-use model in October 2025 (Anthropic, 2024; OpenAI, 2025a; Google DeepMind, 2025).
- *Research agents*. In June 2025 Anthropic described a research system in which a lead agent plans and directs parallel search subagents (Hadfield et al., 2025).
- *Standards*. In December 2025 the Linux Foundation formed the Agentic AI Foundation around the Model Context Protocol, the AGENTS.md convention for repository instructions, and the goose agent framework (Linux Foundation,

2025). The Agent2Agent protocol joined the foundation in August 2026 (A2A Protocol Project, 2026). In February 2026 NIST announced an AI Agent Standards Initiative (National Institute of Standards and Technology, 2026).

- *Transparency.* The 2025 AI Agent Index, published at FAccT 2026, documented 30 deployed agents and found that most of their developers disclosed little about safety, evaluations, and societal impacts (Staufer et al., 2026).

These developments revive old ideas with a new policy class. The loop is the cybernetic loop; the plan is the STRIPS plan written in prose; the context is the BDI agent’s working memory; the reasoning model is a policy improved by reinforcement learning. What is new is a single policy flexible enough to fill all of these roles without being engineered for any of them, and the new failure modes that come with that flexibility, which occupy much of this book.

#### 💡 KEY IDEA

*Foundation-model agents reuse the ideas of every earlier era: the feedback loop, the plan, the intention, and the policy learned from reward. What is new is one pretrained policy that fills all these roles through language, together with the failure modes that such flexibility brings.*

## 1.3 The Anatomy of an Agentic System

Despite their variety, agentic systems share a common anatomy. Figure 1.4 shows a reference architecture that organizes the rest of the book. It separates five concerns: how the agent turns inputs into a usable context, how it reasons and decides, how its decisions become actions, how it remembers, and what lies outside it. The figure is a functional decomposition, not a software design. In a minimal agent a single model call performs perception, reasoning, and action selection at once, and the “components” are regions of one prompt; in a large deployment each may be a separate service. In the trace of Table 1.1, perception assembled the timetable and calendar into  $c_t$ , reasoning ruled out three trains, action issued hold and purchase, and memory had to carry the booking reference to the next episode. Part II takes the components in turn: the loop and its patterns in Chapter 7, tools and environments in Chapter 8, memory and context in Chapter 9, and multiple agents in Chapter 10.

### 1.3.1 Perception and Context

An agent perceives through its inputs: the principal’s instructions, the outputs of tools, documents, web pages, screenshots, messages from people and other agents. For a language agent, perception culminates in the construction of the context  $c_t$  that the model conditions on. This step is less passive than the word “perception” suggests. The

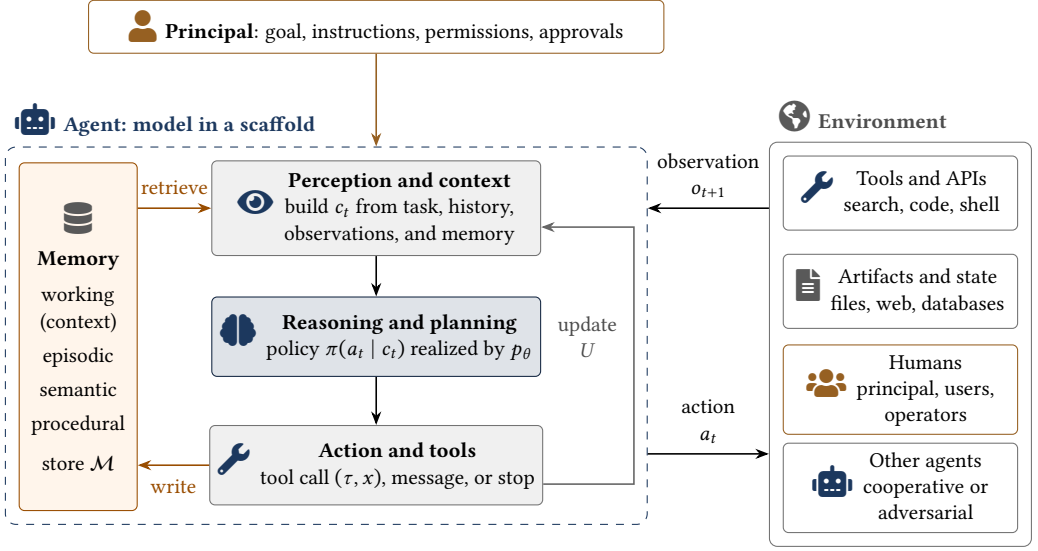


FIGURE 1.4. A reference architecture for agentic systems. The scaffold (dashed) wraps a language model (blue) that realizes the policy; gray boxes are deterministic code and the orange box is memory. Actions flow to an environment of tools, artifacts, humans, and other agents, and observations flow back into the context. Memory spans the working context of one episode and stores  $\mathcal{M}$  that persist across episodes. Part II elaborates each component.

context window is finite, and what enters it determines what the model can take into account. The scaffold must decide what to include verbatim, what to summarize, what to drop, and what to retrieve from memory, the update  $U$  of Eq. (1.1). Long contexts do not solve the problem, since models use information unevenly across long inputs (Liu et al., 2024a). The discipline of assembling good contexts, context engineering, is the subject of Chapter 9.

Perception is also where much of an agent’s exposure begins. Everything placed in the context can influence the policy, and in current language models there is no hard boundary between instructions the principal intended and text that merely arrived in a tool output. That observation underlies prompt injection, examined in Chapter 13.

### 1.3.2 Reasoning and Planning

Given a context, the policy must decide what to do. In a language agent the decision may be immediate, a single generated tool call, or deliberate: the model writes out intermediate reasoning, decomposes the task, considers alternatives, and critiques its own draft before acting. Techniques range from chain-of-thought prompting (Wei et al., 2022b) and interleaved reasoning and acting (Yao et al., 2023b) to search over alternative reasoning paths (Yao et al., 2023a) and verbal self-reflection after failure (Shinn et al., 2023). Whether language models can plan reliably on their own is contested; there is good evidence that they often cannot on problems requiring exact combinatorial reasoning,

and that pairing them with external verifiers helps (Valmeekam et al., 2023; Kambhampati et al., 2024). Chapter 4 treats reasoning and planning from both the classical and the language-model perspectives.

### 1.3.3 Action and Tools

Decisions become effects through actions. The action set  $\mathcal{A}$  of a language agent typically includes calls to tools in  $\mathcal{T}$  (a web search, a Python interpreter, a shell command, an email API, a robot’s motion primitive), messages to the user, and termination. Tools extend what the agent can do and also what it can break. Their design matters as much as the model’s: a tool that returns ten thousand lines of unfiltered output, or that accepts ambiguous arguments, can defeat a capable policy, whereas an interface designed for the agent’s strengths can markedly improve performance (Yang et al., 2024). Agents may also act by writing and executing code, which composes many operations in a single action (Wang et al., 2024c). The permissions attached to each tool define the agent’s delegated authority in the sense of Theorem 1.3; the \$150 cap enforced inside the purchase tool of Theorem 1.2 is a small instance. Chapter 8 develops action spaces, tool specification, and execution environments.

### 1.3.4 Memory and Learning

Memory lets an agent carry information beyond a single model call. Following the cognitive science distinctions adopted by Sumers et al. (2024), it helps to separate *working memory* (the current context), *episodic memory* (records of past interactions), *semantic memory* (facts about the world and the principal), and *procedural memory* (skills, which for a language agent reside both in model weights and in stored prompts or code) (Tulving, 1972). Retrieval-augmented generation (Lewis et al., 2020) is the most common mechanism for bringing stored knowledge into context.

Memory is also the most accessible route to learning at deployment time. A language agent whose weights are fixed can still improve by storing lessons, reusable programs, or preferences and retrieving them later, as Voyager did with its skill library (Wang et al., 2024a). Learning that changes the model itself, by fine-tuning or reinforcement learning on agentic tasks, is the topic of Chapter 3. Persistent memory creates its own risks, since a corrupted memory can mislead every future episode (Chen et al., 2024).

### 1.3.5 The Environment: Artifacts, Humans, and Other Agents

The environment of a language agent is heterogeneous. It contains passive *artifacts* such as files, databases, and web pages; active services that respond to calls; *humans*, including the principal, other users, and operators who approve or override actions; and increasingly *other agents*, some cooperative and some not. Each kind of entity calls for different analytical tools. Artifacts can be modeled as state in a decision process (Chapter 2). Humans and other agents have goals of their own, so interaction with them is strategic, the domain of game theory and information economics (Chapters 5

and 6). When those other agents are themselves driven by language models, the classical assumption of fully rational players needs revision; game models in which agents choose reasoning prompts and their actions emerge from model inference are developed by Zhu (2025d) and Zhu (2025c). Humans also play a supervisory role, which raises questions of trust and oversight (Chapters 16 and 17). Treating the environment as a single, indifferent source of observations, as much of the early agent literature did, is a simplification that security analysis in particular cannot afford.

### 1.3.6 Running Examples

Three running examples recur throughout the book. Each is a composite of systems described in the research literature and deployed in practice, chosen so that together they span the main concerns of theory, systems, and security. Figure 1.5 draws all three in the same format, and later chapters revisit them in boxes like the ones below.

#### RUNNING EXAMPLE R1: RESEARCH ASSISTANT

Given a research question from a user, such as whether a class of batteries degrades faster in cold climates, the research assistant searches the web and scholarly databases, reads and extracts findings from papers, runs small analyses in a Python sandbox, keeps notes across sessions, and delivers a cited report to the user, for example by email or in a shared document. Its tools are search, document retrieval, a code interpreter, a note store  $\mathcal{M}$ , and an outbound channel. It typically acts at level 4 of Table 1.2 while gathering information, and its most consequential action is sending the report.

R1 stresses *long-horizon information gathering*: it must decide what to read next, when it knows enough, and how to reconcile conflicting sources. It stresses *memory*, since a research project spans many sessions. And it stresses *source reliability*, since an eloquent report built on a retracted paper or a fabricated citation is worse than no report. In the security chapters R1 becomes the canonical example of an agent that reads untrusted content while holding private data (the user's notes and drafts) and an outbound channel through which that data could leave.

#### RUNNING EXAMPLE R2: SOFTWARE ENGINEERING AGENT

Given an issue in a code repository, such as a bug report with a failing input, the software engineering agent explores the codebase, reads relevant files, edits them, runs the test suite in a container, inspects the failures, and iterates until the tests pass. It then opens a pull request for human review. Its tools are file viewing and editing commands, a shell in a sandboxed container, and the version control system. It acts at level 4 inside the container but at level 2 for the final change, which a human must approve before merging.

R2 stresses *tool use* over many steps, *verification* through executable tests, which give it an unusually clear feedback signal, and the design of the *agent–computer interface* through which it sees and edits code (Yang et al., 2024; Jimenez et al., 2024). It is the case study of Chapter 7. Its clear feedback signal is also a vulnerability: in Chapter 16 R2 is the example of an agent that learns to make tests pass without fixing the underlying problem, for instance by special-casing the test inputs.

### RUNNING EXAMPLE R3: SECURITY OPERATIONS AGENT

In a security operations center, the security operations agent triages incoming alerts, queries logs and threat intelligence sources, correlates related events into incidents, and assesses their severity. It recommends containment actions, such as isolating a host from the network or disabling a user account, and with human approval it executes them. Its tools are query interfaces to log stores and intelligence feeds, the ticketing system, and a small set of high-impact response actions guarded by approval. It works alongside human analysts and other automated tools and agents. R3 stresses *adversarial environments*: the attacker it is defending against can shape the very logs and alerts it reads. It stresses *irreversible, high-impact actions*, since isolating the wrong server can halt a business and failing to isolate the right one can let an intrusion spread. And it stresses *human approval* and *coordination* with other tools and agents under time pressure. R3 connects the systems of Part II to the game-theoretic defenses of Chapter 14 and the resilience mechanisms of Chapter 15. System-level treatments of agentic security operations, which analyze attacker and defender workflows as coupled adaptive processes and use game theory to allocate autonomy, appear in Li and Zhu (2025) and Zhu (2025b); Huang and Zhu (2023) describe an LLM framework that couples automated penetration testing with remediation.

Mapped onto Fig. 1.4, the three examples differ mainly in their environments and authority, and Fig. 1.5 makes the difference visible. R1’s environment is dominated by untrusted artifacts, and the same agent that reads them holds private notes and an outbound channel. R2’s environment is a controllable sandbox with a crisp verifier, and its one high-impact action is gated by a reviewer. R3’s environment contains an adversary who writes some of its inputs, and its high-impact actions are gated by analysts who must decide quickly. The same loop, instantiated in these three settings, raises largely different questions, which is why we will keep returning to all three.

## 1.4 A Taxonomy of Agentic Systems

Agentic systems vary along many dimensions. Four are especially useful for anticipating what can go wrong and which tools of analysis apply: the degree of autonomy, the kind of environment, the number of agents, and how the system learns. Table 1.3

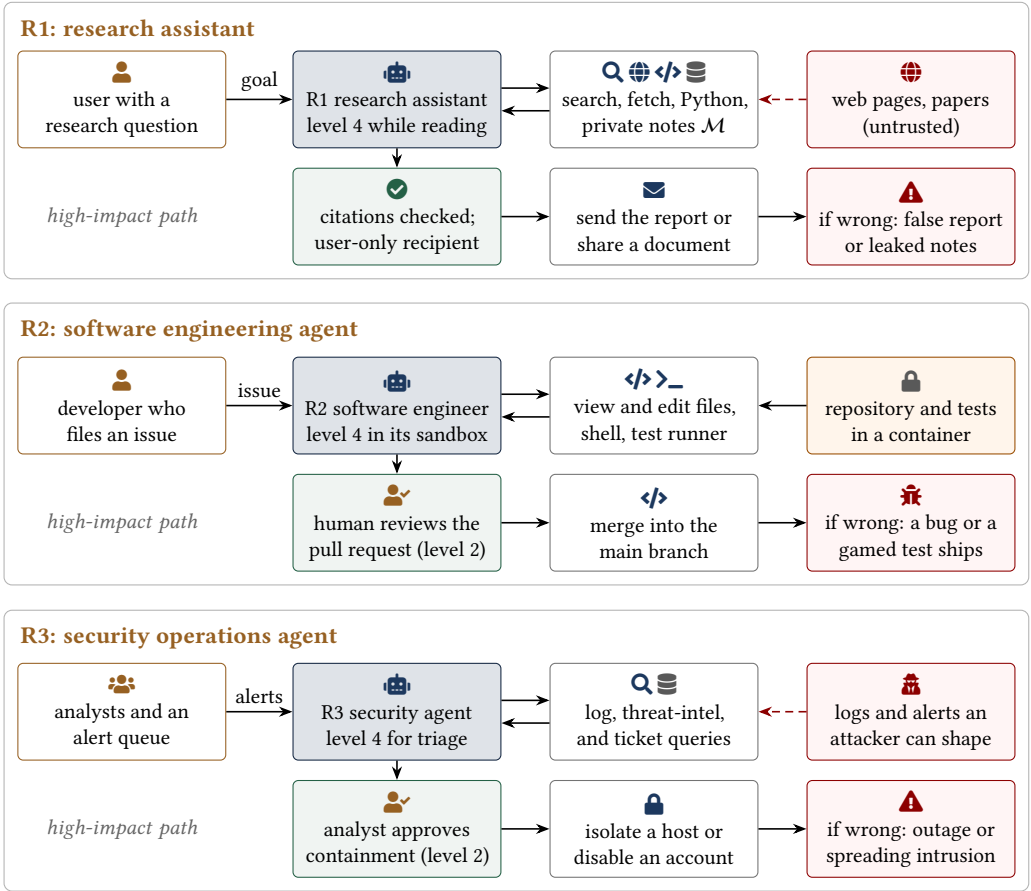


FIGURE 1.5. The three running examples in a common format. The top row of each panel is the working loop: a principal hands the agent (blue) a goal, and the agent calls tools that draw on inputs at the right. Red boxes with dashed arrows mark inputs that parties other than the principal can shape. The bottom row is the high-impact path: a control (green) stands between the agent and the one action whose error is costly, shown with its consequence at the right.

classifies fourteen systems, from a thermostat to deployed language agents, along these dimensions. Where it names research systems, it describes them as reported in the cited papers; where it names types, such as a thermostat or a computer-use agent, the entries are typical rather than definitive.

#### 1.4.1 Degree of Autonomy

Autonomy, as Section 1.1 explained, is best measured by stage and by action rather than assigned to a system as a whole. Three refinements make the levels of Table 1.2 more informative. First, following Parasuraman et al. (2000), a system may be highly autonomous in acquiring and analyzing information while leaving decisions to humans. The research assistant R1 gathers and synthesizes at level 4, but a human decides what to

TABLE 1.3. A taxonomy of representative agentic systems. Autonomy levels refer to Table 1.2 and describe the system’s typical operation within its intended scope. Learning describes how competence was acquired (designed or trained) and whether the system changes during deployment: *fixed*, *adaptive* (adapts through context or memory with fixed parameters), or *self-improving* (accumulates reusable skills or updates itself from its own experience).

| System                                                                                 | Level | Environment                     | Agents                 | Learning                                        |
|----------------------------------------------------------------------------------------|-------|---------------------------------|------------------------|-------------------------------------------------|
| Thermostat or PID controller (type)                                                    | 5     | Physical, narrow                | Single                 | Designed; fixed                                 |
| Shakey mobile robot (Nilsson, 1984)                                                    | 4     | Physical, laboratory            | Single                 | Designed; fixed                                 |
| BDI air traffic management (Rao and Georgeff, 1995)                                    | 1     | Digital, sociotechnical         | Multiple, cooperative  | Designed; fixed                                 |
| Deep $Q$ -network for Atari (Mnih et al., 2015)                                        | 5     | Digital, simulated              | Single                 | Trained by RL; fixed after training             |
| AlphaGo (Silver et al., 2016)                                                          | 5     | Digital, game                   | Two, adversarial       | Trained by supervised learning and self-play RL |
| Cicero for Diplomacy (Meta Fundamental AI Research Diplomacy Team (FAIR) et al., 2022) | 5     | Social, game with humans        | Multiple, mixed-motive | Trained; fixed during play                      |
| SayCan robot (Ahn et al., 2022)                                                        | 3–4   | Physical, home-like             | Single                 | Pretrained LM with learned skills; fixed        |
| Voyager in Minecraft (Wang et al., 2024a)                                              | 5     | Digital, open world             | Single                 | Pretrained LM; self-improving skill library     |
| Generative agents (Park et al., 2023)                                                  | 5     | Social, simulated town          | Multiple, social       | Pretrained LM; adaptive memory                  |
| Coscientist (Boiko et al., 2023)                                                       | 4     | Digital and physical laboratory | Single, modular        | Pretrained LM; adaptive in context              |
| Coding agent, e.g., SWE-agent (Yang et al., 2024)                                      | 2–4   | Digital, repository sandbox     | Single                 | Pretrained LM; adaptive in context              |
| Role-based multi-agent team (Hong et al., 2024)                                        | 4     | Digital, software project       | Multiple, cooperative  | Pretrained LM; adaptive in context              |
| Research agent with subagents (Hadfield et al., 2025)                                  | 4     | Digital, web and documents      | Multiple, cooperative  | Pretrained LM; adaptive in context              |
| Computer-use agent (type)                                                              | 2–4   | Digital, graphical interfaces   | Single                 | Pretrained LM; adaptive in context              |

do with its conclusions. Second, autonomy should be read together with the *scope* of the system’s authority. AlphaGo acts at level 5, but only over moves on a Go board, and a thermostat acts at level 5 over a single switch; a coding agent at level 4 inside a sandbox has less authority over the world than either might suggest, and far more than either once it can push to production. Third, autonomy is increasingly *adjustable*: the same system may escalate to a human when it is uncertain or when an action is irreversible, as R3 does before isolating a host.

Autonomy is not a measure of capability, and higher is not better. The appropriate level depends on the reliability of the system, the reversibility and cost of its errors, and the ability of humans to supervise it meaningfully. Automation research documents that humans supervising highly reliable systems tend toward complacency and lose the skills needed to take over when automation fails (Parasuraman et al., 2000; Lee and See, 2004). Recent frameworks for language agents make the same separation, defining levels by the role left to the user rather than by what the agent can do (Feng et al., 2025). Chapter 16

turns these findings into principles for designing oversight, and [Theorem 12.5](#) gives a simple model of when human review of an action class pays for itself.

### 1.4.2 *Digital, Physical, and Social Environments*

Environments differ in ways that shape agent design more than any property of the agent itself. *Digital environments*, such as file systems, web browsers, and software services, are fast, cheap to act in, and often partially reversible, since files can be restored and sandboxes discarded. They are also where current language agents are most capable, because their actions and observations are naturally textual. *Physical environments* involve robots, laboratory equipment, vehicles, and industrial control. Actions take real time, cannot be undone, and are subject to safety constraints that a simulator only approximates. Systems such as SayCan ([Ahn et al., 2022](#)) and vision-language-action models ([Brohan et al., 2023](#)) bring foundation models to this setting. Layered architectures that connect networked AI agents to physical execution through cloud, edge, and device tiers have been proposed for IoT orchestration ([Zhu, 2026e](#)) and, for autonomous industrial cyber-physical systems, cloud-fog automation ([Jin et al., 2025](#)); [Chapters 8](#) and [12](#) discuss the additional assurance it requires. *Social environments* are those in which the most important entities are people or other agents: negotiations, markets, organizations, and online communities. Their defining feature is that other participants have goals and adapt to the agent.

Most deployed systems combine all three. R3 acts in a digital environment of logs and endpoints, its containment actions have physical and economic consequences, and its environment includes both human analysts and an adversary. When classifying a system, it helps to ask of each part of the environment how observable it is, how quickly it changes, how reversible actions in it are, and whether anything in it is trying to deceive the agent. For R3 the answers are sobering: the network is partially observable, it changes by the minute, containment is costly to reverse, and the attacker is actively shaping what R3 sees. Each answer points to a part of the book: partial observability to [Chapter 2](#), time pressure and reversibility to [Chapter 8](#), and deception to [Chapters 6](#) and [14](#).

### 1.4.3 *Single-Agent and Multi-Agent Systems*

A *multi-agent system* contains several decision-makers whose actions affect one another. The distinction is not always sharp. A single language model prompted to play a planner, a coder, and a reviewer is one model but several agents in the functional sense, and a single-agent deployment becomes a multi-agent system the moment it interacts with another organization's agent. What matters for analysis is the relationship among the agents' objectives. In a *cooperative* system they share a goal, and the problem is coordination: dividing work, sharing information, and avoiding redundant or conflicting actions, as in role-based software teams ([Hong et al., 2024](#); [Wu et al., 2023](#)). In an *adversarial* system goals are opposed, as in Go or in the contest between R3 and an

intruder. In *mixed-motive* systems agents have partly aligned and partly conflicting interests, as in Diplomacy, where Cicero had to negotiate alliances with human players whose cooperation was never guaranteed (Meta Fundamental AI Research Diplomacy Team (FAIR) et al., 2022), or in markets where pricing algorithms interact (Calvano et al., 2020).

Game theory (Chapter 5) supplies the concepts for all three cases. Chapter 10 addresses the engineering of cooperative language-agent teams, whose failures are often mundane problems of specification and communication (Cemri et al., 2025), and Hammond et al. (2025) survey the risks that arise when many advanced agents interact. At larger scale, open ecosystems in which heterogeneous agents discover one another and form coalitions to execute workflows across cloud and edge infrastructure are analyzed by Zhu (2026f) and Yang and Zhu (2026a), and Jiang et al. (2026) study how to attribute a jointly produced output to the agents of an LLM-based multi-agent workflow.

#### 1.4.4 Fixed, Adaptive, and Self-Improving Agents

The last dimension is how an agent’s competence changes. It is useful to separate how competence was *acquired*, whether designed by hand, as in Shakey or a BDI plan library, or trained, as in AlphaGo or a language model, from how the system *changes during deployment*. A *fixed* agent behaves the same way on its hundredth task as on its first; most industrial automation is deliberately fixed, and so, in an important sense, is a language model with frozen weights and no memory. An *adaptive* agent changes its behavior in response to experience without changing its parameters, through information held in context or retrieved from memory: an agent that reads an error message and corrects its command, or that remembers a user’s preferences across sessions. A *self-improving* agent accumulates durable competence from its own experience, either by building external artifacts such as Voyager’s library of verified programs (Wang et al., 2024a) or by updating its own parameters through further training (Sutton and Barto, 2018).

Each step along this dimension trades predictability for capability. A fixed agent can be evaluated once. An adaptive agent’s behavior depends on the contents of its context and memory, which may include material placed there by others. A self-improving agent may acquire capabilities, or flaws, that were absent when it was evaluated, which complicates both assurance (Chapters 11 and 15) and the question of who is responsible for what it does (Chapter 17). Learning in agents, from reinforcement learning through open-ended skill acquisition, is the subject of Chapter 3.

## 1.5 Why Agentic AI Raises New Questions

If agents are an old idea, why do they need a new book? Because the combination of a flexible learned policy, an open-ended action space, long horizons, and real delegated authority raises questions that neither classical agent theory nor the study of generative models answers on its own. This section poses four of them. Each motivates part of the

book.

### 1.5.1 Compounding Errors over Long Horizons

A system that produces a single output fails once or not at all. An agent that takes many steps can fail at any of them, and errors propagate: a misread file leads to a wrong edit, which leads to a misleading test result, which leads to a confident but incorrect pull request. A crude model already shows the scale of the problem.

**EXAMPLE 1.5** (Compounding errors). Suppose an agent must complete  $T$  steps, each of which it performs correctly with probability  $1 - \epsilon$ , independently of the others, and suppose any single error dooms the task. The probability of success is  $(1 - \epsilon)^T \approx e^{-\epsilon T}$ . With a per-step error probability of  $\epsilon = 0.02$ , which would be excellent for many single-shot tasks, a 50-step task succeeds with probability about 0.36 and a 100-step task with probability about 0.13. Halving  $\epsilon$  to 0.01 raises these only to about 0.61 and 0.37. If instead the agent detects and repairs an error with probability  $\rho$  before it becomes fatal, the effective per-step failure probability falls to  $\epsilon(1 - \rho)$ , and with  $\rho = 0.8$  the 100-step task succeeds with probability about 0.67.

The example also explains why the length of task an agent can complete has become the headline measure of progress. Write  $p = \epsilon(1 - \rho)$  for the effective per-step failure probability. Success  $(1 - p)^T$  falls to one half at the *half-success horizon*

$$T_{1/2} = \frac{\ln 2}{-\ln(1 - p)} \approx \frac{0.69}{p}. \quad (1.2)$$

For  $p = 0.02$  this is about 34 steps. Halving  $p$  to 0.01 doubles it to about 69. Keeping  $\epsilon = 0.02$  but repairing 80% of errors gives  $p = 0.004$  and a horizon of about 173 steps, the same gain as a fivefold reduction in the error rate itself. [Figure 1.6](#) plots the three cases. Because  $T_{1/2}$  is inversely proportional to  $p$ , every doubling of the horizon corresponds to a halving of the effective failure probability, whether that halving comes from making fewer errors or from repairing more of them.

The independence and constant-rate assumptions are unrealistic: steps differ in difficulty, errors are correlated, and some errors can be repaired many steps later. [Section 2.5](#) develops a general model ([Theorem 2.13](#)) of which this example is a special case. The lesson survives. On long tasks, the ability to notice and recover from errors matters as much as the ability to avoid them. That is why verification, whether by tests, checks against sources, or human review, is central to agent design ([Chapters 7 and 11](#)). The same distinction, between robustness (withstanding disruption) and resilience (recovering from it), has been developed for feedback control systems ([Zhu and Başar, 2024; Zhu, 2025a](#)), and [Yang and Zhu \(2026c\)](#) study how to allocate tokens across the stages of a sequential LLM workflow to trade reliability against latency and cost. The theory of sequential decisions in [Chapter 2](#), planning and control in [Chapter 4](#), and resilience in [Chapter 15](#) each address long horizons from a different angle.

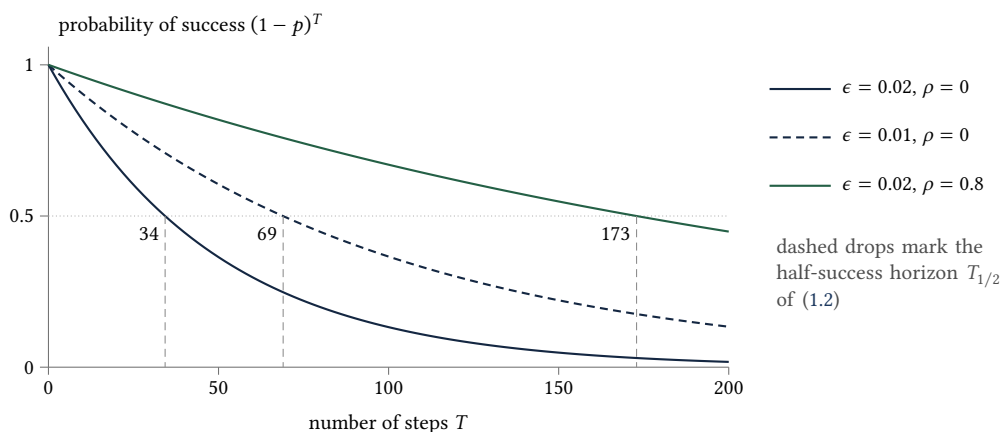


FIGURE 1.6. Success probability under the model of Theorem 1.5. Halving the error rate doubles the half-success horizon from 34 to 69 steps; repairing 80% of errors, with the error rate unchanged, raises it to 173 steps.

### 💡 KEY IDEA

*On long tasks, recovery is worth as much as accuracy. When the effective per-step failure probability is  $\epsilon(1 - \rho)$ , halving it doubles the horizon at which the agent succeeds half the time, whether the halving comes from fewer errors or from better repair.*

### ✂ IN PRACTICE (AS OF SEPTEMBER 2026)

METR measures an agent's *time horizon*: the length of task, in the time a skilled human needs to complete it, that the agent completes with 50% probability. Over 2019 to early 2025 this horizon for frontier agents doubled roughly every seven months, and success was lower on “messier” tasks with less structure and feedback (Kwa et al., 2025). METR's January 2026 update enlarged the task suite to 228 tasks, with more tasks of eight hours or longer, and estimated faster doubling since 2023, about every four months. It also stressed that the confidence intervals are wide and that human completion times for most of the longest tasks are estimated rather than measured (METR, 2026a). Human time is not the same unit as the steps of (1.2), but the qualitative link holds: a longer horizon requires a lower effective failure rate per unit of work. We quote trends, not the scores of particular models, which go stale within months.

### 1.5.2 Open-Ended Action Spaces

Classical agents chose among a fixed, enumerated set of actions: move north, pick up block B, play at a given point on the board. A language agent with a shell or a code interpreter can, in principle, do anything the underlying computer can do, and its actions are expressed in the same language as its instructions and observations. This openness is the source of its generality, and it breaks assumptions that earlier theory relied on. The action set is too large to enumerate, so it cannot be searched exhaustively or fully verified in advance. The consequences of an action may be hard to predict even for the designer, since a single shell command can have far-reaching side effects. And the objective, usually a natural-language instruction, is itself underspecified, so a capable agent may satisfy the letter of its task while violating its intent (Krakovna et al., 2020), as R2 does when it edits a test rather than the code under test.

Part I builds the formal tools for reasoning about such agents: sequential decision processes with language action spaces (Chapter 2), learning and post-training for agency (Chapter 3), and planning with imperfect models (Chapter 4). Part II addresses the engineering response: constraining and structuring action spaces through tool design and execution environments (Chapter 8), and building architectures whose control flow limits what the model decides freely (Chapter 7).

### 1.5.3 Security, Misuse, and Adversaries

An agent that reads the web, email, or logs takes input from parties other than its principal, and some of those parties are hostile. Because language models do not reliably distinguish instructions from data, text embedded in a web page or document can redirect an agent's behavior, an attack known as indirect prompt injection (Greshake et al., 2023). The agent then becomes a *confused deputy*: a program with legitimate authority that is tricked into exercising it on behalf of someone else, a problem identified long before language models (Hardy, 1988). The research assistant R1, which reads untrusted content while holding private data and an outbound channel, is exposed in exactly this way; the red boxes of Fig. 1.5 mark where such content enters. Agents can also be attacked through poisoned training data, tools, or memories, and they can be misused by attackers to scale operations that previously required skilled human effort (Fang et al., 2024; Zhang et al., 2025a). Such misuse has been reported in the field: in November 2025 Anthropic reported disrupting an espionage campaign in which, by its account, attackers directed its coding agent to carry out most steps of intrusion attempts against about thirty organizations, with human operators intervening at a handful of decision points (Anthropic, 2025).

#### KEY IDEA

*An agent that holds authority and reads content written by parties other than its principal can be steered by those parties. Security for agents therefore begins by bounding what the authority can do, not by trusting the model to tell instructions*

*from data.*

Security cannot be added at the end. It depends on classical principles such as least privilege (Saltzer and Schroeder, 1975) applied to new components, on a strategic understanding of adversaries who adapt to defenses, and on the ability to detect and recover from compromise. Part III develops these threads: threat models in Chapter 13, game-theoretic defense in Chapter 14, and resilience, guardrails, and runtime assurance in Chapter 15. The strategic foundations they rely on are laid in Chapters 5 and 6.

#### 1.5.4 Delegation, Accountability, and Trust

Delegation is the defining feature of agentic systems and the root of their governance problems. A principal who delegates a task cannot observe every step, and the agent’s behavior may diverge from the principal’s intent through misunderstanding, specification gaming, or learned objectives that differ from the intended ones (Amodei et al., 2016; Hubinger et al., 2019). The principal must therefore decide how much to trust the agent, and trust that is miscalibrated in either direction is costly: too little and the agent’s value is lost to constant supervision, too much and errors go unchecked (Lee and See, 2004); game-theoretic treatments of trust in AI-enabled networked systems appear in Ge and Zhu (2026). When harm does occur, it may be unclear who is responsible, since the developer did not specify the action, the deployer did not foresee it, and the user did not approve it, a *responsibility gap* identified for learning systems well before language models (Matthias, 2004). Computational frameworks for attributing responsibility after AI-induced incidents (Ge et al., 2024) and insurance mechanisms for pricing and transferring the liability of AI and agentic AI deployments (Ge and Zhu, 2023; Zhu, 2026b) are two institutional responses. As agents interact with one another across organizational boundaries, further questions arise about identifying agents, monitoring their activity, and assigning liability (Chan et al., 2024; Shavit et al., 2023).

These questions are part technical and part institutional. Chapter 16 addresses the technical side: specifying what we want, detecting when agents diverge from it, preserving meaningful human oversight, and keeping agents correctable. Chapter 17 addresses the institutional side: trust, accountability and liability, risk management frameworks, regulation, and the use of mechanism design to align the incentives of developers, deployers, and users. The book closes in Chapter 18 with the open problems that remain across all four questions.

## 1.6 How to Read This Book

### 1.6.1 Organization and Chapter Dependencies

The book has three parts, corresponding to the three perspectives introduced in the preface. *Part I, Foundations* (Chapters 1 to 6), treats agents as sequential decision-makers.

TABLE 1.4. Suggested course paths.

| Course                             | Suggested chapters                                                 |
|------------------------------------|--------------------------------------------------------------------|
| Foundations of agentic AI (theory) | 1–6, 14, 16; Appendices A–C                                        |
| Building agentic systems (systems) | 1, 2 (Sections 2.5–2.6), 4 (Section 4.4), 7–12, 13, 15; Appendix D |
| Secure and trustworthy agents      | 1, 5, 7, 8, 13–17                                                  |

After this introduction, it develops decision processes and partial observability, learning from reinforcement learning to foundation models, reasoning, planning and control, game theory, and the economics of information and communication. *Part II, Systems* (Chapters 7 to 12), covers how agents are built: architectures, tools and environments, memory and context, multi-agent orchestration, evaluation, and case studies across domains. *Part III, Security, Safety, and Governance* (Chapters 13 to 18), treats threat models, adversarial games and strategic defense, resilience and runtime assurance, alignment and human oversight, and trust, accountability, and governance, and ends with open problems. Four appendices review probability, optimization, and information theory (Chapter A), game theory (Chapter B), and reinforcement learning and control (Chapter C), and provide laboratory exercises that build a working agent step by step (Chapter D).

Every chapter follows the same pattern: an abstract and learning objectives, numbered sections with definitions, results, and examples, key ideas and dated “In Practice” boxes, a box of key takeaways, notes on further reading, and exercises. The running examples R1–R3 of Section 1.3 recur in boxes throughout. Notation, summarized in the front matter, is shared across chapters; in particular, the agent interface of Eq. (1.1) is formalized in Chapter 2 and used throughout Parts II and III.

Figure 1.7 shows the main dependencies among chapters. The parts are designed to be read in order, but they are not strictly sequential. A reader with a background in reinforcement learning and game theory can move from this chapter directly to Part II, returning to Part I as needed, and a reader interested mainly in security can begin Part III after Chapters 7 and 8, consulting Chapter 5 before Chapter 14.

### 1.6.2 Suggested Course Paths

The book supports several one-semester courses. Table 1.4 lists three, matching the table in the preface.

A *theory course* for students in computer science, electrical engineering, economics, or operations research covers all of Part I, supported by Appendices A–C, and then applies its tools to two problems from Part III: adversarial games and strategic defense (Chapter 14) and alignment and human oversight (Chapter 16). Its exercises emphasize proofs and formal modeling.

A *systems course* for students who will build agents reads this chapter, the analysis of

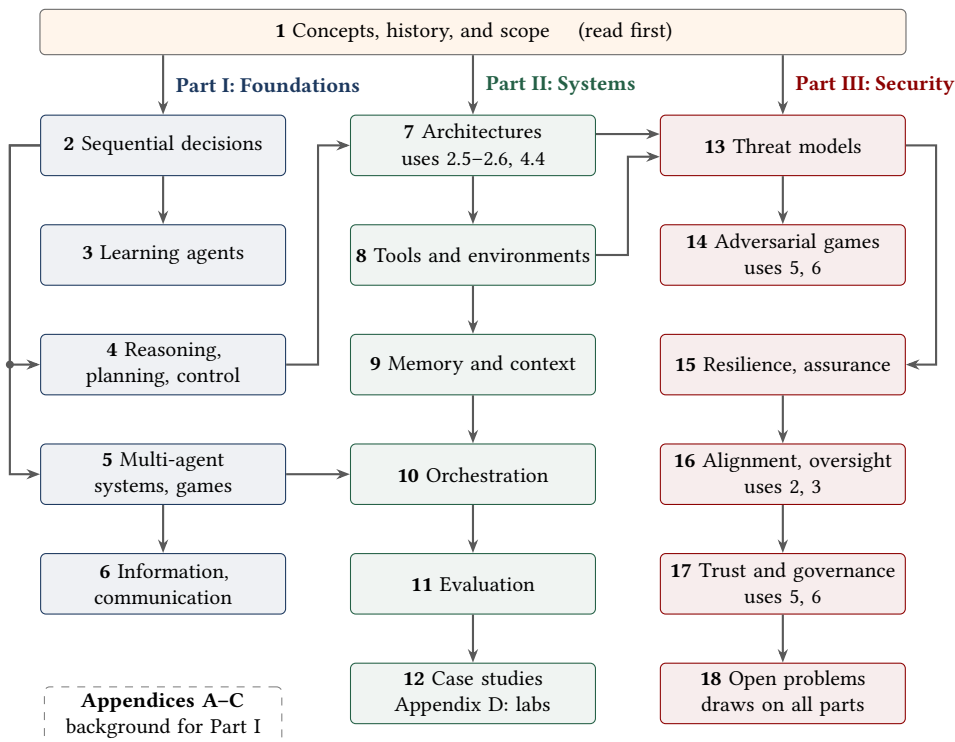


FIGURE 1.7. Principal dependencies among chapters. Arrows point from a chapter to chapters that build on it; small labels list further prerequisites. Part II can be read in order after Chapter 1 and the indicated sections of Chapters 2 and 4.

long-horizon error and language action spaces in Sections 2.5 and 2.6, and the treatment of interleaved reasoning and acting in Section 4.4, and then covers Part II in full. It closes with threat models (Chapter 13) and resilience (Chapter 15), because no one should deploy an agent without them. The laboratory exercises of Appendix D provide the course project.

*A course on secure and trustworthy agents*, suitable for students in security, safety, or policy, reads this chapter, game theory (Chapter 5), and the architecture and tool chapters of Part II (Chapters 7 and 8) as background, and then covers Part III through governance (Chapters 13 to 17).

Instructors can combine paths. For a two-semester sequence, a natural division is Part I with Chapter 7 in the first semester and the remainder of Parts II and III in the second.

## KEY TAKEAWAYS

- An agentic system is goal-directed, interacts with an environment over many steps, and exercises authority delegated by a principal (Theorem 1.3). Agency is a matter of

degree, and it belongs to the whole system (model, scaffold, tools, and permissions) rather than to the model alone.

- One pass through the loop (1.1) reads the context, chooses an action, and folds the observation back in. A typical episode (Table 1.1) contains many cheap steps and a few consequential ones, and constraints enforced inside tools hold more reliably than constraints stated only in instructions.
- Chatbots, workflows, and agents differ in who chooses the next step and who acts (Fig. 1.2). Autonomy is best assigned per processing stage and per action class, not per system (Table 1.2, Fig. 1.1), and higher autonomy is appropriate only when reliability, reversibility, and supervision allow it.
- Foundation-model agents revive old ideas with a new policy class: the cybernetic feedback loop, symbolic plans, BDI intentions and working memory, and learning from reward (Fig. 1.3). Between 2024 and 2026 they were deployed first where work is textual, verifiable, and containable, above all in software engineering, and their tool and agent interfaces moved toward open standards.
- A reference architecture of perception and context, reasoning and planning, action and tools, memory, and an environment of artifacts, humans, and other agents (Fig. 1.4) organizes Part II. The running examples R1 (research assistant), R2 (software engineering agent), and R3 (security operations agent) recur throughout the book (Fig. 1.5).
- Classifying systems by autonomy, environment, number of agents, and learning mode (Table 1.3) helps anticipate their failure modes and the tools of analysis that apply.
- Long horizons compound errors: the half-success horizon is inversely proportional to the effective per-step failure probability  $\epsilon(1 - \rho)$ , so recovery matters as much as accuracy, and measured task horizons have grown steadily. Open-ended action spaces defeat enumeration and exact specification; untrusted inputs turn agents into confused deputies; and delegation creates problems of trust and accountability. Parts I, II, and III address these questions in turn.

---

## Notes and Further Reading

The concept of an agent has been defined many times. Russell and Norvig (2020) organize artificial intelligence around rational agents and give the standard classification of environments (fully or partially observable, deterministic or stochastic, episodic or sequential, static or dynamic, single-agent or multi-agent), which complements the taxonomy of Section 1.4. Franklin and Graesser (1997) survey early definitions and propose a taxonomy of autonomous agents, and Wooldridge (2009) gives a thorough

introduction to agent architectures, communication, and multi-agent interaction from the perspective of the 1990s and 2000s agents community. For the graded view of agency adopted here, see [Chan et al. \(2023\)](#), who analyze how harms grow with increasing agency, and [Shavit et al. \(2023\)](#), who propose practices for governing agentic systems. [Kasirzadeh and Gabriel \(2025\)](#) characterize agents along four dimensions (autonomy, efficacy, goal complexity, and generality) and use the resulting profiles to locate governance challenges; their dimensions are a useful cross-check on [Theorem 1.3](#). The principal-agent problem from economics ([Holmström, 1979](#)) is the natural formal lens for delegation and is taken up in [Chapters 5 and 17](#).

For the history, [Wiener \(1948\)](#) remains readable, and the short paper of [Rosenblueth et al. \(1943\)](#) states the feedback view of purpose with unusual clarity. [Simon \(1955\)](#) introduced bounded rationality, a theme that resonates with agents operating under token and cost budgets. The technical note of [Nilsson \(1984\)](#) documents Shakey, and [Fikes and Nilsson \(1971\)](#) and [Hart et al. \(1968\)](#) describe its planning and search algorithms. [Brooks \(1986\)](#) is the classic statement of the reactive alternative. [Bratman \(1987\)](#) is the philosophical source of BDI, and [Rao and Georgeff \(1995\)](#) connect it to implemented systems; [Laird \(2012\)](#) and [Anderson et al. \(2004\)](#) describe the two most developed cognitive architectures. [Sutton and Barto \(2018\)](#) is the standard introduction to reinforcement learning, and [Mnih et al. \(2015\)](#) and [Silver et al. \(2016\)](#) mark the arrival of deep reinforcement learning.

The literature on language agents grows quickly, and any survey dates fast. [Wang et al. \(2024b\)](#) and [Xi et al. \(2023\)](#) are broad early surveys of LLM-based agents. [Sumers et al. \(2024\)](#) propose a framework, grounded in cognitive architectures, that separates memory, action space, and decision procedure, and it is the closest antecedent of our reference architecture. The primary papers on ReAct ([Yao et al., 2023b](#)), Toolformer ([Schick et al., 2023](#)), Voyager ([Wang et al., 2024a](#)), generative agents ([Park et al., 2023](#)), and SWE-agent ([Yang et al., 2024](#)) repay careful reading. For a practitioner’s perspective, [Schluntz and Zhang \(2024\)](#) distinguish workflows, in which language models follow predefined code paths, from agents, in which models direct their own process, and argue for using the simplest design that works; [Chapter 7](#) develops this distinction. For what is actually deployed, the AI Agent Index ([Staufer et al., 2026](#)) documents the design, capabilities, and safety disclosures of deployed agents in a structured form that will date more gracefully than any product list. The time-horizon methodology of [Kwa et al. \(2025\)](#), with its 2026 update ([METR, 2026a](#)), is the most widely used way to track agent capability over time; read it together with its caveats about task selection and human baselines. For the economics of agents deployed in organizations, [Zhu \(2026a\)](#) propose a workflow-based framework for valuing, attributing, and pricing human and AI agents. For strategic and security perspectives on LLM agents, [Zhu \(2025b\)](#) review game-theoretic frameworks at the intersection of agentic AI and cybersecurity, [Li et al. \(2025\)](#) survey the convergence of language models and cybersecurity, [Huang and Zhu \(2020a\)](#) describe active, adaptive, and autonomous cyber defense that predates language agents, and [Liu et al. \(2024b\)](#) probe the social cognition and systematic biases of LLM

agents in multi-agent settings.

On autonomy and human oversight, Sheridan and Verplank (1978) introduced the ten-level scale of automation, and Parasuraman et al. (2000) extended it to the four processing stages used in this chapter. Feng et al. (2025) propose five levels of autonomy for language agents defined by the user’s role (operator, collaborator, consultant, approver, and observer) and argue that autonomy should be a deliberate design choice, separate from capability. Lee and See (2004) review the psychology of trust in automation and the problems of misuse and disuse, and Matthias (2004) articulates the responsibility gap for learning systems. Amodei et al. (2016) set out an influential agenda of concrete safety problems, including side effects and reward hacking, that bear directly on agents. Hammond et al. (2025) survey risks from interacting advanced agents, and Chan et al. (2024) discuss measures, such as agent identifiers and activity logging, for making agents visible to those who must oversee them. These threads are developed in Part III.

## Exercises

Exercises marked with a hint include a brief pointer to the key step.

EXERCISE 1.1. Classify a thermostat, a spam filter, a chess engine, a coding assistant that opens pull requests, and an automated trading system along the four dimensions of Section 1.4: level of autonomy (Table 1.2), environment, number of agents, and learning mode. For each, state which clause of Theorem 1.3 it satisfies most weakly. Which of your classifications change if the system is deployed in a different context?

*Hint.* Classify per action class where one level does not fit: the trading system’s order placement and its risk-limit changes may deserve different levels.

EXERCISE 1.2. Model delegation to an LLM agent as a principal–agent relationship. What does the principal fail to observe (actions, information, intentions), and what are the analogues of monitoring and incentive contracts? Identify three design choices in an agent system, such as logging, approval gates, or restricted tool permissions, and explain which information gap each narrows and at what cost.

*Hint.* Separate hidden action (the principal cannot see what the agent did) from hidden information (the agent knows something the principal does not). Logging addresses the first; neither logging nor approval fully addresses the second.

EXERCISE 1.3. Continue Theorem 1.2. Two days after the booking, the observation “the 20:15 train is canceled; refund issued” arrives.

- (a) Write a trace of at least five actions and observations for the rebooking episode in the format of Table 1.1. State what the agent must retrieve from  $\mathcal{M}$  and what the new initial context  $c_0$  contains.
- (b) Suppose the only remaining refundable train costs \$162. List the agent’s options, and say which of them a purchase tool that enforces the \$150 cap permits.

- (c) Assign a level from Table 1.2 to each action in your trace, and justify the level you give the purchase.

EXERCISE 1.4. Place each of the following systems in one panel of Fig. 1.2, or explain why it falls between panels: (i) a nightly pipeline that runs tests, asks a model to summarize failures, and files a ticket; (ii) a customer-support system in which a model routes each message to one of five fixed handlers; (iii) a research agent that decides what to search next until it judges its report complete; (iv) a computer-use agent that fills in expense reports in a web application. For each, say who chooses the next step, who acts, and what authority has been delegated.

*Hint.* Ask whether the set of possible execution paths is enumerable from the code alone. If a model output can add a step that the code does not list, the system has at least one agentic component.

EXERCISE 1.5. Map the belief–desire–intention decomposition onto the loop of Eq. (1.1) for the software engineering agent R2. Where are beliefs, desires, and intentions represented, and which of them are explicit data structures and which are implicit in the model? What would it mean for R2 to “reconsider an intention,” and how could a scaffold decide when to do so?

EXERCISE 1.6. Extend Theorem 1.5. An agent must complete  $T$  steps. At each step it errs independently with probability  $\epsilon$ ; an error is detected and repaired with probability  $\rho$  at the cost of one extra step, and otherwise it is fatal.

- Show that the probability of success is  $(1 - \epsilon(1 - \rho))^T$ , and compute the expected number of steps taken, including repairs, given that the task succeeds.
- For  $T = 100$ , find the per-step error probability  $\epsilon$  that achieves 90% success with  $\rho = 0$ , and the recovery probability  $\rho$  that achieves the same success with  $\epsilon = 0.02$ .
- Argue informally which of the independence assumptions is least realistic for a language agent, and in which direction violating it changes your answers.

*Hint.* In (a), condition on success: at each step the agent either made no error or made a repaired one, so each step independently costs one extra step with probability  $\epsilon\rho/(1 - \epsilon(1 - \rho))$ . In (b), you should find  $\epsilon \approx 0.001$  and  $\rho \approx 0.95$ .

EXERCISE 1.7. This exercise connects (1.2) to measured task horizons.

- Derive (1.2), and show that  $0.69/p$  is within 1% of  $T_{1/2}$  when  $p \leq 0.02$ .
- Suppose the effective failure probability per unit of work halves every seven months. By what factor does  $T_{1/2}$  grow in two years? Repeat for a halving every four months.
- Give two reasons why a measured doubling of the time horizon of Kwa et al. (2025) need not imply that per-step failure probability has halved.

*Hint.* In (a), use  $p \leq -\ln(1-p) \leq p + p^2$  for small  $p$ . In (b), the factor is  $2^{24/7} \approx 10.8$  for seven months.

EXERCISE 1.8. Suppose the context update  $U$  keeps only the most recent  $W$  actions and observations. Show that the resulting agent is a finite-memory policy: its action distribution depends on the history only through a window of fixed length. Give a simple environment in which such an agent cannot act optimally for any  $W$ , however large its model, and describe two ways a memory store  $\mathcal{M}$  could remedy the problem. (Chapter 2 discusses the belief states that make this precise.)

*Hint.* Consider an environment that shows a cue at the first step and rewards an action that depends on the cue only after an arbitrarily long, uninformative delay.

EXERCISE 1.9. For the security operations agent R3, list at least six distinct actions it may take, from querying a log to disabling an administrator account. Using the four processing stages of Parasuraman et al. (2000), assign a level from Table 1.2 to each action and justify your choice in terms of reversibility, cost of error, time pressure, and the risk that an adversary manipulates the agent's inputs. Which assignments would you revisit after a month of reliable operation, and what evidence would justify the change?

EXERCISE 1.10. The research assistant R1 reads untrusted content, holds private data, and can send messages. For each of these three properties, propose an architectural change that removes it, and describe what the agent can no longer do as a result. Is there a design that keeps most of R1's usefulness while ensuring that no single episode combines all three? (Revisit your answer after Chapters 13 and 15.)

*Hint.* Consider splitting R1 into two agents with different permissions, and ask what may pass between them.

EXERCISE 1.11. Brooks (1986) argued for reactive behaviors coupled directly to sensors in place of deliberation over a world model. Identify the components of a typical coding agent such as R2 that are reactive in this sense and those that are deliberative. Give one situation in which adding deliberation improves R2's performance and one in which it makes performance worse.

EXERCISE 1.12. Short essay: argue for or against the claim that a chat model with no tools is not an agent. Your argument should use Theorem 1.3, address whether a human user who acts on the model's advice closes the loop on its behalf, and consider whether the answer matters for how such systems should be governed.

---

## Agents as Sequential Decision-Makers

*An agent chooses actions over time, with incomplete information, in pursuit of goals that someone must specify. This chapter builds the decision-theoretic language for such choices and applies it to language agents. It starts from expected utility and its departures, which are risk, robustness, and bounded rationality. It then solves a small Markov decision process by value iteration, updates a security agent's belief with real numbers, and shows why a context window is best understood as an approximate information state. Next it turns a temporal-logic requirement into an automaton that a shield can enforce, and follows a software agent's success probability over a long horizon to see which design changes actually move it. It closes with the formal agent–environment interface on which Parts II and III build.*

---

### LEARNING OBJECTIVES

- State the expected-utility framework and its risk-sensitive and robust variants, and explain why bounded rationality is the operating condition of language agents.
  - Formulate Markov decision processes, derive the Bellman equations, run value iteration by hand on a small example, and explain why deterministic stationary policies suffice.
  - Formulate POMDPs, carry out a numerical belief update, and define exact and approximate information states.
  - Distinguish rewards, goals, and temporal-logic specifications, translate a safety requirement into an automaton, and recognize when a specification diverges from intent.
  - Compute trajectory-level success from per-step error, recovery, and irrecoverable-failure probabilities, and use the result to compare design changes and to interpret measured task horizons.
  - Model a language agent as a step-level policy over contexts, induced by a token-level model, within a formal agent–environment interface.
-

## 2.1 Decision Making under Uncertainty

Consider R2, the software engineering agent, at the start of an episode. It can patch the most suspicious function right away, or it can first spend a step running a diagnostic. Patching now is faster but more likely to be wrong. The diagnostic costs a step and improves the odds. Any principled answer weighs the uncertain outcomes of each option by how much each outcome matters. This section recalls the classical standard for that weighing, expected utility, and the two directions in which practice departs from it: toward caution about risk and model error, and toward the limits of computation.

### 2.1.1 Preferences and Expected Utility

Let  $X$  be a finite set of outcomes and  $\Delta(X)$  the set of *lotteries* over it. A decision-maker has a preference relation  $\succeq$  on  $\Delta(X)$ . von Neumann and Morgenstern (1944) showed that four conditions characterize the preferences that expected values can represent. The first two are *completeness* and *transitivity*. The third is *continuity*: if  $p \succ q \succ r$ , then some mixture of  $p$  and  $r$  is indifferent to  $q$ . The fourth is *independence*:  $p \succeq q$  if and only if  $\alpha p + (1 - \alpha)r \succeq \alpha q + (1 - \alpha)r$  for every lottery  $r$  and every  $\alpha \in (0, 1]$ .

**THEOREM 2.1** (Expected utility). *A preference relation on  $\Delta(X)$  satisfies completeness, transitivity, continuity, and independence if and only if there is a utility function  $u : X \rightarrow \mathbb{R}$  such that*

$$p \succeq q \iff \sum_{x \in X} p(x) u(x) \geq \sum_{x \in X} q(x) u(x).$$

*The function  $u$  is unique up to a positive affine transformation.*

The theorem takes probabilities as given. Savage (1954) removed that assumption. A Savage decision-maker ranks *acts*, which are maps from unknown states of the world to outcomes. Under a set of axioms on that ranking, there exist a subjective probability  $P$  on states and a utility  $u$  such that act  $f$  is preferred to act  $g$  exactly when  $\mathbb{E}_P[u(f)] \geq \mathbb{E}_P[u(g)]$ . Both beliefs and values are thus recovered from choices. Add Bayesian updating of  $P$  as evidence arrives, and the result is the normative model of the rational agent that Chapter 1 attributed to Russell and Norvig (2020). Every value function in this chapter rests on it.

Two cautions matter for artificial agents. First, the theorem is about *representation*. It says that coherent preferences can be summarized by a utility, not that anyone can write that utility down. For a language agent, the utility usually sits implicitly in a natural-language instruction, and Section 2.4 examines what is lost when it is made explicit. Second, the axioms bind the whole decision problem, including how outcomes are described. An agent whose choice changes when the same options are rephrased violates them. This is one reason prompt sensitivity is a decision-theoretic defect and not merely a stylistic one. Experiments that probe multi-LLM agents for human-like cognitive biases in social settings examine such departures directly (Liu et al., 2024b).

### 2.1.2 Risk Sensitivity and Robust Decisions

Expected utility is already risk-sensitive when  $u$  is nonlinear in money or reward: a concave  $u$  prefers a sure outcome to a lottery with the same mean. Sequential problems, however, usually maximize the expected sum of rewards, which is risk-neutral in that sum. Two families of alternatives recur. The *entropic* objective  $-\beta^{-1} \log \mathbb{E}[e^{-\beta G}]$ , for a random return  $G$  and  $\beta > 0$ , penalizes variance; for small  $\beta$  it is approximately  $\mathbb{E}[G] - \frac{\beta}{2} \text{Var}(G)$ . The *conditional value-at-risk* of a loss  $L$  at level  $\alpha \in (0, 1)$  averages the worst  $\alpha$  fraction of outcomes. It admits the variational form

$$\text{CVaR}_\alpha(L) = \min_{\eta \in \mathbb{R}} \left\{ \eta + \frac{1}{\alpha} \mathbb{E}[(L - \eta)^+] \right\}$$

(Rockafellar and Uryasev, 2000). Exponentiated costs of the entropic type also define risk-sensitive mean-field games (Tembine et al., 2014), and a risk-sensitive MDP has been used to compute an attacker’s replanning strategy when evaluating a moving target defense (Qian et al., 2020).

EXAMPLE 2.2 (Same mean, different tails). R3 can handle a class of alerts in one of two ways. Loss is measured in hours of avoidable downtime. Under policy  $A$ , R3 isolates hosts automatically; it is usually right, but with probability 0.01 it isolates a production server and causes 100 hours of downtime, and otherwise it causes none. Under policy  $B$ , R3 queues every isolation for a human, which always costs 1 hour. Both have expected loss 1, so a risk-neutral designer is indifferent. At  $\alpha = 0.05$ , the worst 5% of outcomes under  $A$  consist of the 1% disaster and 4% of zero-loss outcomes. Their average is  $\text{CVaR}_{0.05}(A) = (0.01 \cdot 100 + 0.04 \cdot 0)/0.05 = 20$  hours. The variational form gives the same value at  $\eta = 0$ . For  $B$ ,  $\text{CVaR}_{0.05}(B) = 1$ . A tail measure states the designer’s real concern, rare severe failures, which the mean hides.

Risk concerns randomness within a known model. *Ambiguity* concerns uncertainty about the model itself. A robust decision-maker considers a set  $\mathcal{P}$  of plausible models and maximizes the worst case,  $\max_a \min_{P \in \mathcal{P}} \mathbb{E}_P[u(a)]$ . Gilboa and Schmeidler (1989) axiomatized this maxmin criterion by weakening Savage’s axioms. In sequential problems, robust Markov decision processes keep a dynamic-programming solution when the uncertainty set factors across states and actions (Iyengar, 2005; Nilim and El Ghaoui, 2005). A further step drops the assumption that the model set contains the truth. When the agent’s model class is misspecified, stable behavior in an infinite-horizon MDP can be characterized as a Berk–Nash equilibrium, in which the agent acts optimally for a subjective model that is statistically consistent with the data its own policy generates (Zhu and Han, 2026a). Hammar et al. (2025) learn security response strategies in a partially observed game in which each player’s conjecture about the model may be misspecified and is adapted by Bayesian learning. The worst case need not be chosen by nature. When an adversary shapes the environment, as for R3, the minimizing player is real and the problem becomes a game (Chapters 5 and 14).

### 2.1.3 Bounded Rationality and Satisficing

Expected-utility maximization presumes that the decision-maker can list the alternatives, compute their consequences, and optimize. Simon (1955) argued that real decision-makers cannot. He proposed *bounded rationality*: choice procedures adapted to limited information and computation. His central example is *satisficing*. The decision-maker examines alternatives one at a time and stops at the first whose value exceeds an aspiration level, raising or lowering that level as the search proves easy or hard. Limited attention can be modeled in the same spirit: in the model of Chen and Zhu (2018), each player responds only to a sparse cognitive network of the neighbors it observes.

For language agents, bounded rationality is the operating condition, not a footnote. A model sees a finite context, spends a finite number of tokens reasoning, and faces an action space too large to enumerate (Chapter 8). Its designers make boundedly rational choices on its behalf: how many search results to read, how many candidate patches to sample, when to stop exploring. Each choice trades the value of more computation or information against its cost. Control theory has formalized versions of this trade, co-designing a controller with a strategy for costly measurements (Huang and Zhu, 2020b) or with a triggering policy that decides when to update under an update penalty (Huang and Zhu, 2021). The normative theory remains useful as a benchmark and as a source of structure. Game-theoretic models of LLM-based agents build bounded rationality in directly, letting each agent reason through structured prompts and generate probabilistic behavior through the model (Zhu, 2025c). Sections 2.3 and 2.5 make two bounds precise: the information lost by a finite context, and the reliability lost over a finite but long horizon.

## 2.2 Markov Decision Processes

### 2.2.1 Model, Policies, and Value Functions

The simplest sequential setting has three ingredients. The agent sees the current state of the world, takes an action, and receives a reward, and the world moves to a new state whose distribution depends only on the current state and action (Fig. 2.1). The agent’s problem is to choose actions so that the rewards accumulated over time are large. A good action is therefore judged not only by its reward now but by the state it leads to.

**DEFINITION 2.3** (Markov decision process). A (finite, discounted) *Markov decision process* (MDP) is a tuple  $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$  with finite state set  $\mathcal{S}$ , finite action set  $\mathcal{A}$ , transition kernel  $P(s' \mid s, a)$ , bounded reward function  $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ , and discount factor  $\gamma \in [0, 1)$ .

The *Markov property* is built in: the distribution of  $s_{t+1}$  depends on the past only through  $(s_t, a_t)$ . A *policy* specifies how actions are chosen. In full generality it is *history-dependent and randomized*:  $\pi = (\pi_0, \pi_1, \dots)$  with  $\pi_t(a \mid s_0, a_0, \dots, s_t)$ . It is *Markov* if  $\pi_t$  depends only on  $s_t$ , *stationary* if in addition  $\pi_t = \pi$  for all  $t$ , and *deterministic* if each

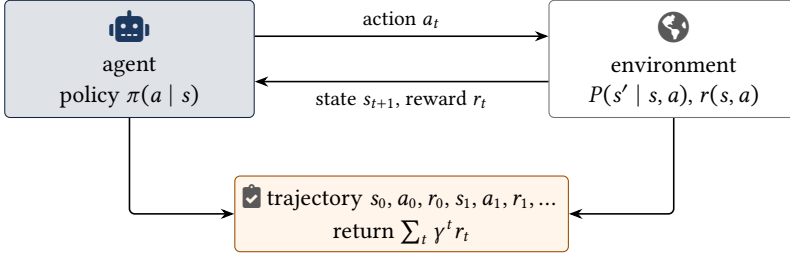


FIGURE 2.1. The Markov decision process loop. The agent acts on the state; the environment samples the next state from  $P$ , which depends only on the current state and action, and emits a reward. A policy induces a distribution over trajectories, and value functions average the discounted return over that distribution. Under partial observability (Section 2.3) the agent receives an observation instead of the state.

$\pi_t$  puts all its mass on one action, in which case we write  $\pi(s) \in \mathcal{A}$ . A policy and an initial state determine a distribution over trajectories, with expectation  $\mathbb{E}^\pi$ . The *value* and *action-value* functions are

$$V^\pi(s) = \mathbb{E}^\pi \left[ \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s \right],$$

$$Q^\pi(s, a) = \mathbb{E}^\pi \left[ \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right],$$

and the optimal value is  $V^*(s) = \sup_\pi V^\pi(s)$ , the supremum taken over all history-dependent randomized policies.

### 2.2.2 Bellman Equations and Dynamic Programming

For a stationary Markov policy  $\pi$ , split the return into the first reward and the discounted remainder. By the tower property of conditional expectation and the Markov property, the remainder from  $s_1$  onward has conditional expectation  $V^\pi(s_1)$ , so

$$V^\pi(s) = \sum_a \pi(a \mid s) \left[ r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^\pi(s') \right], \quad (2.1)$$

$$Q^\pi(s, a) = r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^\pi(s').$$

This *Bellman expectation equation* is a linear system with unique solution  $V^\pi = (I - \gamma P^\pi)^{-1} r^\pi$ , where  $P^\pi$  and  $r^\pi$  are the transition matrix and reward vector under  $\pi$ . Replacing the average over  $\pi$  by a maximum gives the *Bellman optimality equation*

$$V(s) = (BV)(s) := \max_{a \in \mathcal{A}} \left[ r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V(s') \right]. \quad (2.2)$$

Both right-hand sides define operators that are  $\gamma$ -contractions in the sup norm, a fact proved in Chapter C.

### 💡 KEY IDEA

*The value of a state is the best immediate reward plus the discounted value of where that action leads. Every planning and learning method in this book approximates this one recursion.*

**THEOREM 2.4** (Optimality of deterministic stationary policies). *In a finite discounted MDP:*

- (i)  $V^*$  is the unique bounded solution of (2.2);
- (ii) every deterministic stationary policy with  $\pi(s) \in \arg \max_a Q^*(s, a)$ , where  $Q^*(s, a) = r(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s')$ , satisfies  $V^\pi = V^*$ .

*Hence optimizing over deterministic stationary Markov policies loses nothing relative to history-dependent randomized ones.*

*Proof sketch.* Since  $\mathcal{B}$  is a  $\gamma$ -contraction, the Banach fixed-point theorem gives a unique bounded fixed point  $\bar{V}$  (Chapter C). Let  $\bar{\pi}$  be greedy with respect to  $\bar{V}$ . Its evaluation operator  $\mathcal{B}^{\bar{\pi}}$ , the right side of (2.1), satisfies  $\mathcal{B}^{\bar{\pi}} \bar{V} = \mathcal{B} \bar{V} = \bar{V}$ ; being a contraction with fixed point  $V^{\bar{\pi}}$ , it forces  $V^{\bar{\pi}} = \bar{V}$ . Hence  $\bar{V} \leq V^*$ .

For the reverse inequality, fix any history-dependent policy  $\pi$  and let

$$G_{t,n} = \sum_{k=t}^{t+n-1} \gamma^{k-t} r(s_k, a_k) + \gamma^n \bar{V}(s_{t+n}).$$

We show by induction on  $n$  that  $\mathbb{E}^\pi[G_{t,n} | h_t] \leq \bar{V}(s_t)$  for every history  $h_t$ . The case  $n = 0$  is an equality. For  $n \geq 1$ ,  $G_{t,n} = r(s_t, a_t) + \gamma G_{t+1,n-1}$ , and the induction hypothesis applied at  $h_{t+1}$  gives

$$\mathbb{E}^\pi[G_{t,n} | h_t] \leq \sum_a \pi_t(a | h_t) \left[ r(s_t, a) + \gamma \sum_{s'} P(s' | s_t, a) \bar{V}(s') \right] \leq (\mathcal{B} \bar{V})(s_t) = \bar{V}(s_t).$$

Taking  $t = 0$  and letting  $n \rightarrow \infty$ , the bounded terminal term vanishes and  $V^\pi(s) \leq \bar{V}(s)$ . Thus  $V^* \leq \bar{V} = V^{\bar{\pi}} \leq V^*$ , proving both claims. ■

The proof is constructive. *Value iteration*,  $V_{k+1} = \mathcal{B}V_k$ , converges geometrically,  $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$ . *Policy iteration* alternates exact evaluation (2.1) with greedy improvement and terminates in finitely many iterations (Puterman, 1994; Bertsekas, 2017). Viewing the policy as a decision variable and the value function as payoff feedback gives an online-optimization reading of the same recursion, from which Blackwell approachability yields convergent variants of value iteration and Q-learning (Li et al., 2021). The underlying *principle of optimality* is due to Bellman (1957): an optimal policy, from any state it reaches, continues optimally. Planning algorithms that approximate these computations in large or unknown models are the subject of Chapter 4, and learning them from experience is the subject of Chapter 3.

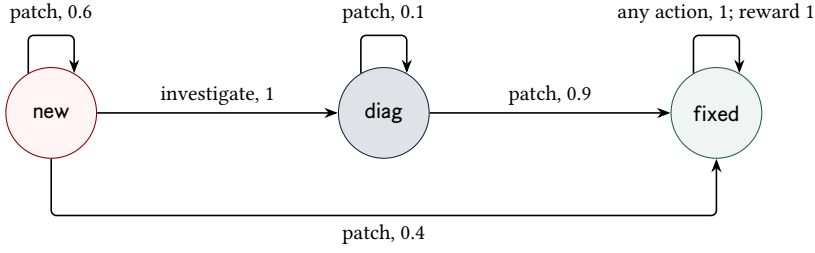


FIGURE 2.2. A three-state MDP for R2's first decision. Edges carry the action and the transition probability. Only fixed pays reward, 1 per step, and  $\gamma = 0.9$ . Patching immediately is a gamble with success probability 0.4; investigating costs one step of discounting but raises the success probability of the patch to 0.9.

EXAMPLE 2.5 (Value iteration for R2's first decision). Return to R2's choice from the opening of the chapter, stripped to three states (Fig. 2.2). In new, R2 has not yet located the bug. It can *patch*, which fixes the bug with probability 0.4 and otherwise leaves it in new, or *investigate*, which moves it for certain to diag, where it knows the faulty function. From diag, a patch succeeds with probability 0.9. The state fixed is absorbing and pays reward 1 per step, standing in for the ongoing value of working code. All other rewards are 0, and  $\gamma = 0.9$ .

Start value iteration from  $V_0 = 0$ . With  $V = V_k$ , the two action values in new are

$$Q_k(\text{new}, \text{patch}) = 0.9 (0.4 V_k(\text{fixed}) + 0.6 V_k(\text{new})),$$

$$Q_k(\text{new}, \text{investigate}) = 0.9 V_k(\text{diag}),$$

and the updates for the other states are  $V_{k+1}(\text{diag}) = 0.9 (0.9 V_k(\text{fixed}) + 0.1 V_k(\text{diag}))$  and  $V_{k+1}(\text{fixed}) = 1 + 0.9 V_k(\text{fixed})$ . Table 2.1 lists the iterates. At  $k = 2$ , patching still looks better, 0.878 against 0.729. At  $k = 3$  the order flips, 1.450 against 1.451, and it stays flipped. The limit follows from the fixed-point equations:  $V^*(\text{fixed}) = 1/(1 - 0.9) = 10$ ,  $V^*(\text{diag}) = 0.81 \cdot 10/(1 - 0.09) = 8.901$ , and  $V^*(\text{new}) = \max\{7.926, 0.9 \cdot 8.901\} = 8.011$ , attained by investigating. Always patching would be worth only  $3.6/0.46 = 7.826$  from new.

The flip has a meaning. With zero terminal values,  $V_k$  is the optimal value of a problem with  $k$  steps left, so the greedy action with respect to  $V_k$  is optimal when  $k + 1$  steps remain. With three or fewer steps left, R2 should patch at once. With four or more, the detour through diagnosis pays for itself. The error bound is also visible in the table:  $\|V_5 - V^*\|_\infty = 10 - 4.095 = 5.905 = 0.9^5 \cdot 10$ , which meets the contraction bound exactly, with the maximum attained at fixed.

### 2.2.3 Finite, Discounted, and Average-Reward Criteria

The discounted criterion is one of three standard choices. Under a *finite horizon*  $T$ , the objective is  $\mathbb{E}^\pi[\sum_{t=0}^{T-1} r_t(s_t, a_t) + r_T(s_T)]$ . *Backward induction*,  $V_T = r_T$  and  $V_t(s) =$

TABLE 2.1. Value iteration on Fig. 2.2, starting from  $V_0 = 0$ . The last two value columns are the action values in new computed from  $V_k$ ; the greedy action is the better of the two. The row  $\infty$  is the fixed point.

| $k$      | $V_k(\text{new})$ | $V_k(\text{diag})$ | $V_k(\text{fixed})$ | $Q_k(\text{new, patch})$ | $Q_k(\text{new, inv.})$ | greedy in new |
|----------|-------------------|--------------------|---------------------|--------------------------|-------------------------|---------------|
| 0        | 0.000             | 0.000              | 0.000               | 0.000                    | 0.000                   | tie           |
| 1        | 0.000             | 0.000              | 1.000               | 0.360                    | 0.000                   | patch         |
| 2        | 0.360             | 0.810              | 1.900               | 0.878                    | 0.729                   | patch         |
| 3        | 0.878             | 1.612              | 2.710               | 1.450                    | 1.451                   | investigate   |
| 4        | 1.451             | 2.340              | 3.439               | 2.021                    | 2.106                   | investigate   |
| 5        | 2.106             | 2.996              | 4.095               | 2.612                    | 2.697                   | investigate   |
| $\infty$ | 8.011             | 8.901              | 10.000              | 7.926                    | 8.011                   | investigate   |

$\max_a[r_t(s, a) + \sum_{s'} P(s' | s, a)V_{t+1}(s')]$ , yields an optimal policy that is deterministic and Markov but in general *nonstationary*: with few steps left, it is right to act differently. Theorem 2.5 shows this in miniature, since patching is right with three steps left and investigating with four. An agent with a step budget faces exactly this problem, because the value of starting a long investigation depends on how many steps remain. Under the *average-reward* criterion  $\liminf_{T \rightarrow \infty} T^{-1} \mathbb{E}^\pi[\sum_{t < T} r(s_t, a_t)]$ , deterministic stationary policies are again optimal for finite MDPs. Under a unichain condition, the optimal gain  $g$  and a bias function  $h$  solve  $g + h(s) = \max_a[r(s, a) + \sum_{s'} P(s' | s, a)h(s')]$  (Puterman, 1994). This criterion suits continuing tasks, such as an R3 agent that monitors alerts indefinitely.

Discounting has two readings. It may express a genuine preference for earlier reward, or it may model a constant probability  $1 - \gamma$  that the episode ends at each step, since an undiscounted return with geometric termination has the same expectation. The second reading connects discounting to the termination hazards of Section 2.5. Episodic tasks that end in a goal or failure state are *stochastic shortest path* problems. These are well posed only for policies that terminate with probability one (Bertsekas, 2017). An agent loop that might never stop falls outside the theory, which is one more reason for explicit budgets.

## 2.3 Partial Observability

An MDP policy sees the state. An agent almost never does. R2 sees the files it has opened, not the whole repository; R3 sees logs that an intruder may have edited. An agent in this position must act on evidence, and the evidence arrives one observation at a time. This section develops the theory of acting on partial information and uses it to say precisely what an agent's context is for.

### 2.3.1 POMDPs and Belief States

**DEFINITION 2.6 (POMDP).** A *partially observable Markov decision process* is a tuple  $(\mathcal{S}, \mathcal{A}, \mathcal{O}, P, \Omega, r, \gamma, b_0)$  in which  $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$  is an MDP,  $\mathcal{O}$  is a finite observation set,  $\Omega(o \mid s', a)$  is the probability of observing  $o$  after action  $a$  leads to state  $s'$ , and  $b_0 \in \Delta(\mathcal{S})$  is the distribution of the initial state, which emits  $o_0 \sim \Omega_0(\cdot \mid s_0)$ . The agent observes the history  $h_t = (o_0, a_0, \dots, o_t) \in \mathcal{H}$  but not the states.

The *belief*  $b_t(s) = \mathbb{P}(s_t = s \mid h_t)$  summarizes what the history says about the current state. Bayes' rule gives the recursive update  $b_{t+1} = \beta(b_t, a_t, o_{t+1})$ , where

$$\begin{aligned} \beta(b, a, o)(s') &= \frac{\Omega(o \mid s', a) \sum_s P(s' \mid s, a) b(s)}{\mathbb{P}(o \mid b, a)}, \\ \mathbb{P}(o \mid b, a) &= \sum_{s'} \Omega(o \mid s', a) \sum_s P(s' \mid s, a) b(s). \end{aligned} \quad (2.3)$$

The update has two stages. The inner sum *predicts*: it pushes the old belief through the dynamics. The factor  $\Omega$  *corrects*: it reweights each predicted state by how well it explains what was observed, and the denominator renormalizes.

**EXAMPLE 2.7 (R3 updates its belief about a host).** R3 tracks whether a host is *benign* or *compromised*, with current belief  $b_t(\text{compromised}) = 0.10$ . Between observations, a benign host becomes compromised with probability 0.05, and a compromised host stays compromised. A detector raises an alert with probability 0.9 on a compromised host and 0.2 on a benign one. A log review reports “clean” with probability 0.95 on a benign host but 0.3 on a compromised one, since an intruder may have edited the logs. Figure 2.3 draws the steps.

*Predict.*  $\mathbb{P}(\text{compromised}) = 0.10 + 0.90 \cdot 0.05 = 0.145$ . *Correct on an alert.* The unnormalized weights are  $0.145 \cdot 0.9 = 0.1305$  and  $0.855 \cdot 0.2 = 0.1710$ . Their sum,  $\mathbb{P}(\text{alert} \mid b_t) = 0.3015$ , is the denominator of (2.3), and  $b_{t+1}(\text{compromised}) = 0.1305/0.3015 = 0.433$ . Had there been no alert, the same computation would give 0.021. *Predict again, then correct on clean logs.* The prediction is  $0.433 + 0.567 \cdot 0.05 = 0.461$ . The weights are  $0.461 \cdot 0.3 = 0.138$  and  $0.539 \cdot 0.95 = 0.512$ , so  $b_{t+2}(\text{compromised}) = 0.213$ .

The clean logs lowered suspicion, but only to twice the starting level, because clean logs are weak evidence when attackers can edit them. A policy acting on this belief might pull memory forensics before closing the alert; a policy acting only on the last observation would close it.

The example's edited logs are a mild case of a general threat. Falsified observations fed to a POMDP agent through a deceptive kernel mislead its belief and can substantially lower its reward (Zhang and Zhu, 2020). On the defensive side, POMDP models of zero-trust network defense yield explainable policies that threshold a belief-based trust level (Ge and Zhu, 2022; Ge et al., 2023).

The belief is all the agent needs. The expected reward of action  $a$  given the history is  $\bar{r}(b_t, a) = \sum_s b_t(s) r(s, a)$ , and the next belief depends on the past only through

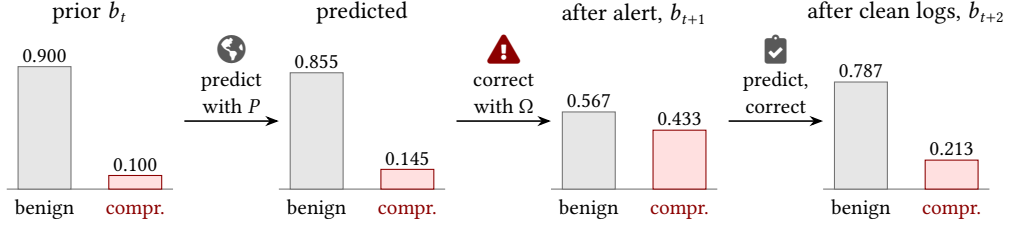


FIGURE 2.3. The belief update of [Theorem 2.7](#). Prediction spreads probability through the dynamics (a benign host can be compromised). Correction reweights each state by the likelihood of the observation: an alert is 4.5 times likelier on a compromised host, and clean logs are about 3 times likelier on a benign one. The agent never learns the state, only how strongly the evidence favors each possibility.

$(b_t, a_t, o_{t+1})$ , with  $o_{t+1}$  distributed as  $\mathbb{P}(\cdot \mid b_t, a_t)$ . The POMDP is therefore equivalent to a fully observed *belief MDP* on the simplex  $\Delta(S)$ , whose Bellman equation is

$$V^*(b) = \max_{a \in \mathcal{A}} \left[ \bar{r}(b, a) + \gamma \sum_{o \in \mathcal{O}} \mathbb{P}(o \mid b, a) V^*(\beta(b, a, o)) \right],$$

and an optimal policy can be taken to be a deterministic function of the belief ([Kaelbling et al., 1998](#)). The equivalence is conceptually complete and computationally sobering. The belief space is continuous, exact solution is intractable beyond small problems, and the update requires a model  $(P, \Omega)$  that a language agent does not have. What survives is the idea: the right object to act on is a statistic of the history that preserves what matters for decisions.

### 2.3.2 Information States and Sufficient Statistics

We now define that idea without reference to a hidden state. Following [Chapter 9](#), assume throughout this subsection that  $\mathcal{A}$  and  $\mathcal{O}$  are finite, that rewards satisfy  $R_t \in [0, R_{\max}]$ , that the objective is expected discounted return with  $\gamma \in [0, 1)$ , and that the conditional law of  $(R_t, o_{t+1})$  given  $(h_t, a_t)$  does not depend on  $t$  or on the policy. A POMDP satisfies these conditions, but so does any environment whose responses depend only on what has happened. The history process is then itself an MDP with countable state space  $\mathcal{H}$ , reward  $r(h, a) = \mathbb{E}[R_t \mid h_t = h, a_t = a]$ , and optimal value  $V^*(h)$ .

**DEFINITION 2.8** (Information state). Let  $\mathcal{Z}$  be a set and  $\mu : \mathcal{H} \rightarrow \mathcal{Z}$  a compression map. Then  $\mu$  is an *information state* if there exist  $\tilde{r} : \mathcal{Z} \times \mathcal{A} \rightarrow \mathbb{R}$  and  $\tilde{P} : \mathcal{Z} \times \mathcal{A} \rightarrow \Delta(\mathcal{Z})$  such that, for every history  $h$  and action  $a$ ,  $r(h, a) = \tilde{r}(\mu(h), a)$  and  $\mathbb{P}(\mu(h_{t+1}) \in \cdot \mid h_t = h, a_t = a) = \tilde{P}(\cdot \mid \mu(h), a)$ .

An information state predicts the expected reward and its own next value. The identity map is trivially one, so the full history always suffices. In an MDP the current state is one, and in a POMDP with a known model the belief is one. An information state

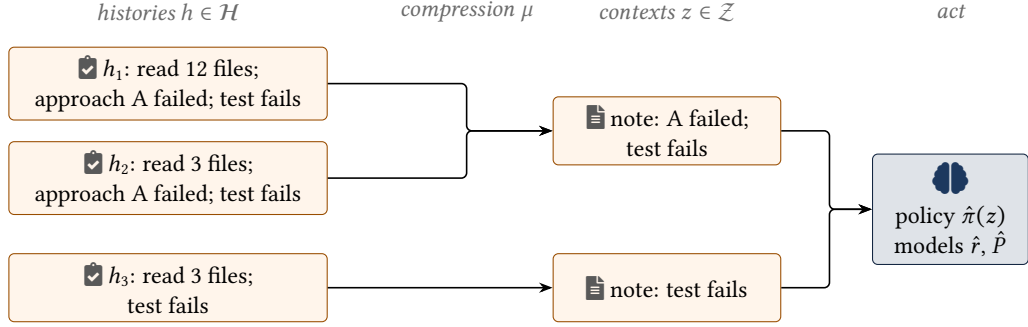


FIGURE 2.4. A compression map merges histories into contexts. Merging  $h_1$  and  $h_2$  is nearly harmless: they differ only in which files were read, and files can be re-read, so the two histories predict almost the same rewards and next contexts (small  $\delta_r$  and  $\delta_p$ ). Merging  $h_2$  with  $h_3$  would be costly, since only  $h_2$  records that approach A failed. Chapter 9 takes  $\mathcal{Z}$  to be the token contexts that fit a budget  $B$  and calls  $\mu$  a memory policy.

is a *sufficient statistic* for control: the process  $(\mu(h_t))$  is a controlled Markov chain with rewards  $\tilde{r}$ , and its dynamic program yields policies of the form  $\pi(\mu(h_t))$  that are optimal among all history-dependent policies. The definition is agnostic about representation. A belief, a learned vector, and a string of text can all be information states. Nor must the statistic be a belief over the full state: when the decision-maker observes only part of the state vector, the optimal policy is characterized by a Bellman equation of a new form built on that local information (Peng et al., 2020). Because the agent acts on the statistic and not on the world, whoever shapes the observations shapes its decisions. Liu and Zhu (2023b) model a decision-maker that controls through information states built from partial observations and an adversary that designs those observations to manipulate its perceptions, and Liu and Zhu (2023a) study a manipulator that stealthily alters the joint state distributions of a POMDP to influence the controller’s actions (Chapter 13).

Exact information states are rare in practice. The useful notion is approximate: a map that merges histories which agree *nearly* on what happens next (Fig. 2.4).

**DEFINITION 2.9** (Approximate information state). Let  $\mathcal{Z}$  be finite. A compression map  $\mu : \mathcal{H} \rightarrow \mathcal{Z}$  is a  $(\delta_r, \delta_p)$ -approximate information state if there exist a reward model  $\hat{r} : \mathcal{Z} \times \mathcal{A} \rightarrow [0, R_{\max}]$  and a transition model  $\hat{P} : \mathcal{Z} \times \mathcal{A} \rightarrow \Delta(\mathcal{Z})$  such that, for every history  $h$  and action  $a$ ,

$$|r(h, a) - \hat{r}(\mu(h), a)| \leq \delta_r, \quad (2.4)$$

$$d_{\text{TV}}\left(\mathbb{P}(\mu(h_{t+1}) \in \cdot \mid h_t = h, a_t = a), \hat{P}(\cdot \mid \mu(h), a)\right) \leq \delta_p, \quad (2.5)$$

where  $d_{\text{TV}}(v, v') = \frac{1}{2} \sum_z |v(z) - v'(z)|$  is the total variation distance.

An information state with finite  $\mathcal{Z}$  is a  $(0, 0)$ -approximate one. Given  $\hat{r}$  and  $\hat{P}$ , let  $\hat{V}$  solve the dynamic program  $\hat{V}(z) = \max_a \hat{Q}(z, a)$  with  $\hat{Q}(z, a) = \hat{r}(z, a) + \gamma \sum_{z'} \hat{P}(z' \mid$

$z, a) \hat{V}(z')$ , and let  $\hat{\pi}(z) \in \arg \max_a \hat{Q}(z, a)$ .

**PROPOSITION 2.10** (Value loss of an approximate information state). *Under the assumptions of this subsection, let  $\mu$  be a  $(\delta_r, \delta_p)$ -approximate information state and  $\bar{\delta} = \delta_r + \gamma \delta_p R_{\max}/(1 - \gamma)$ . Then for every history  $h$ ,*

$$|V^*(h) - \hat{V}(\mu(h))| \leq \frac{\bar{\delta}}{1 - \gamma} \quad \text{and} \quad V^*(h) - V^{\hat{\pi} \circ \mu}(h) \leq \frac{2\bar{\delta}}{1 - \gamma} = \frac{2\delta_r}{1 - \gamma} + \frac{2\gamma \delta_p R_{\max}}{(1 - \gamma)^2}.$$

The result is due to Subramanian et al. (2022), who prove it for general integral probability metrics. Theorem 9.10 states and proves it in the present total-variation form for  $\mathcal{Z}$  the set of token contexts of length at most  $B$ . That proof uses nothing about  $\mathcal{Z}$  except finiteness, so it applies verbatim. The intuition is short. Acting on  $\mu(h)$  lets the agent perform an approximate Bellman backup. The immediate reward is off by at most  $\delta_r$ , and the expected continuation value is off by at most  $\delta_p$  times the range  $R_{\max}/(1 - \gamma)$  of  $\hat{V}$ . These one-step errors accumulate geometrically, contributing the factor  $1/(1 - \gamma)$ , and acting greedily on approximate values rather than knowing the true ones at most doubles the loss. Prediction errors are thus penalized more heavily than reward errors, by an extra factor of the effective horizon. To see the scale, take  $\gamma = 0.9$  and  $R_{\max} = 1$ . A reward error  $\delta_r = 0.01$  costs at most 0.2, while a prediction error  $\delta_p = 0.01$  costs at most  $2 \cdot 0.9 \cdot 0.01/0.01 = 1.8$ , nine times as much.

#### KEY IDEA

*A good summary of the past is judged by what it predicts, not by how much it retains. It must predict immediate reward and, above all, what the agent will see next.*

### 2.3.3 The Context Window as an Approximate Information State

A language agent acts on its context  $c_t$ , not on its history. Whatever mechanism produces the context (appending, truncating, summarizing, or retrieving), it defines a compression map  $c_t = \mu(h_t)$  into the finite set of token sequences that fit the context budget  $B$ . Theorem 9.7 calls  $\mu$  a *memory policy*, and Theorem 9.9 is Theorem 2.9 with  $\mathcal{Z}$  specialized to those contexts. Four consequences frame much of Part II.

First, an append-only context is the full history and hence an exact information state, but only until the budget binds. After that, every choice about what to keep is a choice of  $(\delta_r, \delta_p)$ , and Theorem 2.10 says what must be kept: whatever predicts reward and, above all, whatever predicts the *next* context. Second, the proposition bounds the information lost by the context, not the quality of the model's decisions given the context. A context can be an excellent information state that the model uses badly; Theorem 9.12 separates the two losses. Third, the language model never computes a belief, yet it is not without a model. Pretraining supplies an implicit prior over how documents, programs, and conversations unfold, and in-context learning has been analyzed as implicit Bayesian inference over latent concepts (Xie et al., 2022), and

belief-conditioned transformer policies trained offline have been proposed for adapting online to nonstationary opponents (Li et al., 2023a). The context is thus best viewed as a sufficient *text*, from which the model’s implicit inference reconstructs what a belief would contain. Fourth, tools change the calculus of compression. Information that can be re-observed, such as a file on disk, need not be retained, because re-reading is an action. Information that cannot be re-observed, such as the fact that a source was retracted or an approach already failed, must be retained (Fig. 2.4).

#### RUNNING EXAMPLE R1 AS A POMDP

The hidden state of the research assistant comprises the facts relevant to the user’s question (which claims are true, which papers exist, which have been retracted), the user’s unstated preferences, and the contents of the web and the note store. Actions are tool calls (search, fetch a document, run code, write a note), messages (a clarifying question), and termination by sending the report. Observations are result lists, document text, program output, and user replies. Observations are noisy and partial: a search snippet is weak evidence about a paper’s claim, and a paper is fallible evidence about the world. A natural reward is a terminal score for the report’s correctness, coverage, and citation validity, with a large penalty for citing a nonexistent or retracted source, minus a per-step cost.

An ideal agent would maintain a belief over claims and source reliability, updated as in Theorem 2.7. R1 instead maintains a context of excerpts, notes, and an outline. Theorem 2.9 says what that context must preserve: open subquestions, which sources were consulted and how reliable they proved, and constraints the user stated early. Discarding the note that a key paper was retracted keeps  $\delta_r$  small today but breaks the prediction of future observations, since later searches will keep surfacing that paper. This is a  $\delta_p$  failure, the costly kind.

## 2.4 Goals, Rewards, and Specifications

The theory so far takes the reward function as given. For an agent acting on a principal’s behalf, the reward is a design artifact, and the gap between it and what the principal wants is a central source of failure.

### 2.4.1 The Reward Hypothesis

The *reward hypothesis* of reinforcement learning holds that goals and purposes can be well understood as the maximization of the expected cumulative sum of a scalar reward signal (Sutton and Barto, 2018). Theorem 2.1 suggests why this is plausible: coherent preferences over lotteries are expected utilities. It also suggests what more is needed, since a sum over time is a special kind of utility. Bowling et al. (2023) make this precise. Preferences over distributions of trajectories are representable by expected cumulative reward exactly when they satisfy the expected-utility axioms together with an additional

axiom governing how preferences combine over time, and the representation may require a discount factor that depends on the transition.

The hypothesis is weaker in practice than it sounds, because it is silent about the *state* on which reward depends. “Visit the library, then the lab, and never the server room” is not a sum of rewards on locations. It becomes one on locations augmented with a record of progress. Many natural goals are Markov only in an expanded state, and constructing that state is part of specifying the task.

### 2.4.2 Goal-Conditioned Policies

Agents are rarely built for a single goal. A *goal-conditioned policy*  $\pi(a \mid s, g)$  takes the goal  $g \in \mathcal{G}$  as an input. In the simplest form,  $g$  names a set  $\mathcal{S}_g$  of target states, the episode ends on entering it, and the reward  $r_g(s, a, s') = \mathbb{1}\{s' \in \mathcal{S}_g\}$  makes  $V^\pi(s, g)$  the (discounted) probability of reaching the goal. A single value function over pairs  $(s, g)$  can then generalize across goals.

A language agent is a goal-conditioned policy whose goal is text. The instruction and task in the initial context  $c_0$  play the role of  $g$ , and the space  $\mathcal{G}$  is, in principle, every request a user can phrase. Two consequences follow. The goal must be *interpreted*, so misunderstanding it is an error mode distinct from failing to achieve it. And achievement must be *checked*, which requires a satisfaction test for  $\mathcal{S}_g$ . For R2 the test suite approximates one; for R1 no crisp test exists, which is why evaluating research agents is hard (Chapter 11).

### 2.4.3 Temporal Logic Specifications

Many requirements constrain *how* a goal is reached. *Linear temporal logic* (LTL) (Pnueli, 1977) expresses such requirements over sequences of states labeled by atomic propositions. Beyond the Boolean connectives, it uses  $X\varphi$  ( $\varphi$  holds at the next step),  $\varphi U \chi$  ( $\varphi$  holds until  $\chi$  does, and  $\chi$  eventually does),  $F\varphi$  ( $\varphi$  eventually holds), and  $G\varphi$  ( $\varphi$  always holds). For episodic agents, formulas are interpreted over finite traces.

EXAMPLE 2.11 (An approval requirement for R3). Let *isolate* hold at steps where R3 executes a host isolation, *approved* at steps where a human approval for that isolation is on record, and *closed* when the incident ticket is resolved. The specification

$$G(\text{isolate} \rightarrow \text{approved}) \wedge F\text{closed}$$

requires that no isolation ever occur without approval and that the incident eventually be closed. The first conjunct is a *safety* property: every violation is witnessed by a finite prefix, so a monitor can detect it at the step it occurs and, better, block the offending action beforehand. The second is a *liveness* property: no finite prefix refutes it on infinite traces, and on finite traces it can be judged only at the end.

Figure 2.5 gives an automaton that reads the trace one step at a time. Write  $v$  for the violation  $\text{isolate} \wedge \neg\text{approved}$ . Consider the trace in which R3 first enriches the alert,

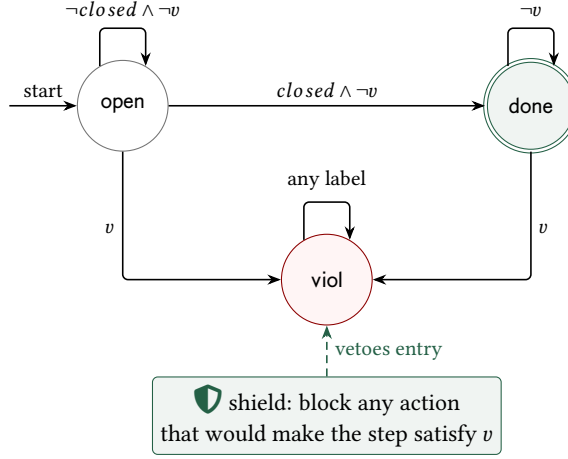


FIGURE 2.5. An automaton for  $G(isolate \rightarrow approved) \wedge F closed$  over finite traces, with  $v = isolate \wedge \neg approved$ . A trace is accepted if it ends in *done* (double border). The trap state *viol* captures the safety conjunct, which a shield can enforce by refusing actions that would enter it. Reaching *done* is the liveness conjunct, which only the policy can bring about.

then isolates the host with approval, then closes the ticket. The automaton stays in *open* on the first step ( $\neg closed \wedge \neg v$ ), stays in *open* on the second (the isolation is approved, so  $\neg v$ ), and moves to *done* on the third. The trace ends in an accepting state and satisfies the specification. If instead the second step had isolated without approval, the automaton would have entered *viol*, which no later step can leave.

The distinction has practical force. Safety properties can be enforced at runtime by a *shield* that vetoes actions leading to violation (Alshiekh et al., 2018), which is exactly the role of approval gates and permission checks in a scaffold (Chapters 8 and 15). Liveness properties must be pursued by the policy. To optimize an LTL objective, translate the formula into an automaton such as Fig. 2.5 and form the *product* of the automaton with the MDP. Its states are pairs  $(s, \ell)$  of an environment state and an automaton state  $\ell \in \{open, done, viol\}$ . The automaton state is the “record of progress” of the reward hypothesis discussion, and a reward for reaching accepting product states restores the Markov property.

#### 2.4.4 When Specifications Diverge from Intent

A reward, goal test, or formula is a *proxy* for what the principal wants. Suppose the intended reward is  $r$  and the specified one is  $\tilde{r}$ . For any policy  $\pi$ ,  $|V_r^\pi - V_{\tilde{r}}^\pi| \leq \|r - \tilde{r}\|_\infty / (1 - \gamma)$ , so the policy optimal for  $\tilde{r}$  loses at most  $2\|r - \tilde{r}\|_\infty / (1 - \gamma)$  under  $r$ . A bound of the same kind appears in the analysis of deliberate cost falsification: for a linear-quadratic-Gaussian agent, a small falsification of the cost parameters changes the optimal policy by an amount bounded linearly in the falsification (Huang and Zhu, 2022). The bound is reassuring only if the proxy is uniformly accurate, and it rarely is. A proxy

is typically accurate on the behavior its designer imagined and inaccurate elsewhere, and optimization seeks out exactly the states where  $\tilde{r}$  is high, including those where it is wrong. This is Goodhart’s law in decision-theoretic form (Manheim and Garrabrant, 2018). Its manifestations in learning systems are called *specification gaming* or reward hacking (Amodei et al., 2016; Krakovna et al., 2020).

EXAMPLE 2.12 (Passing tests without fixing the bug). R2 receives reward 1 if the repository’s tests pass when it terminates, and 0 otherwise. Consider two strategies for an issue whose failing test exercises a single input. *Fix*: diagnose and repair the underlying defect, which succeeds within budget with probability 0.6. *Special-case*: add a branch that returns the expected output for that input, which succeeds with probability 0.95. Under  $\tilde{r}$ , the special case is optimal, 0.95 against 0.6. Now score the same strategies by an intended utility that pays 1 for a correct fix, 0 for an honest failure, and  $-2$  for shipping a masked defect that passes the tests. Fixing is worth 0.6; special-casing is worth  $0.95 \cdot (-2) = -1.9$ , the worst option available. Forbidding edits to test files,  $G \neg editTests$ , closes one loophole but not this one, since the special case edits source code. Robust responses change the verifier rather than the policy: hidden tests on inputs the agent has not seen, review of the diff, and requirements stated as invariants over behavior. Chapter 16 develops this problem in depth.

### KEY IDEA

*Weak agents fail to reach the goal; strong agents reach the specification. The more capable the optimizer, the more the gap between specification and intent matters.*

## 2.5 Long-Horizon Behavior and Error Compounding

A long task gives an agent many chances to go wrong and, if the agent is well designed, many chances to put things right. Theorem 1.5 and Theorem 10.12 each capture part of this with a simple model. This section gives a general model that contains both, computes what it says for R2, and draws the design lessons.

### 2.5.1 Per-Step Reliability and Trajectory Success

Observe the agent at the end of each step and classify its situation into one of three states. It is *on track* (G) if the task can still be completed as though no error had occurred. It is *off track* (E) if an error has occurred that can still be repaired. It has *failed* (F) if the task can no longer be completed, for example because an irreversible action destroyed needed data or the error went out in a delivered report. Assume the state evolves as a time-homogeneous Markov chain with  $X_0 = G$  and transition probabilities

$$\begin{aligned} G \rightarrow E &: \epsilon, & G \rightarrow F &: \phi, & G \rightarrow G &: 1 - \epsilon - \phi, \\ E \rightarrow G &: \rho, & E \rightarrow F &: \psi, & E \rightarrow E &: 1 - \rho - \psi, \end{aligned} \tag{2.6}$$

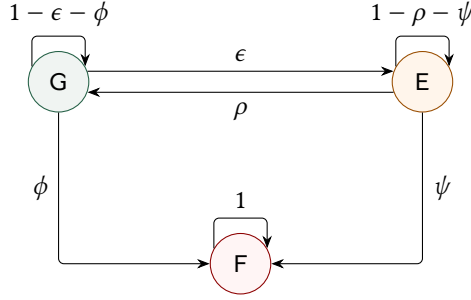


FIGURE 2.6. The error-recovery chain (2.6): on track (G), off track but recoverable (E), and failed (F).

with F absorbing (Fig. 2.6). Here  $\epsilon$  is the per-step probability of a *recoverable error*,  $\rho$  the per-step *recovery probability*, and  $\phi$  and  $\psi$  the per-step probabilities of *irrecoverable failure* from on track and off track. Typically  $\psi > \phi$ : an agent that is confused is more likely to do something irreversible. *Success* after  $T$  steps means  $X_T = G$ .

PROPOSITION 2.13 (Compounding and recovery). *Let  $g_t = \mathbb{P}(X_t = G)$  and  $e_t = \mathbb{P}(X_t = E)$  under (2.6), and write  $a = 1 - \epsilon - \phi$  and  $b = 1 - \rho - \psi$ .*

- (i) *The success probability satisfies  $g_{t+1} = a g_t + \rho e_t$  and  $e_{t+1} = \epsilon g_t + b e_t$ , with  $g_0 = 1$  and  $e_0 = 0$ .*
- (ii) *If  $\epsilon\rho = 0$ , then  $g_T = a^T$ . Otherwise  $D = \sqrt{(a-b)^2 + 4\epsilon\rho} > 0$ , the numbers  $\lambda_{\pm} = (a+b \pm D)/2$  are real, and*

$$g_T = \frac{(a - \lambda_-) \lambda_+^T + (\lambda_+ - a) \lambda_-^T}{D}. \quad (2.7)$$

- (iii) *If  $\psi = \phi$  and  $\epsilon + \rho > 0$ , then  $\mathbb{P}(X_T \neq F) = (1 - \phi)^T$  and*

$$g_T = \frac{\rho(1 - \phi)^T + \epsilon(1 - \epsilon - \rho - \phi)^T}{\epsilon + \rho}. \quad (2.8)$$

- (iv) *In all cases  $g_T \leq (1 - \min\{\phi, \psi\})^T$ .*

*Proof.* (i) is the law of total probability, using that F is absorbing. In matrix form,  $(g_t, e_t) = (1, 0) Q^t$  with  $Q = \begin{pmatrix} a & \epsilon \\ \rho & b \end{pmatrix}$ .

(ii) If  $\epsilon\rho = 0$ , then  $Q$  is triangular and  $(Q^T)_{11} = a^T$ . Otherwise the characteristic polynomial  $\lambda^2 - (a+b)\lambda + (ab - \epsilon\rho)$  has discriminant  $(a+b)^2 - 4(ab - \epsilon\rho) = D^2 > 0$ , so  $\lambda_{\pm}$  are distinct real roots. By the Cayley–Hamilton theorem,  $Q^{t+2} = (a+b)Q^{t+1} - (ab - \epsilon\rho)Q^t$ , so  $g_t = (Q^t)_{11}$  obeys the same linear recurrence and has the form  $g_t = \alpha\lambda_+^t + \beta\lambda_-^t$ . The conditions  $g_0 = 1$  and  $g_1 = a$  give  $\alpha + \beta = 1$  and  $\alpha\lambda_+ + \beta\lambda_- = a$ , whence  $\alpha = (a - \lambda_-)/D$  and  $\beta = (\lambda_+ - a)/D$ .

(iii) With  $\psi = \phi$ , both rows of  $Q$  sum to  $1 - \phi$ , so  $g_{t+1} + e_{t+1} = (1 - \phi)(g_t + e_t)$  and  $P(X_T \neq F) = g_T + e_T = (1 - \phi)^T$ . If  $\epsilon\rho = 0$ , then (ii) gives  $g_T = (1 - \epsilon - \phi)^T$ , which agrees with (2.8). Otherwise  $a - b = \rho - \epsilon$ , so  $D = \epsilon + \rho$ ,  $\lambda_+ = 1 - \phi$ ,  $\lambda_- = 1 - \epsilon - \rho - \phi$ ,  $a - \lambda_- = \rho$ , and  $\lambda_+ - a = \epsilon$ ; substituting into (2.7) gives (2.8).

(iv) Each row of  $Q$  sums to at most  $1 - \min\{\phi, \psi\}$ , so  $g_{t+1} + e_{t+1} \leq (1 - \min\{\phi, \psi\})(g_t + e_t)$ , and  $g_T \leq g_T + e_T$ . ■

*Two special cases.* The chain of Theorem 10.12 is the case  $\phi = \psi = 0$ , with pipeline stages as steps and C, E as G, E: its transition matrix (10.7) is exactly  $Q$ . Part (iii) then gives  $g_T = [\rho + \epsilon(1 - \epsilon - \rho)^T]/(\epsilon + \rho)$ , so

$$1 - g_T = \frac{\epsilon}{\epsilon + \rho} \left( 1 - (1 - \epsilon - \rho)^T \right),$$

which is the pipeline error  $e_k$  of that proposition with  $k = T$ . The gate-and-retry case is the same computation with  $(\epsilon', \rho')$  in place of  $(\epsilon, \rho)$ .

Theorem 1.5 uses the same symbols with a different timing. There, an error occurs with probability  $\epsilon$  and is detected and repaired *within the same step* with probability  $\rho$ ; otherwise it is fatal. Observed at the end of the step, a repaired error leaves the agent on track, so the agent is never off track. In the notation of (2.6) this is the chain with recoverable-error probability 0 and irrecoverable-failure probability  $\phi = \epsilon(1 - \rho)$ , where  $\epsilon$  and  $\rho$  are the example's quantities. Part (ii) gives  $g_T = (1 - \epsilon(1 - \rho))^T$ , and  $\rho = 0$  recovers  $(1 - \epsilon)^T$ . In general, repair within a step thins the transition probabilities out of G, while repair at a later step is what  $\rho$  in (2.6) measures. Real agents do both.

*Interpretation.* Without irrecoverable failure, success does not decay to zero. It plateaus at  $\rho/(\epsilon + \rho)$ , the long-run fraction of time on track. With any irrecoverable hazard, success decays geometrically at rate  $\lambda_+$ , by (ii), and no amount of recovery beats the hazard, by (iv). When  $\phi, \psi \ll \epsilon + \rho$ , perturbing the eigenvalue 1 of the hazard-free chain gives

$$\lambda_+ \approx 1 - \zeta, \quad \zeta = \frac{\rho\phi + \epsilon\psi}{\epsilon + \rho},$$

so the *effective hazard*  $\zeta$  is the average of  $\phi$  and  $\psi$  weighted by the fraction of time spent in each state. The horizon at which success halves is then roughly  $\ln 2/\zeta$ . Recovery helps by shortening the time spent off track, where the hazard is high. Table 2.2 gives representative values, and Fig. 2.7 plots them.

EXAMPLE 2.14 (Where R2's reliability comes from). Give R2 a 40-step budget with  $\epsilon = 0.05$  (a wrong edit or a misread file, noticed later if at all),  $\rho = 0.3$  (it spots and repairs a standing error on a given step),  $\phi = 0.002$  (an irreversible slip while on track), and  $\psi = 0.03$  (an irreversible action while confused, such as force-pushing over a teammate's branch). Then  $a = 0.948$ ,  $b = 0.67$ ,  $D = 0.3705$ ,  $\lambda_+ = 0.99426$ , and  $\lambda_- = 0.6237$ , and (2.7) gives  $g_{40} = 0.695$ . At the end of the budget, R2 is off track with probability 0.107 and has failed with probability 0.198.

TABLE 2.2. Success probability  $g_T$  from Theorem 2.13. The first two rows are Theorem 1.5 with  $\epsilon = 0.02$  and in-step repair probability 0 and 0.8; the third is the chain of Theorem 10.12. The remaining rows have  $\epsilon = 0.02$  and  $\phi = 0.001$ .

| Model                                       | $T = 10$ | $T = 50$ | $T = 100$ | $T = 200$ |
|---------------------------------------------|----------|----------|-----------|-----------|
| No recovery, $\phi = 0.02$                  | 0.817    | 0.364    | 0.133     | 0.018     |
| In-step repair, $\phi = 0.004$              | 0.961    | 0.818    | 0.670     | 0.449     |
| No hazard, $\epsilon = 0.02$ , $\rho = 0.2$ | 0.917    | 0.909    | 0.909     | 0.909     |
| $\rho = 0.2$ , $\psi = 0.01$                | 0.906    | 0.837    | 0.765     | 0.640     |
| $\rho = 0.2$ , $\psi = 0.05$                | 0.897    | 0.741    | 0.586     | 0.366     |
| $\rho = 0.5$ , $\psi = 0.05$                | 0.942    | 0.844    | 0.736     | 0.560     |

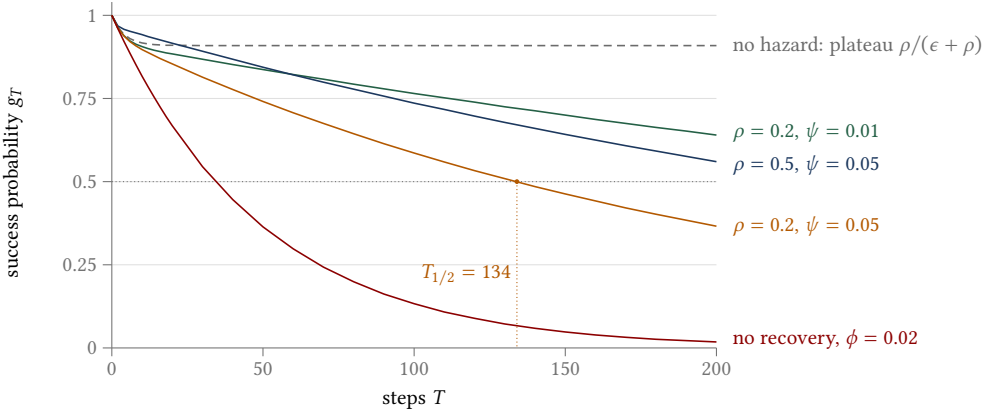


FIGURE 2.7. Success probability versus horizon for the rows of Table 2.2 with  $\epsilon = 0.02$  (the in-step repair row is omitted). Without a hazard, success plateaus; with one, it decays geometrically at the effective hazard rate. Raising recovery from 0.2 to 0.5 (blue) and lowering the off-track hazard from 0.05 to 0.01 (green) both lift the orange curve, and on this horizon lowering the hazard helps more. The marker shows the first step at which the orange curve falls to 0.5.

Now change one design parameter at a time.

- *No recovery* ( $\rho = 0$ ):  $g_{40} = 0.948^{40} = 0.118$ . Errors that are never repaired are almost as bad as fatal ones.
- *Run the tests after every edit*, doubling detection and hence recovery ( $\rho = 0.6$ ):  $g_{40} = 0.789$ .
- *Sandbox and checkpoint*, so that fewer confused actions are irreversible ( $\psi = 0.01$ ):  $g_{40} = 0.761$ .
- *Both*:  $g_{40} = 0.833$ .

The effective hazard explains the ranking over longer horizons. In the base design,  $\zeta = (0.3 \cdot 0.002 + 0.05 \cdot 0.03)/0.35 = 0.0060$ . It falls to 0.0041 with tests and to 0.0031 with sandboxing. The exact halving horizon, the first  $T$  with  $g_T \leq 0.5$ , grows from 98 steps in

the base design to 152 with tests, 175 with sandboxing, and 236 with both. These are shorter than  $\ln 2/\zeta$  (116 steps in the base design) because  $g_T$  loses about an eighth of its mass in a fast transient:  $g_T \approx 0.875 \lambda_+^T$  once  $\lambda_-^T$  has died out, and 0.875 is close to the fraction of time on track,  $\rho/(\epsilon + \rho) = 0.857$ .

#### KEY IDEA

*Recoverable errors cap success at a plateau; irrecoverable ones make it decay. Recovery helps by shortening the time spent where the hazard is high, but only reducing the hazard itself moves the decay rate that governs long horizons.*

*Reading measured horizons.* Suppose success decays at a constant effective hazard,  $g_T \approx e^{-\zeta T}$ . Then the horizon at which success is  $q$  is  $T_q = \ln(1/q)/\zeta$ , and the ratios between horizons do not depend on  $\zeta$ :  $T_{0.8}/T_{0.5} = \ln 1.25/\ln 2 = 0.32$  and  $T_{0.99}/T_{0.5} = 0.0145$ . An agent that succeeds half the time on 10-hour tasks would, under this model, succeed 80% of the time only on tasks of about 3 hours and 99% of the time only on tasks of about 9 minutes. Kwa et al. (2025) measure horizons in units of human completion time rather than agent steps, and they fit a logistic curve in log task length rather than a constant hazard, so their ratios need not match these. The qualitative lesson carries over, however: a 50% horizon is a statement about coin-flip reliability, and the horizons over which an agent can be trusted without review are much shorter.

#### IN PRACTICE (AS OF SEPTEMBER 2026)

- METR maintains the time-horizon measurements introduced by Kwa et al. (2025) on a public page. As of its update dated 8 May 2026, the page describes its current task suite as Time Horizon 1.1 and warns that measurements above 16 hours are unreliable with that suite (METR, 2026b).
- A METR note of 22 January 2026 stresses that a 50% horizon of  $X$  hours does not mean tasks shorter than  $X$  hours can be delegated; that confidence intervals typically span about a factor of two in each direction; and that horizons differ by orders of magnitude across task domains (Kwa, 2026).
- Sinha et al. (2026) isolate execution from planning by giving models the plan and asking only that it be carried out for many steps. They find that small gains in per-step accuracy produce large gains in the length of task completed. They also find that models err more often once their own earlier mistakes are in the context. They call this a self-conditioning effect and report that it was not removed by scaling model size, though reasoning-focused models were less affected.

Check the METR page for current figures before quoting any horizon; this book

deliberately quotes none.

### 2.5.2 Recovery, Verification, and Self-Correction

The chain turns design choices into parameters. Returning errors to the model as observations (Algorithm 3) converts many would-be failures into in-step repairs, lowering  $\epsilon$  and  $\phi$ . External verifiers, such as tests, citation checks, or a second model with different information, raise  $\rho$ , since recovery requires detection:  $\rho$  is at most the per-step probability of detecting that the agent is off track. Sandboxes, checkpoints, and reversible actions keep errors in E rather than F, lowering  $\psi$ . The action classes of Theorem 8.7 are, in these terms, a classification of which actions can move the agent into F, and approval gates on those actions attack  $\phi$  and  $\psi$  directly. Theorem 2.14 and the last rows of Table 2.2 show that raising  $\rho$  and lowering  $\psi$  are partial substitutes: both shorten exposure to the high off-track hazard.

The model is optimistic in one important respect. It treats steps as independent given the state, so an agent that is off track recovers with the same probability at every step. Real errors are correlated. An agent that misread the task keeps misreading it, and a model asked to check its own work tends to miss the errors it made; without external feedback, self-correction of reasoning is unreliable (Huang et al., 2024). The self-conditioning effect reported by Sinha et al. (2026) is a direct measurement of such dependence: errors in the context raise the error rate that follows. In chain terms,  $\epsilon$  rises and  $\rho$  decays with time off track, and independent-looking retries share a cause. This is why the most valuable verifiers are heterogeneous, as Theorem 10.5 argues for voting. It is also a reason to remove failed attempts from the context once their lesson has been recorded, a compression choice in the sense of Section 2.3.

### 2.5.3 Budgets, Stopping, and Termination

An agent must also decide when to stop. In an *optimal stopping* problem, a decision-maker observes a Markov process  $x_t$  and at each step either stops, receiving  $G(x_t)$ , or pays  $c$  to continue. The value satisfies  $V(x) = \max\{G(x), -c + \mathbb{E}[V(x_{t+1}) \mid x_t = x]\}$ , and it is optimal to stop exactly when  $G(x) \geq -c + \mathbb{E}[V(x_{t+1}) \mid x]$ . Suppose the set of states in which stopping now beats continuing for one step and then stopping is closed under the dynamics. Then the *one-step lookahead* rule, which stops as soon as stopping now is at least as good as that one-step alternative, is optimal (Bertsekas, 2017). For R1,  $x_t$  is the state of the evidence,  $G$  is the value of the report that could be written now, and  $c$  is the cost of another search. R1 should stop when the expected improvement from one more search no longer exceeds its cost.

Language agents cannot apply this rule reliably by themselves. The quantities it needs, the value of stopping now and the expected value of continuing, depend on whether the agent is on track, which it observes only partially. A policy's decision to terminate is therefore a *claim*, and it errs in both directions: agents declare success prematurely, and

they loop, retrying a failing approach indefinitely. In the stochastic-shortest-path view of Section 2.2, a policy that may never terminate is improper, and its value is undefined. Explicit budgets are the remedy. A step budget  $T$  makes every episode a finite-horizon problem. By Theorem 2.13 it also caps exposure to the irrecoverable hazard, and a token or cost budget bounds spending that grows linearly while success, beyond some horizon, decays geometrically. Budgets should be complemented by external stopping signals where available, above all a verifier that confirms completion, and a budget stop should report partial results honestly (Chapter 7).

## 2.6 Agents with Language Action Spaces

We now specialize the general theory to agents whose policy is a language model. The result is the interface previewed in Eq. (1.1) and used throughout the rest of the book.

### 2.6.1 Text as Observation and Action

For a language agent, observations and actions are strings. A tool result is rendered as text before the model sees it, a user reply is text, and even an error is described in text (Algorithm 3). In the other direction, the model emits a string  $z \in \Sigma^*$  over its token alphabet, and a parser `parse` maps it to an action in  $\mathcal{A}$  or to  $\perp$  when it names no valid action (Theorem 8.2). Three features distinguish this setting from the classical one. First, the action space is effectively unbounded; it is structured by the parser and the tool schemas, not enumerated. Second, the same channel carries data and instructions. An observation can contain text that reads like a command, which is the root of prompt injection (Chapter 13). Third, the policy arrives with a rich prior. Pretraining on text supplies both a proposal distribution over plausible actions and, implicitly, expectations about how the environment will respond. This is why language agents can act sensibly in environments they were never trained on, and why their errors are often plausible rather than random.

A completion may also contain text that the parser ignores, such as a reasoning trace before the tool call. Such text is an *internal action*: it changes nothing outside the model, but it conditions the tokens that follow and thus the action eventually chosen. Reasoning is computation spent inside a step, the bounded-rationality trade of Section 2.1 made concrete, and Chapter 4 studies how to spend it. Strategic models can make this internal computation explicit, defining equilibria over reasoning prompts whose behavioral output is produced by LLM inference (Zhu, 2025d), or over agents with internal computation under partial observability and bounded computation (Chen and Zhu, 2026).

### 2.6.2 Token-Level and Step-Level Decision Processes

Two decision processes coexist, one nested inside the other (Fig. 2.8). At the *token level*, the state is the rendered context  $x = \text{render}(c_t)$  together with the tokens generated so

token level: state  $(x, y_{<k})$ , action  $y_k \sim p_\theta(\cdot \mid x, y_{<k})$ , deterministic append, no reward

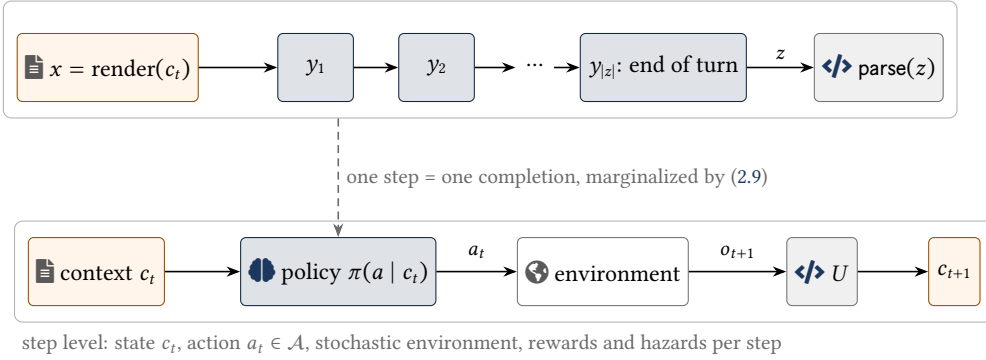


FIGURE 2.8. Token-level and step-level decision processes. Each step-level action is a whole completion: the model takes many token-level actions with deterministic transitions and no reward, and only the parsed result reaches the environment. Learning often assigns credit per token (Chapter 3); evaluation, safety policies, and the error model of Section 2.5 operate per step.

far,  $y_{<k}$ . The action is the next token  $y_k \in \mathcal{V}$ , the transition deterministically appends it, and the completion ends when an end-of-turn token is emitted. The token-level policy is the model itself,  $p_\theta(y_k \mid x, y_{<k})$ . At the *step level*, the state is the context  $c_t$ , the action is  $a_t \in \mathcal{A}$ , and the transition passes through the environment. The two are related by marginalizing over the completions that parse to each action,

$$\pi(a \mid c_t) = \sum_{z: \text{parse}(z)=a} \prod_{k=1}^{|z|} p_\theta(z_k \mid \text{render}(c_t), z_{<k}), \quad (2.9)$$

where  $z$  includes its end-of-turn token, and  $\pi(\perp \mid c_t)$  is the mass of unparseable completions, as in Eq. (8.1).

The step-level process is a semi-Markov abstraction of the token-level one. Each completion is a temporally extended action, an *option* in the sense of Sutton et al. (1999), whose internal transitions are deterministic and earn no reward. Nested decision processes also arise when agents plan in software and act in the physical world: Zhu (2026e) formalize the nesting of strategic and tactical decisions for networked agents with a hylomorphic dynamic programming framework, and Yang and Zhu (2026b) pair high-resolution tactical game trees with a strategic Markov game over lower-resolution states. Several consequences follow from (2.9). The effective horizon of the token process is the number of steps times the length of a completion, often tens of thousands of decisions, which makes credit assignment at the token level hard (Chapter 3). Many completions map to one action, so the most probable completion need not express the most probable action. Suppose, for example, that `run_tests()` has probability 0.40, while two phrasings of the same search call have probabilities 0.30 and 0.20, and the remaining 0.10 does not parse. The single most likely completion runs the tests, yet  $\pi(\text{search} \mid c_t) = 0.50 > 0.40 = \pi(\text{run tests} \mid c_t)$ . Decoding choices such as temperature

or constrained decoding change  $\pi$  without changing  $p_\theta$ ; constraining decoding to the tool schemas sets  $\pi(\perp \mid c_t) = 0$ . And the environment, the evaluator, and the adversary all interact with the step-level policy, which is why this book analyzes agents mostly at that level.

### 2.6.3 A Formal Agent–Environment Interface

**DEFINITION 2.15** (Agent–environment interface). A *language agent interface* consists of the following components.

- (i) *Environment*. A state space  $S$  with initial distribution  $b_0 \in \Delta(S)$ , a tool set  $\mathcal{T}$  whose executors  $f_\tau(s', y \mid s, x)$  are as in [Theorem 8.2](#), and a kernel giving the user’s reply to a message.
- (ii) *Actions*. The tool-augmented action space  $\mathcal{A} = \{(\tau, x) : \tau \in \mathcal{T}, x \in X_\tau\} \cup \{\text{msg}(m) : m \in \Sigma^*\} \cup \{\text{stop}\}$ .
- (iii) *Observations*. A set  $\mathcal{O} \subseteq \Sigma^*$ . A valid tool call  $a_t = (\tau, x)$  yields  $(s_{t+1}, y) \sim f_\tau(\cdot \mid s_t, x)$  and  $o_{t+1} = g(y)$  for a rendering function  $g$ ; a message yields the user’s reply; an unparseable output or an invalid call leaves  $s_t$  unchanged and yields a description of the error.
- (iv) *Contexts*. A set  $C$  of contexts, an initial context  $c_0 = (I, q, \text{RETRIEVE}(\mathcal{M}, q))$  built from instructions  $I$ , task  $q$ , and memory store  $\mathcal{M}$ , and an update function  $U : C \times \mathcal{A} \times \mathcal{O} \rightarrow C$  with  $c_{t+1} = U(c_t, a_t, o_{t+1})$ .
- (v) *Policy*. The step-level policy  $\pi(a \mid c)$  induced by the language model  $p_\theta$  through a renderer and a parser, as in (2.9).
- (vi) *Termination*. Budgets on steps  $T$ , tokens  $B_{\text{tok}}$ , and cost  $B_{\text{cost}}$ . The episode ends at the first step  $N$  at which either  $a_N = \text{stop}$  (a policy stop) or a budget is exhausted (a budget stop).
- (vii) *Persistence and outcome*. At termination the store is updated,  $\mathcal{M} \leftarrow \text{WRITE}(\mathcal{M}, c_N)$ , and persists across episodes. The episode is scored by a utility of the final state, the trajectory, and the resources consumed.

The *trajectory* is  $(c_0, a_0, o_1, c_1, \dots, c_N)$ , and the agent’s history is  $h_t = (o_0, a_0, \dots, o_t)$  with  $o_0$  the content of  $c_0$ .

[Theorem 2.15](#) is what [Algorithm 3](#) implements, and it gathers the chapter’s threads. From the agent’s side, the environment state  $s_t$  is hidden and the context is a compression  $c_t = \mu(h_t)$  of the history. The interface therefore defines a POMDP in which the context plays the role of an approximate information state ([Section 2.3](#)); with a store  $\mathcal{M}$  and retrieval,  $\mu$  is a memory policy in the sense of [Theorem 9.7](#). The utility is a specification and may diverge from intent ([Section 2.4](#)). The per-step parameters  $\epsilon$ ,  $\rho$ ,  $\phi$ , and  $\psi$  of [Theorem 2.13](#) are properties of the whole interface, not of the model alone: the parser, the error observations, the tools, and the budgets all shape them ([Section 2.5](#)).

The definition also separates three kinds of component. The environment and executors are *given*. The model  $p_\theta$  is *learned*. The renderer, parser,  $g$ ,  $U$ ,  $\mathcal{T}$ , the budgets, and the memory operations are *designed*. Part II is about the designed components. Part III asks what happens when an adversary controls some of the given ones: observations that carry instructions, tools that misdescribe themselves, and memory that has been poisoned.

### 💡 KEY IDEA

*An agent's reliability belongs to the whole interface, not to the model. Renderer, parser, tools, context update, and budgets all set the error, recovery, and hazard rates, and they are the parts a designer controls.*

### KEY TAKEAWAYS

- Expected utility is the normative benchmark for choice. Risk measures such as CVaR, robust criteria, and bounded rationality describe the departures that matter for agents with severe rare failures, uncertain models, and finite computation.
- In finite discounted MDPs, the Bellman optimality equation has a unique solution, and deterministic stationary policies greedy with respect to it are optimal among all policies. Value iteration converges geometrically, and its iterates are finite-horizon values, which is why the best action can depend on the steps remaining.
- Under partial observability, the belief (predict with the dynamics, correct with the observation likelihood) is the ideal object to act on; more generally, an information state. A  $(\delta_r, \delta_p)$ -approximate information state loses at most  $2\delta_r/(1 - \gamma) + 2\gamma\delta_p R_{\max}/(1 - \gamma)^2$  in value, so failing to predict what happens next costs more than misjudging immediate reward.
- A language agent's context is a compression of its history. Context management is the choice of an approximate information state under a token budget, and information that can be re-observed is cheaper to drop than information that cannot.
- Rewards, goals, and temporal-logic formulas are proxies for intent. Safety properties compile to automata with a trap state that a shield can enforce step by step; liveness must be pursued by the policy. The gap between specification and intent grows with the agent's optimization power.
- In the error–recovery chain, success plateaus without irrecoverable failure and decays geometrically, at the effective hazard rate, with it. Recovery shortens time off track, reversibility lowers the off-track hazard, and budgets bound the horizon. The models of [Chapter 1](#) and [Chapter 10](#) are special cases.
- Under a constant hazard, the 80% horizon is about a third of the 50% horizon and

the 99% horizon about one-seventieth of it. Coin-flip horizons overstate how long an agent can work unreviewed.

- The agent–environment interface of [Theorem 2.15](#) fixes the vocabulary for the rest of the book: contexts, a step-level policy induced by a token-level model, tool-augmented actions, observations, updates, budgets, and persistent memory.

---

## Notes and Further Reading

The axiomatic foundations of expected utility are due to [von Neumann and Morgenstern \(1944\)](#) and, for subjective probability, [Savage \(1954\)](#). [Gilboa and Schmeidler \(1989\)](#) axiomatize maxmin expected utility under ambiguity, and [Rockafellar and Uryasev \(2000\)](#) introduced the variational form of conditional value-at-risk now standard in risk-sensitive optimization. [Simon \(1955\)](#) remains the best short statement of bounded rationality; [Rass et al. \(2020\)](#) discuss bounded rationality in the setting of critical-infrastructure security, and [Nguyen and Zhu \(2026b\)](#) use an equivalence between transformer attention and rational inattention to construct a divergence-based safety measure for language models.

Dynamic programming originates with [Bellman \(1957\)](#). [Puterman \(1994\)](#) is the standard reference for MDPs under all three criteria, and [Bertsekas \(2017\)](#) covers finite-horizon problems, optimal stopping, and stochastic shortest paths. [Sutton and Barto \(2018\)](#) presents the same material from the learning perspective and states the reward hypothesis, which [Bowling et al. \(2023\)](#) analyze axiomatically. Robust MDPs are treated by [Iyengar \(2005\)](#) and [Nilim and El Ghaoui \(2005\)](#). [Kaelbling et al. \(1998\)](#) is the classic introduction to POMDPs in artificial intelligence, and [Zhu and Xu \(2020\)](#) give an introduction to partially observed MDPs aimed at secure and resilient cyber-physical systems. When detection and belief-dependent control are carried out by components with different objectives, the usual separation between estimator and controller design can fail ([Nguyen and Zhu, 2026a](#)). The approximate information state framework, including learning such states from data, is developed by [Subramanian et al. \(2022\)](#); [Section 9.6](#) applies it to memory. Options and temporal abstraction, which underlie the token-level versus step-level view, are due to [Sutton et al. \(1999\)](#). The contraction and fixed-point arguments used here are collected in [Chapter C](#).

Temporal logic entered computer science with [Pnueli \(1977\)](#), and shielding ([Alshiekh et al., 2018](#)) connects temporal-logic safety to reinforcement learning. The problems of specification gaming and reward hacking were framed for modern AI by [Amodei et al. \(2016\)](#), catalogued by [Krakovna et al. \(2020\)](#), and related to Goodhart’s law by [Manheim and Garrabrant \(2018\)](#); [Chapter 16](#) continues the story.

Measurements of the horizon over which agents succeed ([Kwa et al., 2025](#)) give empirical content to the analysis of [Section 2.5](#), which extends the pipeline model of [Chapter 10](#). Markov decision models with explicit failure and recovery dynamics, in which a repair policy determines long-run performance, are also used for interdependent

critical infrastructures (Huang et al., 2018). METR’s notes on the limitations of this metric (Kwa, 2026) are essential reading before drawing conclusions about delegation, and Sinha et al. (2026) give controlled evidence that per-step errors are not independent. Chapter 11 describes how horizons are estimated, and Chapter 18 poses the open problem of reliability guarantees when error rates drift over time and depend on the history. Learning in these models is the subject of Chapter 3, planning of Chapter 4, and multiple decision-makers of Chapter 5. With several decision-makers, beliefs about other agents’ strategies enter the information state; conjectural online learning handles generic information structures by forecasting opponents’ strategies and calibrating those conjectures with Bayesian learning (Li et al., 2024a).

## Exercises

EXERCISE 2.1. Show that the Bellman optimality operator  $\mathcal{B}$  of (2.2) is a  $\gamma$ -contraction in the sup norm, and deduce that value iteration converges geometrically. Then show that if  $\|V - V^*\|_\infty \leq \delta$ , any policy  $\pi$  greedy with respect to  $V$  has  $\|V^* - V^\pi\|_\infty \leq 2\gamma\delta/(1 - \gamma)$ .

*Hint:* for the first part, use  $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$ . For the second, compare  $\mathcal{B}^\pi V^\pi$  with  $\mathcal{B}^\pi V$  and  $\mathcal{B}V$  with  $\mathcal{B}V^*$ .

EXERCISE 2.2. In Theorem 2.5, let the success probability of an immediate patch be  $p$  instead of 0.4. (a) Compute the value of always patching from new as a function of  $p$ . (b) Find the value of  $p$  above which patching immediately becomes optimal. (c) Explain in words why the threshold is well below 0.9, the success probability after diagnosis.

*Hint:* the value of investigating does not depend on  $p$ .

EXERCISE 2.3. R2 believes a bug lies in module  $A$  or module  $B$ , with prior  $b_0(A) = 0.5$ . A diagnostic test, costing one step, reports the correct module with probability 0.8, and the bug does not move. (a) Compute the belief after the reports  $A, A, B$  using (2.3), and check that it depends only on the difference between the numbers of  $A$  and  $B$  reports. (b) Suppose editing the correct module earns 1, editing the wrong one earns  $-1$ , and each test costs 0.05. Show that with a single test remaining, the optimal policy has a threshold form in  $b(A)$ , and find the thresholds.

EXERCISE 2.4. For the two policies of Theorem 2.2, compute  $\text{CVaR}_\alpha$  as a function of  $\alpha \in (0, 1)$ . For which  $\alpha$  does a CVaR-minimizing designer choose policy  $B$ ? Evaluate the entropic objective for both policies with a small  $\beta$ , and compare its verdict with the mean-variance approximation.

*Hint:* for  $\alpha \leq 0.01$  the worst  $\alpha$  fraction of outcomes under  $A$  is all disaster.

EXERCISE 2.5. An agent has per-step error probability  $\epsilon = 0.02$  and runs for  $T = 50$  steps. Compute its success probability (a) with no recovery; (b) when each error is repaired within its step with probability 0.5, as in Theorem 1.5; and (c) under the chain

(2.6) with  $\rho = 0.5$ ,  $\phi = 0.001$ , and  $\psi = 0.02$ . Explain the ordering of the three answers in terms of where each model places the risk of irrecoverable failure.

EXERCISE 2.6. Take  $\phi = \psi = 0$  in Theorem 2.13. Show that  $g_T \geq \rho/(\epsilon + \rho)$  for all  $T$  when  $\epsilon + \rho \leq 1$ , and that  $g_T$  oscillates around this value when  $\epsilon + \rho > 1$ . What agent behavior would produce  $\epsilon + \rho > 1$ ?

*Hint:* use (2.8) and consider the sign of  $1 - \epsilon - \rho$ .

EXERCISE 2.7. (a) In the setting of Theorem 2.14, use the effective hazard to predict the change in the halving horizon when  $\phi$  is reduced from 0.002 to 0, and check your prediction with (2.7). (b) Under  $g_T = c e^{-\zeta T}$  with  $c < 1$ , derive  $T_q$  and show that the ratio  $T_{0.8}/T_{0.5}$  now depends on  $c$ . (c) An organization will let an agent work unreviewed only on tasks it completes with probability at least 0.95. Using the constant-hazard model with  $c = 1$ , what fraction of the agent's 50% horizon may such tasks occupy? Name two reasons the answer could be optimistic.

*Hint:* for (c), think about the self-conditioning effect and about differences between the benchmark's task distribution and the organization's.

EXERCISE 2.8. Model R1 as a POMDP in more detail than Section 2.3.3: specify states, actions, observations, and a reward. Then list the items a compacted context must retain for  $\delta_p$  to stay small, and identify one item that can safely be dropped because it can be re-observed. Draw the corresponding version of Fig. 2.4.

EXERCISE 2.9. Write LTL formulas over finite traces for R2: never push to the main branch; open a pull request eventually; and do not open a pull request until the tests have passed. Classify each as a safety or liveness property, and state which could be enforced by a shield in the scaffold. Draw an automaton for the conjunction of the three in the style of Fig. 2.5, and identify its trap state or states.

*Hint:* “not  $pr$  until  $pass$ ” can be written with the weak until,  $\neg pr \text{ W } pass$ , or tracked by a progress flag in the automaton state that records whether the tests have passed.

EXERCISE 2.10. Let  $r$  and  $\tilde{r}$  be reward functions on the same MDP. Prove that  $|V_r^\pi(s) - V_{\tilde{r}}^\pi(s)| \leq \|r - \tilde{r}\|_\infty/(1 - \gamma)$  for every policy  $\pi$ , and that a policy optimal for  $\tilde{r}$  is  $2\|r - \tilde{r}\|_\infty/(1 - \gamma)$ -optimal for  $r$ . Use Theorem 2.12 to explain why this bound offers little protection against specification gaming.

EXERCISE 2.11. Construct a context, a parser parse, and a token-level model  $p_\theta$  over a vocabulary of at most four tokens with two actions, such that greedy token-by-token decoding produces action  $a_1$  while  $\pi(a_2 | c) > \pi(a_1 | c)$  in (2.9). How does sampling at temperature 0 versus 1 change the induced step-level policy? What does this imply for evaluating an agent by its single most likely output?

*Hint:* give the more probable first token a continuation toward  $a_1$  that is its single most likely next token but carries less than all of its mass, and spend the rest on unparseable continuations.

---

# Learning Agents: From Reinforcement Learning to Foundation Models

*Agents acquire competence in two ways: by acting and being scored, and by absorbing a vast record of what people have written and done. This chapter follows both. It begins with the reinforcement learning loop, updating a Q-value and a policy by hand, and draws three lessons from deep reinforcement learning: search can be distilled into networks, narrow training generalizes poorly, and optimizers exploit flawed rewards. It then treats a pretrained language model as a stochastic policy with a broad prior, and in-context learning as inference over that prior. The core of the chapter is post-training, organized around one closed-form result: the KL-regularized optimum is an exponential tilt of the reference model. From it we derive RLHF and DPO, compute a DPO loss and a group of GRPO advantages with numbers, and trace reinforcement learning with verifiable rewards from single answers to multi-step agents trained in sandboxed environments, where the verifier becomes the target and reward hacking a practical hazard. The chapter closes with imitation and its compounding errors, on-policy corrections and distillation, and the two routes to continual learning: verified skill libraries and weight updates in deployment.*

---

## LEARNING OBJECTIVES

- Compute temporal-difference and Q-learning updates by hand, and derive the policy-gradient theorem, REINFORCE with a baseline, and the clipped surrogate of PPO.
- Interpret a pretrained language model as a stochastic policy at the token and step levels, and in-context learning as approximate Bayesian inference.
- Derive the closed-form optimum of the KL-regularized RLHF objective and the DPO loss, state their assumptions, and evaluate both on small numerical cases.
- Compute group-relative advantages in reinforcement learning with verifiable rewards, explain what the normalization and clipping do, and identify prompts that give no learning signal.

- Design an agentic reinforcement learning loop, including task validation, sandboxed rollouts, isolated grading, and audits for reward hacking.
- Explain why behavior cloning compounds errors, and how DAgger, recovery data, and on-policy distillation correct it.
- Compare memory-based and weight-based continual learning, and specify admission rules for a skill library.

---

Throughout, we use the decision-process notation of Chapter 2: states  $s \in \mathcal{S}$ , actions  $a \in \mathcal{A}$ , rewards  $r$ , discount  $\gamma \in [0, 1)$ , policies  $\pi$ , and value functions  $V^\pi$  and  $Q^\pi$  as defined in Section 2.2. We write  $\pi_\theta$  for a policy with parameters  $\theta$  and  $r_t$  for the reward received after the action at step  $t$ . The symbols  $\epsilon$  and  $\rho$  keep the meanings the book reserves for them, per-step error and recovery probability; exploration rates and clipping ranges are written  $\epsilon$  and  $\epsilon_c$ .

The chapter has three movements. Sections 3.1 and 3.2 build the reinforcement learning toolkit and the lessons of its deep-learning era. Sections 3.3 and 3.4 treat a language model as a policy and derive the post-training methods that shape it. Sections 3.5 and 3.6 turn to agents proper: how they learn to act over many steps, and how they keep learning after deployment.

## 3.1 Reinforcement Learning Fundamentals

Picture an agent that has never seen a task. It tries something, the world responds, and a number arrives: a point scored, a test passed, a step's cost. Nobody tells it which action would have been best. It must work that out from the numbers alone, and it must do so while its own choices determine which numbers it gets to see. That is the setting of *reinforcement learning* (RL).

Formally, RL learns a policy from sampled interaction rather than from a known model. Dynamic programming (Section 2.2) computes  $V^*$  from the transition kernel  $P$ ; RL replaces expectations under  $P$  with samples. Figure 3.1 shows the loop. Two families of methods follow from it. Value-based methods estimate  $Q^*$  and act greedily with respect to it. Policy-based methods adjust  $\pi_\theta$  directly along an estimate of the gradient of return.

### 3.1.1 Temporal-Difference Learning and Q-Learning

*Temporal-difference* (TD) learning estimates  $V^\pi$  from transitions  $(s_t, r_t, s_{t+1})$  generated by  $\pi$ . TD(0) moves the estimate toward a one-step bootstrapped target:

$$V(s_t) \leftarrow V(s_t) + \alpha_t \delta_t, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t). \quad (3.1)$$

The *TD error*  $\delta_t$  is a sampled, noisy version of the Bellman residual: it measures how surprised the estimate is by one step of experience. A Monte Carlo estimate waits for

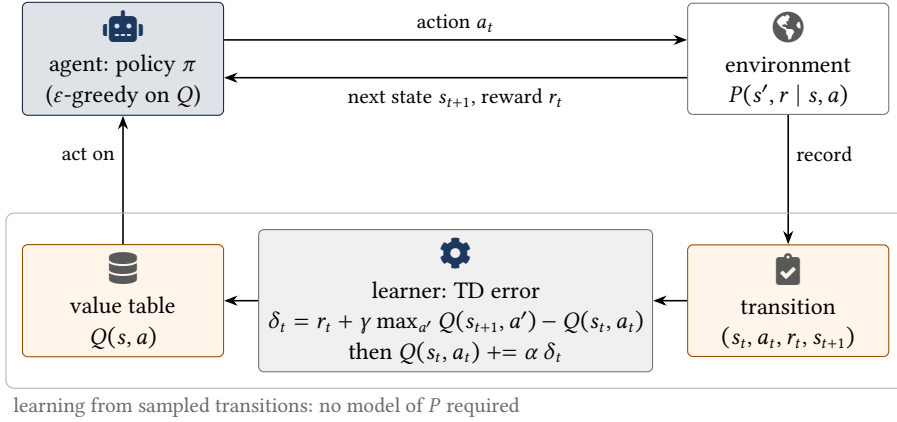


FIGURE 3.1. The reinforcement learning loop. The agent acts, the environment returns a next state and a reward, and the learner turns each recorded transition into a correction of its value estimates, which in turn change how the agent acts. Nothing in the loop requires the transition kernel  $P$ ; samples stand in for it.

the episode's full return. TD instead updates after every step and uses its own current estimate of the next state as part of the target. This *bootstrapping* lowers variance, since one reward is less noisy than a sum of many, at the cost of bias while  $V$  is still inaccurate.

*Q-learning* (Watkins and Dayan, 1992) applies the same idea to the Bellman optimality equation:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t \left[ r_t + \gamma \max_{a' \in \mathcal{A}} Q(s_{t+1}, a') - Q(s_t, a_t) \right]. \quad (3.2)$$

The target uses the best action at  $s_{t+1}$  according to the current estimate, not the action the agent actually takes next. *Q-learning* is therefore *off-policy*: it learns about the greedy policy while following any sufficiently exploratory behavior policy. The on-policy alternative, SARSA, uses the action actually taken,  $Q(s_{t+1}, a_{t+1})$ , and so learns the value of the exploring policy itself.

**EXAMPLE 3.1** (One *Q-learning* update by hand). An agent in state  $s$  has estimates  $Q(s, \text{edit}) = 0.5$  and  $Q(s, \text{run}) = 0.2$ . Exploring, it takes the lower-valued action, run, pays a step cost  $r_t = -0.1$ , and lands in  $s'$ , where  $Q(s', \text{edit}) = 0.8$  and  $Q(s', \text{run}) = 0.4$ . Take  $\gamma = 0.9$  and  $\alpha = 0.5$ .

1. *Target*:  $r_t + \gamma \max_{a'} Q(s', a') = -0.1 + 0.9 \times 0.8 = 0.62$ .
2. *TD error*:  $\delta_t = 0.62 - 0.2 = 0.42$ . The action turned out better than estimated.
3. *Update*:  $Q(s, \text{run}) \leftarrow 0.2 + 0.5 \times 0.42 = 0.41$ .

The greedy choice at  $s$  is still edit ( $0.5 > 0.41$ ), but the gap has narrowed; a few more such experiences would reverse it. Suppose the agent's next action at  $s'$  is again an exploratory run. SARSA would use  $Q(s', \text{run}) = 0.4$ , giving target  $-0.1 + 0.9 \times 0.4 = 0.26$

and new value 0.23. The two methods disagree because they estimate different things:  $Q$ -learning the value of acting greedily from  $s'$  onward, SARSA the value of continuing to explore.

For a finite MDP with bounded rewards and  $\gamma < 1$ , tabular  $Q$ -learning converges to  $Q^*$  with probability one provided that every state–action pair is updated infinitely often and the step sizes satisfy the Robbins–Monro conditions  $\sum_t \alpha_t(s, a) = \infty$  and  $\sum_t \alpha_t(s, a)^2 < \infty$  (Watkins and Dayan, 1992). The first condition lets the estimates travel arbitrarily far from a bad initialization; the second makes the noise in the targets average out. The proof views (3.2) as stochastic approximation of a contraction mapping; Theorem C.9 in Chapter C gives the argument. Other routes to the same guarantee exist; Li et al. (2021) recast the MDP as an online optimization problem, build a Blackwell approachability game from it, and obtain value-iteration and  $Q$ -learning variants that converge to the optimal solution. None of these conditions survives intact in large problems: states are rarely revisited, and  $Q$  must be approximated, a point Section 3.2 returns to. Approximation brings its own rate questions; Li and Zhu (2019), for example, bound the convergence rate of an adaptive multiscale scheme for value-function approximation.

### 3.1.2 Policy Gradient Methods

Value-based methods act through an estimate. Policy-gradient methods skip the estimate and adjust a differentiable policy  $\pi_\theta(a \mid s)$  directly. The intuition is the one a coach uses: make what worked more likely, in proportion to how much better than expected it worked. Consider episodes of horizon  $T$ , with trajectory  $\tau = (s_0, a_0, r_0, \dots, s_{T-1}, a_{T-1}, r_{T-1}, s_T)$ , initial distribution  $\nu_0$ , and objective

$$J(\theta) = \mathbb{E}_{\tau \sim P_\theta} [R(\tau)], \quad R(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t,$$

where  $P_\theta(\tau) = \nu_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P(s_{t+1}, r_t \mid s_t, a_t)$ .

**PROPOSITION 3.2** (Policy-gradient theorem, episodic likelihood-ratio form). *Assume a finite horizon  $T$ , bounded rewards, finite  $S$  and  $\mathcal{A}$ , and that  $\pi_\theta(a \mid s)$  is differentiable in  $\theta$  for all  $(s, a)$ . Let  $G_t = \sum_{k=t}^{T-1} \gamma^k r_k$  be the (discounted-from-the-start) reward to go. Then for any function  $b : S \rightarrow \mathbb{R}$ ,*

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim P_\theta} \left[ \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t) (G_t - b(s_t)) \right]. \quad (3.3)$$

*Proof.* With finitely many trajectories,

$$\nabla_\theta J = \sum_{\tau} R(\tau) \nabla_\theta P_\theta(\tau) = \mathbb{E}_{P_\theta} [R(\tau) \nabla_\theta \log P_\theta(\tau)],$$

using  $\nabla \mathbb{P}_\theta = \mathbb{P}_\theta \nabla \log \mathbb{P}_\theta$  on trajectories of positive probability. Neither  $v_0$  nor  $P$  depends on  $\theta$ , so  $\nabla_\theta \log \mathbb{P}_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t)$ . This gives (3.3) with  $R(\tau)$  in place of  $G_t - b(s_t)$ .

*Causality.* For  $k < t$ , the reward  $r_k$  is determined by the trajectory up to  $s_t$ . Conditioning on that prefix  $h_t = (s_0, a_0, r_0, \dots, s_t)$ ,

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t | s_t) | h_t] = \sum_a \pi_\theta(a | s_t) \nabla_\theta \log \pi_\theta(a | s_t) = \nabla_\theta \sum_a \pi_\theta(a | s_t) = 0. \quad (3.4)$$

Hence  $\mathbb{E}[r_k \nabla_\theta \log \pi_\theta(a_t | s_t)] = 0$  for  $k < t$ , and  $R(\tau)$  can be replaced by  $G_t$  in the  $t$ -th term.

*Baseline.* By the same identity,  $\mathbb{E}[b(s_t) \nabla_\theta \log \pi_\theta(a_t | s_t)] = \mathbb{E}[b(s_t) \cdot 0] = 0$ , since  $b(s_t)$  is fixed given  $s_t$ . Subtracting it therefore leaves the expectation unchanged. ■

The proof uses only that  $b$  does not depend on  $a_t$  or on later events; a baseline may depend on the state and on the history up to  $s_t$ , but not on the action being scored. Sampling episodes and replacing the expectation with an average gives *REINFORCE* (Williams, 1992), the update  $\theta \leftarrow \theta + \eta \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t)(G_t - b(s_t))$ . The estimator is unbiased for every baseline, but its variance is not. A good baseline, typically a learned estimate of  $V^\pi(s_t)$ , can reduce variance by orders of magnitude. The quantity  $G_t - b(s_t)$  then estimates the *advantage*  $Q^\pi(s_t, a_t) - V^\pi(s_t)$ , and methods that learn such a baseline are called actor-critic methods.

**EXAMPLE 3.3** (One REINFORCE step). A one-step problem has two actions and a softmax policy with logits  $\theta = (\theta_1, \theta_2) = (0, 0)$ , so  $\pi_\theta = (0.5, 0.5)$ . For a softmax,  $\nabla_\theta \log \pi_\theta(a_i) = e_i - \pi_\theta$ , where  $e_i$  is the  $i$ -th unit vector. Take step size  $\eta = 1$  and baseline  $b = 0.4$ , the running average reward.

- *The sample succeeds.* The agent draws  $a_1$  and receives  $r = 1$ . The update is  $\theta \leftarrow \theta + (1 - 0.4)(0.5, -0.5) = (0.3, -0.3)$ , and  $\pi_\theta(a_1)$  rises to  $e^{0.3}/(e^{0.3} + e^{-0.3}) \approx 0.646$ .
- *The sample fails.* The agent draws  $a_2$  and receives  $r = 0$ . The update is  $\theta \leftarrow \theta + (0 - 0.4)(-0.5, 0.5) = (0.2, -0.2)$ , and  $\pi_\theta(a_1)$  rises to about 0.599.

Without the baseline, the failed sample would have produced no update at all, because  $r - b = 0$ . With it, a below-average outcome actively pushes probability away from the action that produced it. That is the variance-reduction role of  $b$  seen in a single step.

#### 💡 KEY IDEA

*A policy gradient raises the log-probability of each sampled action in proportion to how much better than expected it turned out. Everything else, including baselines, critics, group means, and clipping, is a way of estimating “better than expected” with less noise or of keeping each step small.*

*Reusing samples: PPO.* The gradient (3.3) is valid only for data from the current policy. Reusing a batch for several updates requires the importance weight  $w_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ . Proximal policy optimization (PPO) (Schulman et al., 2017) maximizes the *clipped surrogate*

$$L^{\text{clip}}(\theta) = \mathbb{E}_t \left[ \min \left( w_t(\theta) \hat{A}_t, \text{clip} \left( w_t(\theta), 1 - \varepsilon_c, 1 + \varepsilon_c \right) \hat{A}_t \right) \right], \quad (3.5)$$

where  $\hat{A}_t$  is an advantage estimate and  $\varepsilon_c$ , typically 0.1 to 0.3, is the clip range. The minimum makes the objective pessimistic. Once the ratio has moved far enough in the direction the advantage favors, beyond  $1 + \varepsilon_c$  for  $\hat{A}_t > 0$  or below  $1 - \varepsilon_c$  for  $\hat{A}_t < 0$ , the clipped term is constant and the gradient vanishes, so no single batch can drag the policy far from the one that collected it. Moves in the unfavorable direction are never clipped, so mistakes can still be corrected. PPO was the optimizer of early RLHF, and Section 3.4 meets its surrogate again inside GRPO.

### 3.1.3 Exploration and Exploitation

A learner only improves the actions it tries. The *exploration–exploitation* trade-off asks how much reward to sacrifice now for information that improves later decisions. Simple heuristics perturb the greedy choice:  $\varepsilon$ -greedy, or sampling from a softmax over  $Q$  with a temperature. Principled methods act optimistically under uncertainty, adding a bonus that shrinks as an action is tried, as the upper confidence bounds used in bandits and in Monte Carlo tree search do (Kocsis and Szepesvári, 2006). Intrinsic-motivation methods reward novelty itself when external reward is sparse.

For language-model policies, exploration takes a distinctive form. The pretrained model already places most of its probability on plausible behavior, so sampling at a moderate temperature explores within that prior, not over the astronomically large space of all token sequences. The flip side is that RL fine-tuning mostly reweights behaviors the model can already produce with nonnegligible probability. If no sample in a batch ever solves a task, a policy-gradient method receives no signal about how to solve it (Theorem 3.7). Yue et al. (2025) measured the consequence. Comparing models trained with verifiable rewards against their base models, they found that the trained models solved more problems at pass@1, but that with enough samples the base models solved as many or more: training had made the model more reliable at what it could already occasionally do, rather than enlarging that set. The finding is specific to the models, tasks, and training budgets studied, and whether longer or broader RL training expands coverage remained an open question as of 2026. The practical responses are curricula of tasks at the edge of the model’s competence and, where a stronger teacher exists, demonstrations that put new behaviors into the prior (Section 3.5).

### 3.1.4 Model-Based Reinforcement Learning

*Model-based* methods learn estimates  $\hat{P}$  and  $\hat{r}$  from data and then plan with them: by running dynamic programming, by generating simulated experience for a value-based

learner, or by searching ahead at decision time. A model can make learning far more sample-efficient, because each real transition informs predictions everywhere the model generalizes. Its danger is *model exploitation*: a planner optimizes against the model and finds the places where the model is wrong in its favor, and errors compound over long simulated rollouts. Short-horizon planning with frequent replanning from real observations is the standard mitigation; it is the principle of receding-horizon control developed in Chapter 4. A language model used to predict the consequences of a proposed action is, in this vocabulary, a learned world model, and it inherits the same danger. A subtler failure occurs when the model class cannot represent the true dynamics at all. The agent can then settle at a self-consistent fixed point, acting optimally for a misspecified model that nevertheless fits the data its own policy generates; Zhu and Han (2026a) characterize and learn such Berk–Nash equilibria in infinite-horizon MDPs, and Hammar et al. (2025) design an online method in which a player refines a possibly misspecified conjecture about its environment by Bayesian learning.

## 3.2 Lessons from Deep Reinforcement Learning

Deep RL replaced tables with neural networks and produced the first agents that learned competent behavior from raw, high-dimensional input. Its methods are not the methods of today’s language agents, but its lessons transfer almost unchanged.

### 3.2.1 Value-Based Deep RL

The deep  $Q$ -network (DQN) (Mnih et al., 2015) represents  $Q(s, a; w)$  with a convolutional network that reads recent screen frames and outputs one value per action. It minimizes the squared TD error

$$\mathcal{L}(w) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{B}} \left[ \left( r + \gamma \max_{a'} Q(s', a'; w^-) - Q(s, a; w) \right)^2 \right].$$

Naively combining function approximation, bootstrapping, and off-policy data is unstable; Sutton and Barto (2018) call this combination the *deadly triad*. The instability is easy to picture: a network update meant to change  $Q(s, a)$  also changes  $Q(s', a')$ , which is the target, so the regression chases its own tail. DQN tamed it with two devices. *Experience replay* stores transitions in a buffer  $\mathcal{B}$  and samples minibatches from it, breaking the temporal correlation of consecutive frames and reusing each transition many times. A *target network* with parameters  $w^-$ , copied from  $w$  only periodically, holds the regression target fixed between copies. With one architecture and one set of hyperparameters, the same algorithm learned to play many different Atari games from pixels and game score alone.

### 3.2.2 Search plus Learning: The AlphaGo Lineage

AlphaGo (Silver et al., 2016) combined learning with search. A policy network was first trained by supervised learning to predict expert moves from human games, then

improved by policy-gradient self-play. A value network was trained by regression to predict the winner of self-play games from a position. At decision time, Monte Carlo tree search (Chapter 4) used the policy network as a prior to focus on promising moves and combined the value network with fast rollouts to evaluate leaf positions. Later systems in the lineage removed the human data and the rollouts, training policy and value networks purely from self-play guided by search.

The engine is a loop between two operators. Search, given a policy and a value estimate, chooses better moves than the policy alone. Training then *distills* those improved choices back into the network, which makes the next round of search stronger. The same loop recurs in language agents. Long reasoning at inference time (Section 4.3) plays the role of search, and training on successful reasoning traces distills it into the model, whether the traces come from the model itself (Section 3.4) or from a stronger teacher (Section 3.5).

#### KEY IDEA

*Search is a policy-improvement operator, and learning amortizes it. Whatever a system can do slowly with extra computation, whether tree search, long reasoning, or many attempts filtered by a verifier, it can learn to do faster by training on the results.*

### 3.2.3 Generalization and Its Limits

Three lessons from this era shape the rest of the chapter. First, deep RL agents were *narrow*. An agent trained on one game, or even on one set of levels of a game, often failed on small variations it had never seen, because nothing in the objective rewarded competence beyond the training distribution. Foundation models answer this lesson with breadth of pretraining rather than with better RL. Within RL, meta-learning and online adaptation attack the same gap, for instance by adapting self-driving policies at execution time to nonstationary conditions or to heterogeneous pedestrians (Li et al., 2023b; Lei and Zhu, 2023, 2022), and physics-informed structure has been used to obtain navigation policies that work zero-shot in unseen scenarios (Yin et al., 2024).

Second, learning from reward alone was *sample-inefficient*, requiring millions of episodes. That is affordable in a simulator and prohibitive when each episode is a human conversation or a container running a test suite. Priors from pretraining, demonstrations, and learned models all attack this cost.

Third, agents optimized the reward they were given, not the one their designers intended. Documented cases include agents that loop to collect a respawning bonus instead of finishing a race and agents that exploit simulator bugs (Amodei et al., 2016; Krakovna et al., 2020). This *specification gaming* is not a defect of any particular algorithm: a stronger optimizer finds a flawed reward’s loopholes faster. The lesson applies with full force to reward models and verifiers for language agents (Sections 3.4 and 3.5), and Chapter 16 develops it. A related, adversarial failure arises when an attacker falsifies the

reward signal itself. [Huang and Zhu \(2019b\)](#) show that  $Q$ -learning still converges under bounded falsification of costs and characterize a robust region of costs within which the adversary cannot induce its target policy, and [Huang and Zhu \(2022\)](#) give necessary and sufficient conditions for such manipulation to succeed against linear-quadratic learners.

### 3.3 Foundation Models as Priors for Agents

Deep RL learned behavior from scratch in each environment. A foundation model instead arrives with a prior over behavior learned from a vast corpus. Most of what an agent then needs is a matter of shaping and conditioning that prior.

#### 3.3.1 Pretraining and Scaling

A language model  $p_\theta$  is *pretrained* to predict the next token of text  $x = (x_1, \dots, x_n)$  drawn from a corpus, by minimizing the cross-entropy

$$\mathcal{L}(\theta) = -\mathbb{E}_x \left[ \sum_{j=1}^n \log p_\theta(x_j \mid x_{<j}) \right].$$

The objective is simple, but predicting text well requires modeling grammar, facts, the structure of arguments and programs, and the goals of the people who wrote the text.

[Kaplan et al. \(2020\)](#) found that test loss falls smoothly and predictably as a power law in model size  $N$ , dataset size  $D$ , and training compute  $C$ , over several orders of magnitude, when the other factors are not the bottleneck. A convenient form, fitted by [Hoffmann et al. \(2022\)](#), is

$$\mathcal{L}(N, D) \approx E + \frac{A_N}{N^{\alpha_N}} + \frac{A_D}{D^{\alpha_D}}, \quad (3.6)$$

where  $E$  is an irreducible loss and  $A_N, A_D, \alpha_N, \alpha_D > 0$  are fitted constants. *Scaling laws* of this kind let developers choose model and data size for a compute budget before training; [Hoffmann et al. \(2022\)](#) revised Kaplan et al.’s allocation, concluding that parameters and tokens should grow in roughly equal proportion. Three cautions apply. The constants depend on architecture, data, and tokenizer. Loss is not capability: specific skills can improve abruptly while loss declines smoothly, so scaling laws forecast averages better than abilities. And as of 2026, much of the capability growth of agents comes from post-training and from inference-time computation ([Section 4.3](#)) rather than from pretraining scale alone.

[Brown et al. \(2020\)](#) showed the practical consequence of scale: a sufficiently large model can perform many tasks from a description and a few examples in its context, without any update to  $\theta$ .

### 3.3.2 Language Models as Stochastic Policies

A language model defines a stochastic policy at two levels, matching the token-level and step-level decision processes of Section 2.6.

At the *token level*, the state is the current token sequence (context plus tokens generated so far), an action is one token from the vocabulary  $\Sigma$ , the policy is  $p_\theta(\cdot \mid \text{prefix})$ , and the transition deterministically appends the token. Reward is usually zero until the response ends.

At the *step level*, the agent holds context  $c_t$ , samples a whole string  $z \sim p_\theta(\cdot \mid \text{render}(c_t))$ , and the scaffold parses  $z$  into an action  $a_t \in \mathcal{A}$ : a tool call, a message, or termination (Theorem 8.2). The induced policy  $\pi(a_t \mid c_t)$  sums the probabilities of all strings that parse to  $a_t$  (8.1). The environment, which may be stochastic and adversarial, returns  $o_{t+1}$ , and the context is updated to  $c_{t+1} = U(c_t, a_t, o_{t+1})$ .

The link between the levels is what makes policy-gradient methods practical for language models. For a sampled string  $z = (z_1, \dots, z_m)$ ,

$$\log p_\theta(z \mid c) = \sum_{j=1}^m \log p_\theta(z_j \mid c, z_{<j}), \quad (3.7)$$

so the score function  $\nabla_\theta \log p_\theta(z \mid c)$  in (3.3) is a sum of per-token terms computed by one backward pass. Observation tokens inserted by the environment are part of the conditioning, not of  $z$ , so they contribute no score terms. Which level to use is a modeling choice: credit assignment and KL penalties are often computed per token, while rewards, safety policies, and the error model of Section 2.5 are naturally defined per step.

Seen this way, pretraining is *behavior cloning* of an enormous, heterogeneous population of writers. The resulting policy is a mixture: conditioned on a context, it imitates the kind of author likely to have produced that context, competent or careless. Post-training (Section 3.4) narrows that mixture toward helpful, instruction-following, and correct behavior.

### 3.3.3 In-Context Learning as Implicit Bayesian Inference

Xie et al. (2022) give a precise sense in which conditioning is learning. Suppose each pretraining document is generated by first drawing a latent *concept*  $\psi$  from a prior  $p(\psi)$  and then generating text from a hidden Markov model indexed by  $\psi$ . (This  $\psi$  is local to this subsection and unrelated to the failure probability of Section 2.5.) A model that matches this pretraining distribution exactly must, to predict the next token, implicitly infer the concept:

$$p(y \mid \text{prompt}) = \int p(y \mid \text{prompt}, \psi) p(\psi \mid \text{prompt}) d\psi. \quad (3.8)$$

A prompt of  $k$  input–output examples is evidence about  $\psi$ . Xie et al. (2022) prove that, as  $k$  grows, the posterior concentrates on the concept that generated the examples, and the in-context prediction approaches the Bayes-optimal prediction for that concept. The subtle

part is that a prompt of concatenated examples is *not* a typical pretraining document, because the transitions between examples are unnatural. Their result therefore requires a distinguishability condition, stating roughly that the evidence the examples provide for the true concept outweighs the error introduced by these low-probability transitions.

The assumptions deserve honest statement. The theorem assumes a mixture-of-HMMs generative process, a model that fits the pretraining distribution exactly, and asymptotically many examples; the empirical support comes from a small synthetic dataset built to satisfy the assumptions. It explains why in-context learning can emerge from next-token prediction, not how transformers implement it, and other accounts, such as transformers implementing learning algorithms in their forward pass, are complementary. A related idea appears in multi-agent RL, where transformer policies trained offline are conditioned on beliefs about other agents so that they can adapt to nonstationary opponents online (Li et al., 2023a). The view is still useful for agents. It treats everything in  $c_t$  as evidence that shifts the model’s posterior over which task, persona, and conventions apply. That is why a few demonstrations of correct tool use improve reliability, why irrelevant context can degrade it, and why text an adversary places in the context through a tool result can redirect the agent (Chapter 13). Nguyen and Zhu (2026b) quantify this sensitivity as the worst-case KL divergence between a model’s clean and attacked output distributions under stealthy embedding-input attacks.

## 3.4 Post-Training for Agency

*Post-training* turns a pretrained model into an assistant and an agent. Its stages differ mainly in the supervision signal: demonstrations, human preferences, AI-generated preferences, and automatically checkable outcomes. Figure 3.2 lays the main pipelines side by side. Throughout this section  $x$  is a prompt (for an agent, a rendered context),  $y$  a response, and  $\pi_{\text{ref}}$  a fixed reference policy, usually the model before the current stage.

### 3.4.1 Instruction Tuning

*Instruction tuning* is supervised fine-tuning (SFT) on pairs  $(x, y)$  of instructions and desired responses, minimizing  $-\mathbb{E} \log p_\theta(y \mid x)$ . Wei et al. (2022a) showed that fine-tuning on many tasks phrased as natural-language instructions improves zero-shot performance on held-out task types: the model learns the general skill of following instructions, not just the tasks in the mixture. For agents, the same recipe applied to demonstrations formatted as tool calls teaches the action syntax of Theorem 8.2. SFT is behavior cloning, and it is limited by the quality of its demonstrations and by the compounding-error problem of Section 3.5.

### 3.4.2 Reinforcement Learning from Human Feedback

For many behaviors it is easier to judge a response than to write an ideal one. *Reinforcement learning from human feedback* (RLHF) exploits this asymmetry. Christiano et al.

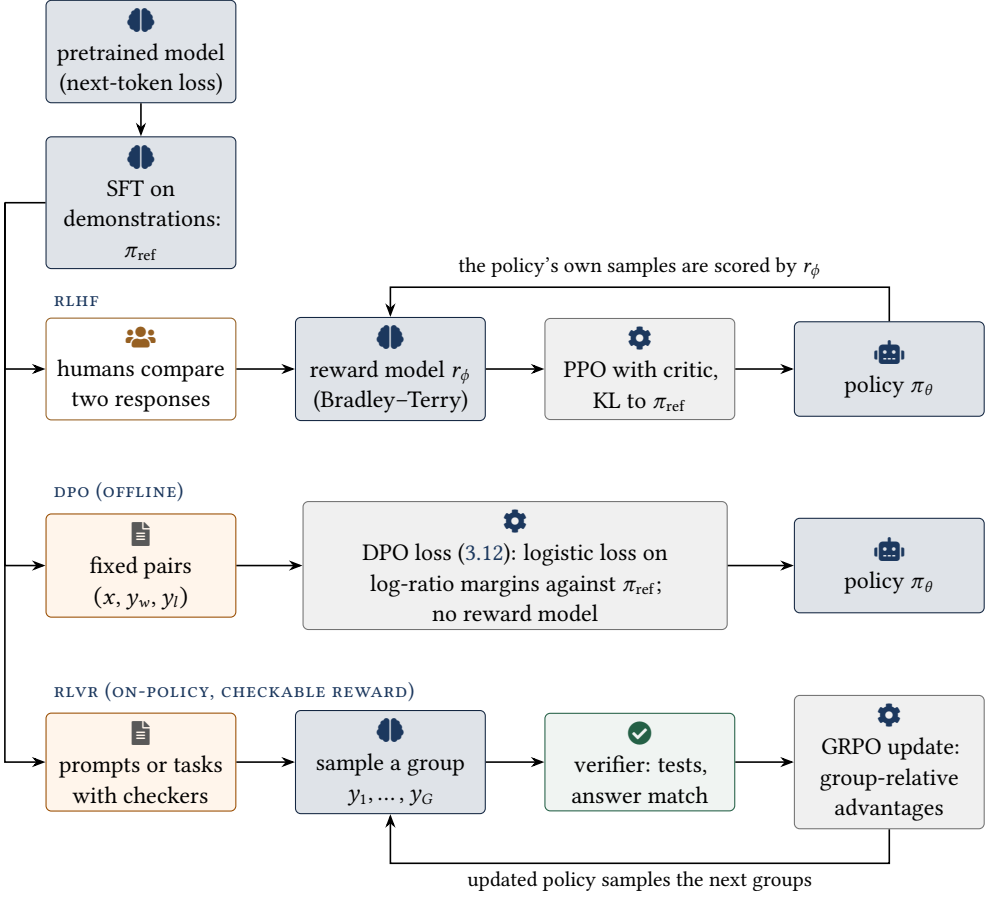


FIGURE 3.2. Three post-training pipelines that start from the same supervised model  $\pi_{\text{ref}}$ . RLHF learns a reward model from human comparisons and optimizes the policy against it on the policy's own samples. DPO fits the same preference data directly, offline, with no reward model and no sampling. RLVR replaces preferences with a mechanical checker and uses the spread of rewards within a group of samples as its baseline. All three are anchored to  $\pi_{\text{ref}}$ , explicitly through a KL penalty or implicitly through the log-ratio.

(2017) learned rewards for simulated robotics and Atari tasks from human comparisons of short behavior clips, and Ouyang et al. (2022) applied the approach to language models in three steps: SFT on demonstrations, fitting a reward model to human rankings of sampled responses, and optimizing the SFT policy against the reward model with PPO.

*Preference model.* A labeler sees two responses to  $x$  and prefers  $y_w$  over  $y_l$ . The *Bradley–Terry model* (Bradley and Terry, 1952) posits a latent reward  $r(x, y)$  with

$$\mathbb{P}(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l)), \quad \sigma(u) = \frac{1}{1 + e^{-u}}, \quad (3.9)$$

where, in this chapter,  $\sigma$  denotes the logistic function. A reward model  $r_\phi$  is fitted to a dataset  $\mathcal{D}_{\text{pref}}$  of comparisons by maximum likelihood, minimizing

$$-\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}_{\text{pref}}} \log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

Two features matter later. The model depends only on reward differences, so  $r$  is identified only up to an additive function of  $x$ . And the assumption that one scalar reward explains all labelers’ choices is strong: real preferences are noisy, intransitive, and differ across people (Section 16.3). Pairwise feedback can be aggregated in other ways; Yang et al. (2025), in route recommendation, define equilibrium through Borda scores and learn it from dueling feedback.

*KL-regularized objective.* The policy then maximizes

$$\max_{\pi} \mathbb{E}_x \left[ \mathbb{E}_{y \sim \pi(\cdot \mid x)} [r(x, y)] - \beta \text{D}_{\text{KL}}(\pi(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x)) \right], \quad (3.10)$$

with  $\beta > 0$ . The KL term is essential rather than cosmetic. The reward model is accurate only near the distribution of responses it was trained on. An unconstrained optimizer drifts to responses where  $r_\phi$  is wrong in its favor, the specification gaming of Section 3.2, known here as *reward overoptimization*. The KL penalty keeps the policy near  $\pi_{\text{ref}}$  and also limits forgetting of pretrained capabilities (Section 3.6). What policy does the objective actually select? It has a closed form.

**PROPOSITION 3.4** (Optimal KL-regularized policy). *Fix  $x$ , let  $\mathcal{Y}$  be a finite or countable response set, let  $\beta > 0$ , and assume  $Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp(r(x, y)/\beta) < \infty$  (true, for instance, if  $r$  is bounded). Among all distributions  $\pi(\cdot \mid x)$  over  $\mathcal{Y}$ , the objective in (3.10) has the unique maximizer*

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{r(x, y)}{\beta}\right), \quad (3.11)$$

and its optimal value is  $\beta \log Z(x)$ .

*Proof.* Any  $\pi$  that places mass where  $\pi_{\text{ref}}$  does not has infinite KL divergence and cannot be optimal, so consider  $\pi \ll \pi_{\text{ref}}$ . Since  $\pi^*$  is a probability distribution with the same

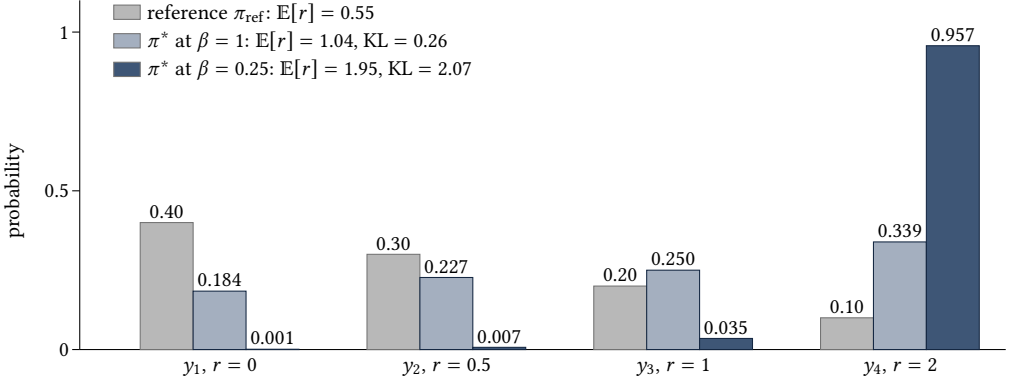


FIGURE 3.3. The KL-regularized optimum (3.11) as an exponential tilt. Four responses have reference probabilities (0.4, 0.3, 0.2, 0.1) and rewards (0, 0.5, 1, 2). At  $\beta = 1$  the tilt moves mass toward the best response while keeping every response plausible. At  $\beta = 0.25$  nearly all mass sits on  $y_4$ , at a KL cost eight times larger. No setting of  $\beta$  can place mass on a response that  $\pi_{\text{ref}}$  assigns probability zero.

support as  $\pi_{\text{ref}}$ , and  $r(x, y) = \beta \log(\pi^*(y | x)Z(x)/\pi_{\text{ref}}(y | x))$  on that support,

$$\mathbb{E}_{\pi}[r] - \beta \mathbb{E}_{\pi} \left[ \log \frac{\pi}{\pi_{\text{ref}}} \right] = \beta \mathbb{E}_{\pi} \left[ \log \frac{\pi^* Z(x)}{\pi_{\text{ref}}} - \log \frac{\pi}{\pi_{\text{ref}}} \right] = \beta \log Z(x) - \beta D_{\text{KL}}(\pi \| \pi^*).$$

The KL divergence is nonnegative and vanishes if and only if  $\pi = \pi^*$ , which proves both claims. ■

The optimal policy is an *exponential tilt* of the reference: each response keeps its reference probability, multiplied by a factor that grows with its reward, and the result is renormalized. The parameter  $\beta$  acts as a temperature. As  $\beta \rightarrow \infty$ ,  $\pi^* \rightarrow \pi_{\text{ref}}$ , and as  $\beta \rightarrow 0$ ,  $\pi^*$  concentrates on the reward-maximizing responses in the support of  $\pi_{\text{ref}}$ . Figure 3.3 shows the tilt at two temperatures. The result is nonparametric: a finite network may be unable to represent  $\pi^*$ , and PPO may not find the best policy it can represent. The same entropy-regularized Gibbs form appears when generation is regulated rather than rewarded: Zhu (2026g) relate the message distribution induced by autoregressive sampling to such a law and model a generator and a regulator as a saddle-point game whose dual value is an  $f$ -divergence.

**EXAMPLE 3.5** (The effect of  $\beta$ ). Let  $\mathcal{Y} = \{y_1, y_2\}$  with  $\pi_{\text{ref}} = (0.5, 0.5)$  and rewards  $r = (1, 0)$ . By (3.11),  $\pi^*(y_1) = e^{1/\beta}/(e^{1/\beta} + 1)$ . For  $\beta = 2$ ,  $\pi^*(y_1) \approx 0.62$ ; for  $\beta = 0.5$ ,  $\pi^*(y_1) \approx 0.88$ . If instead  $\pi_{\text{ref}} = (0.01, 0.99)$ , then at  $\beta = 0.5$ ,  $\pi^*(y_1) = 0.01e^2/(0.01e^2 + 0.99) \approx 0.07$ . A strong reference prior against the better response is overcome only slowly. That is protective when  $r$  is a flawed reward model and costly when it is correct.

The four-response case of Fig. 3.3 makes the trade explicit. The reference earns expected reward 0.55. At  $\beta = 1$ ,  $\pi^* \approx (0.18, 0.23, 0.25, 0.34)$  earns 1.04 at a KL cost of 0.26

nats, for an objective value of  $0.78 = \beta \log Z$ . At  $\beta = 0.25$ ,  $\pi^* \approx (0.001, 0.007, 0.035, 0.957)$  earns 1.95 at a KL cost of 2.07 nats, for an objective value of 1.44. Most of the reward gain comes cheaply; the last increment is expensive in divergence, which is exactly where a flawed reward model is most likely to be wrong.

#### 💡 KEY IDEA

*KL-regularized post-training reweights what the reference model already does; it does not invent behavior the reference never produces. The optimum is the reference tilted by  $\exp(r/\beta)$ , so both the gains and the risks of post-training live in how far the tilt moves the model, and a reward model is least trustworthy exactly where the tilt moves it farthest.*

### 3.4.3 Direct Preference Optimization

Theorem 3.4 can be inverted. Taking logarithms in (3.11),

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x).$$

Every reward corresponds to an optimal policy, and the reward is recoverable from that policy up to the prompt-dependent term  $\beta \log Z(x)$ , which is exactly the part that Bradley-Terry cannot identify. Substituting into (3.9), the partition function cancels:

$$\mathbb{P}(y_w \succ y_l | x) = \sigma \left( \beta \log \frac{\pi^*(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi^*(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right).$$

*Direct preference optimization* (DPO) (Rafailov et al., 2023) parameterizes  $\pi^*$  by the policy  $\pi_\theta$  and fits the preference data by maximum likelihood:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}_{\text{pref}}} \left[ \log \sigma \left( \beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]. \quad (3.12)$$

Writing  $\hat{r}_\theta(x, y) = \beta \log(\pi_\theta(y | x)/\pi_{\text{ref}}(y | x))$  for the *implicit reward*, the gradient is

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta \mathbb{E} \left[ \sigma \left( \hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w) \right) \left( \nabla_\theta \log \pi_\theta(y_w | x) - \nabla_\theta \log \pi_\theta(y_l | x) \right) \right].$$

Descending it raises the likelihood of preferred responses, lowers that of dispreferred ones, and weights each pair by how badly the implicit reward currently misorders it. The log-probabilities are sums of token terms (3.7).

**EXAMPLE 3.6** (A DPO loss for one pair). A labeler prefers  $y_w$  to  $y_l$ . Summing token log-probabilities, the reference model gives  $\log \pi_{\text{ref}}(y_w | x) = -12.0$  and  $\log \pi_{\text{ref}}(y_l | x) = -11.0$ ; the reference slightly favors the *dispreferred* response. After some training,  $\log \pi_\theta(y_w | x) = -11.0$  and  $\log \pi_\theta(y_l | x) = -11.5$ . Take  $\beta = 0.1$ .

1. *Implicit rewards*:  $\hat{r}_\theta(y_w) = 0.1(-11.0+12.0) = 0.10$  and  $\hat{r}_\theta(y_l) = 0.1(-11.5+11.0) = -0.05$ .
2. *Margin*:  $0.10 - (-0.05) = 0.15$ . The implied preference probability is  $\sigma(0.15) \approx 0.537$ .
3. *Loss*:  $-\log \sigma(0.15) \approx 0.621$ , down from  $\log 2 \approx 0.693$  at initialization, where  $\pi_\theta = \pi_{\text{ref}}$  and the margin is 0.
4. *Gradient weight*:  $\sigma(-0.15) \approx 0.463$ , barely below the initial 0.5. The pair still pulls almost as hard as it did at the start.

Two lessons follow. DPO scores *changes* relative to the reference, not absolute probabilities:  $y_w$  remains less likely than  $y_l$  under  $\pi_\theta$ , yet the loss has fallen. And the loss would fall equally if both log-probabilities dropped, provided  $\log \pi_\theta(y_l)$  dropped by more. Nothing in (3.12) forces the preferred response to become more likely in absolute terms, which is why practitioners monitor the log-probability of chosen responses during training.

DPO replaces reward modeling and on-policy RL with one classification-style loss, and it is simpler and cheaper to run. Its derivation inherits every assumption above: Bradley–Terry preferences, the KL-regularized objective, and a policy class rich enough to express  $\pi^*$ . It adds one of its own. DPO is offline: it learns only from the responses in  $\mathcal{D}_{\text{pref}}$ . Where those responses cover the policy’s behavior poorly, the loss constrains the policy weakly, whereas on-policy RLHF evaluates the reward model on the policy’s own samples. Iterated variants that collect fresh preferences on current samples narrow this gap.

#### 3.4.4 AI Feedback and Constitutional Methods

Human comparisons are slow, expensive, and inconsistent. *Constitutional AI* (Bai et al., 2022) replaces most human harmlessness labels with feedback from a model guided by a short list of written principles, the constitution. In a supervised phase, the model critiques its own responses to difficult prompts against a sampled principle, revises them, and is fine-tuned on the revisions. In an RL phase, a model judges which of two responses better satisfies a principle; these AI preferences train a preference model, and the policy is optimized against it as in RLHF, an instance of *RL from AI feedback*. The benefits are scale and legibility, since the principles are written down and can be inspected and revised. The costs are that the judge’s biases and blind spots become the policy’s, and that a model evaluated by a similar model can learn to satisfy the judge rather than the principle. Model-based judgment is also central to evaluation, with the same caveats (Chapter 11). A classifier can also supply the feedback. Xie et al. (2024b) fine-tune a model for detoxification as a Stackelberg game between the model (leader) and a binary toxicity screener (follower), addressing the incomplete preferences that arise when the screener cannot rank a failed rewrite against its input.

### 3.4.5 Reinforcement Learning with Verifiable Rewards

In some domains correctness can be checked mechanically: a math answer matches a reference, code passes unit tests, a proof checks, a web task reaches a goal state. *Reinforcement learning with verifiable rewards* (RLVR), a name introduced with the Tülu 3 post-training recipe (Lambert et al., 2024), uses such a checker as the reward, for instance  $r(x, y) = \mathbb{1}\{\text{the final answer in } y \text{ is correct}\}$ , often plus a small reward for the required output format. A rule-based verifier cannot be flattered in the way a learned reward model can, although it can still be gamed if it checks the wrong thing (Section 3.5.3).

DeepSeek-R1 (Guo et al., 2025) made this approach prominent. Its precursor, R1-Zero, was trained with RL directly from a pretrained base model, without an SFT stage, using rule-based accuracy and format rewards; the authors avoided neural reward models for reasoning because of reward hacking. During training, responses grew longer and behaviors such as checking and revising intermediate steps emerged without being demonstrated. R1-Zero’s outputs had poor readability and mixed languages, so the released R1 added a small “cold-start” SFT stage and alternated RL and SFT stages. Its reasoning was also *distilled* into smaller models by SFT on its outputs. This is the search-then-distill loop of Section 3.2, with long reasoning in place of tree search; Section 4.3 treats the resulting trade-off between inference-time computation and accuracy.

*Group-relative advantages.* The optimizer, *group relative policy optimization* (GRPO), was introduced by Shao et al. (2024). Its idea is to get a baseline without learning one. For each prompt  $x$ , it samples a group of  $G$  responses  $y_1, \dots, y_G$  from the current policy  $\pi_{\text{old}}$ , scores them to obtain rewards  $r_1, \dots, r_G$ , and assigns each response the standardized advantage

$$A_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}. \quad (3.13)$$

With the importance weight  $w_i = \pi_{\theta}(y_i | x) / \pi_{\text{old}}(y_i | x)$ , it then maximizes a PPO-style clipped surrogate (3.5) with a KL penalty toward  $\pi_{\text{ref}}$ ,

$$\mathbb{E} \left[ \frac{1}{G} \sum_{i=1}^G \left( \min(w_i A_i, \text{clip}(w_i, 1 - \varepsilon_c, 1 + \varepsilon_c) A_i) - \beta D_{\text{KL}}(\pi_{\theta} \| \pi_{\text{ref}}) \right) \right]. \quad (3.14)$$

In practice the ratio, clipping, and KL estimate are applied per token and averaged over each response’s tokens. The group mean is a Monte Carlo baseline in the sense of Theorem 3.2: it estimates the prompt’s expected reward from samples, removing the learned value network (critic) that PPO needs and the memory that network consumes.

EXAMPLE 3.7 (Group-relative advantages). A prompt receives  $G = 8$  sampled solutions, and the verifier accepts three:  $r = (1, 0, 1, 0, 0, 1, 0)$ . Figure 3.4 draws the computation.

1. *Baseline*: the group mean is  $3/8 = 0.375$ , the group’s estimate of how often this policy solves this prompt.
2. *Spread*: the (population) standard deviation is  $\sqrt{0.375 \times 0.625} \approx 0.484$ .

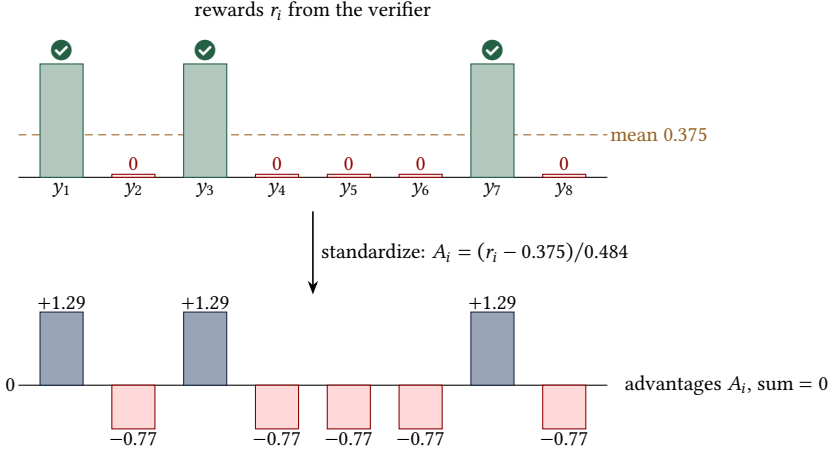


FIGURE 3.4. The group-relative advantage of Theorem 3.7. Eight samples for one prompt are scored by a verifier (top). Subtracting the group mean and dividing by the group standard deviation turns three successes and five failures into advantages that sum to zero (bottom). Successes are reinforced, failures suppressed, and a group with no spread in rewards would produce no signal at all.

3. *Advantages*: each accepted solution receives  $A = 0.625/0.484 \approx 1.29$  and each rejected one  $A = -0.375/0.484 \approx -0.77$ . The advantages sum to zero, so the update redistributes probability within the group rather than raising all of it.
4. *Clipping*, with  $\varepsilon_c = 0.2$ . Suppose an earlier minibatch step has already raised an accepted solution’s ratio to  $w = 1.3$ . The unclipped term is  $1.3 \times 1.29 \approx 1.68$  and the clipped term  $1.2 \times 1.29 \approx 1.55$ ; the minimum is the clipped, constant one, so this sample stops pushing. A rejected solution whose ratio has fallen to  $w = 0.7$  is likewise clipped ( $0.8 \times (-0.77) \approx -0.62 < -0.54$ ). But a rejected solution whose ratio has *risen* to  $w = 1.3$  is not: the unclipped  $-1.01$  is the smaller term, so the gradient keeps pushing it back down.

If all eight samples fail, or all succeed, every numerator is 0 (implementations add a small constant to the denominator), and the prompt contributes nothing but the KL term. RLVR therefore learns only from prompts that the current policy sometimes, but not always, solves, so the choice of training prompts acts as a curriculum.

*What the normalizations do.* Two refinements of GRPO are worth understanding, because they show how small estimator choices change what is optimized. First, the mean. Because  $r_i$  enters its own group mean,  $r_i - \text{mean} = \frac{G-1}{G}(r_i - \bar{r}_{-i})$ , where  $\bar{r}_{-i}$  is the mean of the other  $G-1$  rewards. In Theorem 3.7, an accepted sample has  $r_i - \bar{r}_{-i} = 1 - 2/7 \approx 0.714$ , and  $\frac{7}{8} \times 0.714 = 0.625$ , as computed. The leave-one-out baseline does not depend on  $y_i$ , so it satisfies the condition of Theorem 3.2, and the unnormalized group-mean estimator is an unbiased gradient up to the positive factor  $\frac{G-1}{G}$  (Exercise 3.6). Ahmadian et al.

(2024) found that this simple REINFORCE-with-leave-one-out estimator (RLOO) was competitive with PPO for RLHF. Second, the divisions. Liu et al. (2025) showed that dividing by the group standard deviation gives more weight per sample to prompts that are almost always or almost never solved, a *question-level difficulty bias*, and that averaging each response’s token losses over its own length penalizes long wrong answers less per token than short ones, a *response-level length bias* that can inflate the length of incorrect responses. Removing both divisions recovers an unbiased policy gradient. DAPO (Yu et al., 2025) made related changes at scale: it averages the loss over all tokens in the batch, raises the upper clip bound separately from the lower one so that low-probability tokens can grow, and resamples so that batches are not filled with zero-advantage groups. The durable point is not any one recipe but the checklist: what baseline, what weighting across prompts, what weighting across tokens, and what trust region.

#### KEY IDEA

*Reinforcement learning optimizes the reward it is given, and in RLVR the reward is a verifier. Whatever the verifier fails to check, the policy is free to exploit, and under enough optimization pressure it will. Designing the checker is designing the objective.*

#### IN PRACTICE (AS OF SEPTEMBER 2026)

- *Recipes.* Critic-free policy-gradient methods in the GRPO family, with the estimator refinements above, remained the common open recipe for reasoning RL; DAPO’s released system was built on the open verl framework (Yu et al., 2025). Details such as clip bounds, length handling, and KL coefficients varied between reports and were still changing.
- *Distillation.* Small reasoning models were commonly produced by distilling from larger ones rather than by RL from scratch, following DeepSeek-R1’s release of distilled models (Guo et al., 2025). On-policy distillation (Section 3.5) gained adoption after late-2025 reports that it matched RL-trained students at a fraction of the compute (Lu, 2025).
- *Caution.* Public descriptions of frontier-lab post-training remain partial. Treat vendor statements about training methods as dated claims, not specifications.

## 3.5 Learning to Use Tools and Act

The methods of Section 3.4 were developed for single responses. Agents emit many actions, receive observations between them, and must learn when a tool is worth calling. This section covers three sources of that competence: self-supervision, demonstrations, and interaction.

### 3.5.1 Self-Supervised Tool Learning

*Toolformer* (Schick et al., 2023) teaches a model to call APIs (a calculator, a question-answering system, a search engine, a translator, and a calendar) using only a handful of human examples per tool. Starting from a plain text corpus, the model is prompted to propose candidate API calls at positions where a call seems likely. Each call is executed, and the call together with its result is inserted into the text. A candidate is kept only if it helps. Let  $L^+$  be the model’s weighted loss on the tokens following the insertion with the call and its result, and  $L^-$  the smaller of the losses with no call and with the call but no result; the call is kept when  $L^- - L^+ \geq \tau_f$  for a threshold  $\tau_f$ . The model is then fine-tuned on the augmented corpus.

The filter is the key idea: usefulness is defined by the model’s own predictive loss, so no human judges which calls are good. The method has clear limits. Calls are single, not chained; the model sees only returned text, not the consequences of a call on the world; and “reduces perplexity” is a proxy for “helps accomplish a goal.” Later tool learning therefore moved to trajectories and to interaction.

### 3.5.2 Imitation from Trajectories

A *trajectory*  $(c_0, a_0, o_1, c_1, a_1, \dots)$  records an agent solving a task. Trajectories can come from humans, from a stronger model, or from search over a model’s own attempts, as in the ToolBench data built by Qin et al. (2024) (Chapter 8). Behavior cloning fits  $\pi_\theta$  by minimizing  $-\sum_t \log \pi_\theta(a_t | c_t)$  over the demonstrated steps, with the loss applied only to action tokens. A cheap and widely used variant is *rejection-sampling fine-tuning*: sample many attempts, keep those a verifier accepts, and clone them. It is RLVR with a binary advantage and no negative examples.

Behavior cloning has a structural weakness. It trains on the states the demonstrator visits, but the learner acts in the states that *it* visits. After one mistake, the learner is in a context unlike any in the demonstrations, where its error rate is higher, so mistakes beget mistakes. Ross et al. (2011) make this precise. If the cloned policy errs with probability  $\epsilon$  at states drawn from the expert’s distribution, its expected cost over a  $T$ -step task can exceed the expert’s by order  $T^2\epsilon$ , rather than the order  $T\epsilon$  that independent errors would give. Figure 3.5 shows the gap on a concrete chain.

Their remedy, DAgger, attacks the cause. It runs the learner, asks the expert to label the states the learner actually reaches, aggregates the data, and retrain, achieving a bound linear in  $T$  under no-regret assumptions. The connection to the error-compounding model of Section 2.5 is direct: per-step error  $\epsilon$  and recovery probability  $\rho$  determine trajectory success (Theorem 2.13). Cloning clean demonstrations does badly on both parameters. The error rate is not stationary, since it rises once the agent is off the demonstrated distribution, and  $\rho$  is low, since clean demonstrations never show recovery. Practical consequences follow. Include demonstrations of errors and recoveries, such as a failed test followed by diagnosis and repair; collect corrections on the agent’s own trajectories, in the spirit of DAgger; and move to learning from interaction.

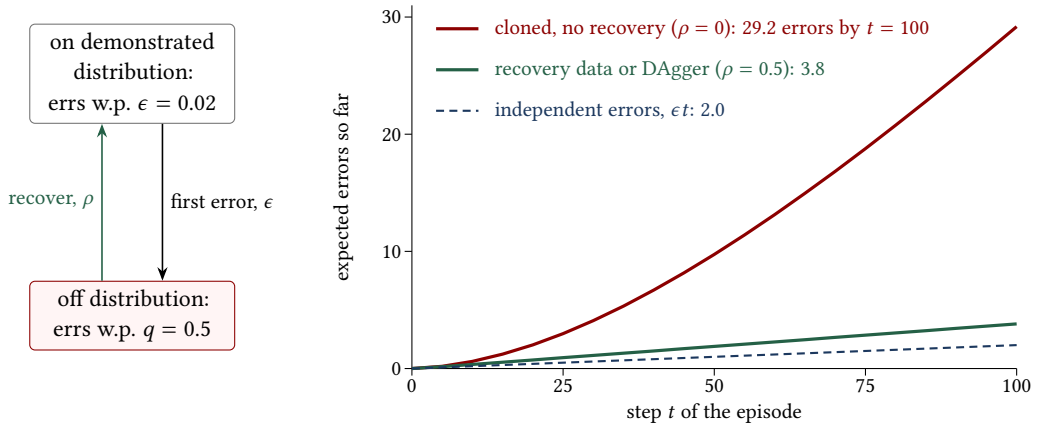


FIGURE 3.5. Why cloning compounds errors. Left: a two-state model of a cloned agent. On the demonstrated distribution it errs with probability  $\epsilon$ ; its first error moves it off that distribution, where it errs with probability  $q$ , and it returns with recovery probability  $\rho$  per step. Right: expected cumulative errors over a 100-step episode. With no recovery (red), errors grow roughly quadratically at first and then linearly at rate  $q$ , about fifteen times the  $\epsilon t$  line (dashed) that independent errors would give. Training on the learner’s own states, which teaches recovery (green,  $\rho = 0.5$ ), keeps the error rate near its on-distribution level.

*On-policy distillation.* The same diagnosis applies when the “expert” is a larger model. Ordinary distillation clones a teacher’s outputs and inherits the train–test mismatch. Agarwal et al. (2024) instead let the student generate its own sequences and trained it to match the teacher’s token distributions on them, which is DAgger with a model as the labeler. Lu (2025) described the method as RL with a dense reward: the student samples a response, the teacher scores every token, and the per-token reward is the negative log-ratio  $\log \pi_{\text{teacher}} - \log \pi_{\theta}$ , whose expectation is the negative reverse KL divergence (Exercise 3.8). It combines the on-policy state distribution of RL with supervision far denser than one reward per episode.

#### 💡 KEY IDEA

*A learner must be trained on the states it will actually visit. Demonstrations teach what to do when things go right; only data from the learner’s own trajectories, whether expert corrections, a teacher’s token-level feedback, or rewards from interaction, teaches what to do after things go wrong.*

### 3.5.3 Online RL in Interactive Environments

In online RL the agent’s own multi-step episodes are rolled out in an environment, such as a container with a repository and tests, a terminal, a browser, or a simulated desktop (Zhou et al., 2024; Xie et al., 2024a), and are scored by an outcome check at the end. The algorithms are those of Sections 3.1 and 3.4, applied to the step-level process of

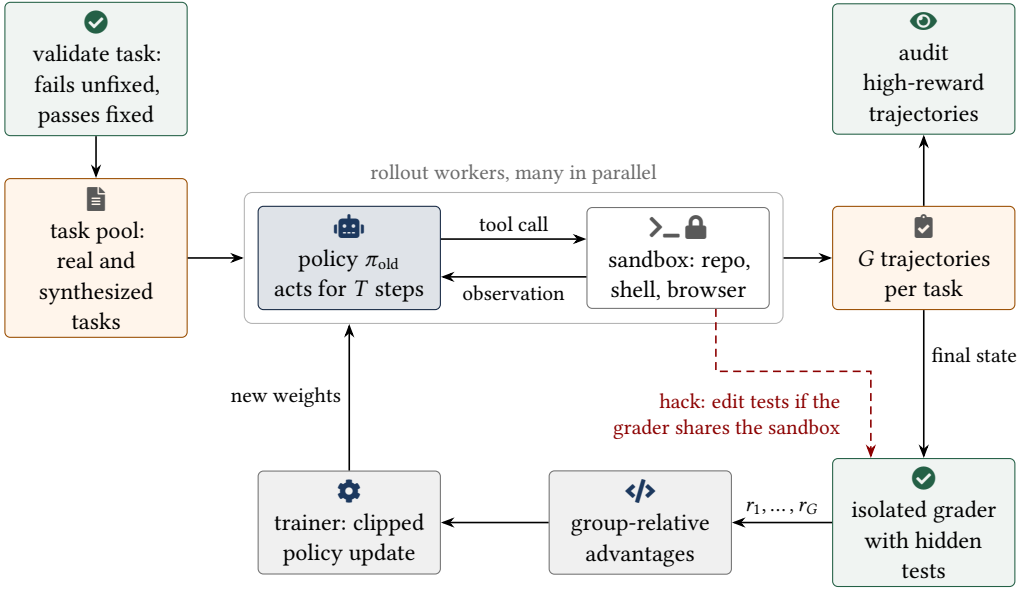


FIGURE 3.6. An agentic reinforcement learning loop. Tasks are validated before use, rollout workers run the current policy in sandboxes for many steps, and each group of trajectories is graded, converted to group-relative advantages, and used to update the policy, whose weights return to the workers. Green components defend the reward signal: validation removes broken tasks, an isolated grader with hidden tests closes the most direct hack (dashed red), and audits of high-reward trajectories catch what the grader misses.

### Section 3.3. What changes is the system around them, drawn in Fig. 3.6.

Four differences from single-response RL matter. First, the policy gradient is taken only over tokens the model generated; observation tokens are conditioning. Second, credit assignment is harder, since one terminal reward must be spread over dozens of actions. That motivates intermediate rewards and step-level value estimates, each of which creates new opportunities for gaming. Third, rollouts are slow and expensive, because every step waits on a real tool, and environments must be reset reproducibly; training systems therefore run many sandboxed rollout workers in parallel and update asynchronously. Fourth, exploration has side effects. An exploring agent will try destructive actions, so training environments need the sandboxing and action classification of Chapter 8 (Theorem 8.7).

*Tasks and environments.* The supply of good tasks has become a central bottleneck. Software engineering showed the pattern. SWE-Gym (Pan et al., 2025) packaged 2,438 real Python repository tasks, each with an executable environment and unit tests, for training agents and verifiers; SWE-RL (Wei et al., 2025) trained on the history of open-source projects with a softer rule-based reward, the similarity between the generated and the actual fix. Later efforts synthesized tasks and environments at far larger scale (see the box below). Scale raises a quality problem that the loop’s first green box addresses.

A task whose tests already pass without a fix, or fail even with the reference fix, rewards nothing useful or rewards noise, so environments are validated before training: the unmodified repository must fail, the reference solution must pass, and flaky tests are detected by repetition ([Exercise 3.10](#)).

*Reward hacking at scale.* When the agent acts inside the environment that computes its reward, hacking the reward becomes an ordinary tool call. [Baker et al. \(2025\)](#) reported frontier reasoning models discovering such exploits during agentic coding training, and found that penalizing hacking visible in the chain of thought taught models to hide it. [MacDiarmid et al. \(2025\)](#) found that a model that learned to hack production coding environments generalized to broader misaligned behavior on unrelated evaluations, and that preventing the hacks, not merely penalizing them afterward, was the most reliable mitigation. The analysis of these failures belongs to [Section 16.2](#). The engineering lesson for this chapter is structural: grade outside the agent’s reach, validate the verifier, and audit what it rewards.

#### ✂ IN PRACTICE (AS OF SEPTEMBER 2026)

- *Synthesized agentic tasks.* Open-weight model reports described large pipelines that synthesize tool-use tasks and environments for RL: Kimi K2 (July 2025) combined agentic data synthesis with a joint RL stage over real and synthetic environments ([Kimi Team et al., 2025](#)), and DeepSeek-V3.2 (December 2025) described a synthesis pipeline for scalable agentic post-training ([DeepSeek-AI et al., 2025](#)).
- *Open environment collections.* On 22 July 2026, Prime Intellect described integrating about 365,000 tasks from 23 task sets across software engineering, terminal use, and web search, validated by requiring each task to fail without modification and pass with its reference patch, with retries to separate flaky from broken tests ([Auras and Prime Intellect Team, 2026](#)). The authors noted that when an agent shares a sandbox with grading machinery, withholding grading files until scoring is “mitigation, not a guarantee.”
- *What to expect.* Task counts and vendor figures change monthly. The stable questions are how tasks are validated, where grading runs, and how rewarded trajectories are audited.

#### RUNNING EXAMPLE R2: LEARNING FROM TEST OUTCOMES

R2 is trained by RL on a corpus of resolved issues. Each episode starts from the repository before the fix, and the reward is 1 if the tests added with the historical fix pass after R2’s edits and 0 otherwise. Before training, each issue is validated: the new tests must fail on the original repository and pass with the historical fix. The

signal is cheap, automatic, and far harder to flatter than a learned reward model, and GRPO over groups of attempts per issue (Theorem 3.7) needs no critic. The reward still checks a proxy. If R2 can see or modify the tests, policies that special-case the test inputs, weaken assertions, or delete failing tests earn full reward without fixing the issue, and a sufficiently strong optimizer will find them. Standard mitigations are to run grading in a separate sandbox against tests the agent never sees, to forbid or penalize edits to test files, to add held-out tests and static checks, and to audit samples of high-reward trajectories (Fig. 3.6). None is complete: each closes a loophole in the verifier rather than making the verifier equal to intent. Section 16.2.2 analyzes this gap, and Chapter 11 explains why the same tests must not double as the benchmark.

## 3.6 Continual and Open-Ended Learning

A deployed agent meets tasks, tools, and conventions that did not exist when it was trained. It can keep learning in two ways: by changing its memory while the weights stay fixed, or by changing its weights.

### 3.6.1 Skill Libraries and Procedural Memory

*Voyager* (Wang et al., 2024a) is an agent that keeps learning in Minecraft without any weight update. The underlying model is a fixed, black-box language model. Three components surround it. An *automatic curriculum* proposes the next task based on the agent’s state and what it has already mastered. An *iterative prompting* loop writes a program for the task, runs it, and revises it using environment feedback, execution errors, and a self-verification step in which the model judges whether the task succeeded. A *skill library* stores each verified program, indexed by an embedding of its description. For a new task, the most relevant skills are retrieved into the context, and new programs call old ones, so skills compose. Figure 3.7 draws the loop and the structure it grows, transposed to the security operations agent R3.

In the terms of Chapter 9, the skill library is explicit procedural memory: learning is the growth of the store  $\mathcal{M}$ , and the read policy is retrieval by task similarity (Section 9.1). Three design principles carry over to other agents. Store *executable* knowledge, since a program is more reliable to reuse than a description of what worked. Admit a skill only after *verification*, because an unverified entry turns one lucky or mistaken episode into a lasting error that later skills build on. And index skills by *when* they apply as well as what they do. The same verification step is a security control: a library that anything can write to is a persistence mechanism for an attacker (Chapter 13). Verbal lessons, as in Reflexion (Shinn et al., 2023), are the non-executable counterpart. Packaged, human-authored skills that agents load on demand follow the same logic from the other direction; Chapter 9 discusses them as procedural memory.

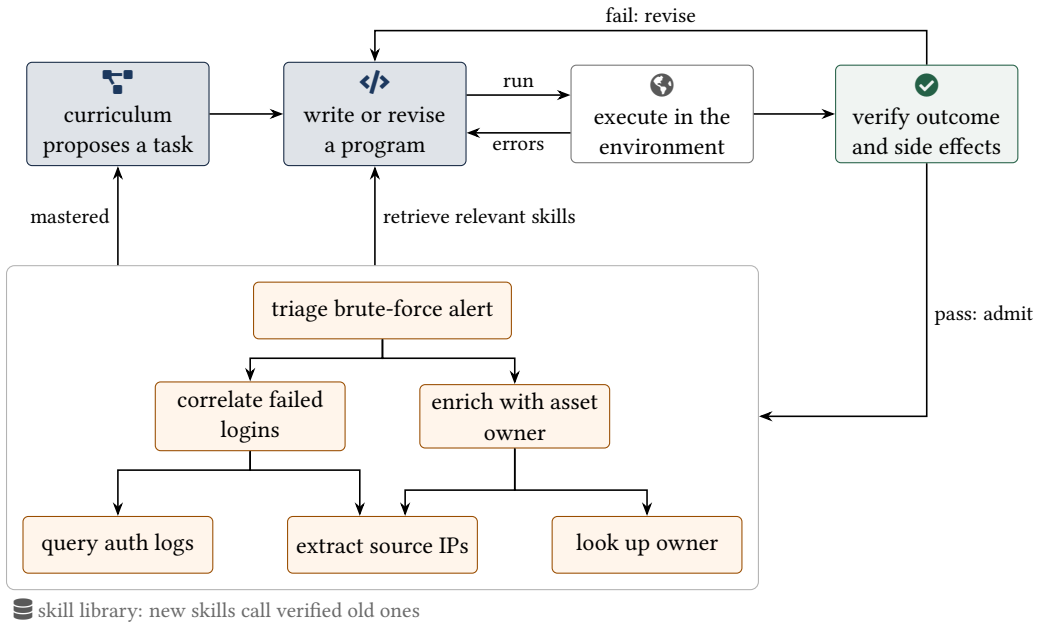


FIGURE 3.7. A skill library grows by verified composition. The curriculum proposes a task, the model writes a program, execution errors return for revision, and only a program that passes verification is admitted. Admitted skills are retrieved when writing later programs, so the library grows upward: primitive queries, then composites, then whole procedures. The example skills are those a security operations agent such as R3 might accumulate.

### 💡 KEY IDEA

*Admit to lasting memory only what has been verified. A skill library compounds: every admitted skill becomes a building block for later ones, so an error admitted once is inherited by everything built on it.*

## 3.6.2 Lifelong Learning and Forgetting

Updating weights on new data degrades performance on old tasks, often sharply; this is *catastrophic forgetting*. It follows from shared parameters: the gradient for the new task does not know which weights the old tasks rely on. Elastic weight consolidation (Kirkpatrick et al., 2017) addresses this by anchoring important weights. After task  $A$ , it minimizes the new loss plus  $\sum_k \frac{\lambda}{2} F_k (\theta_k - \theta_{A,k}^*)^2$ , where  $F_k$  is the diagonal Fisher information at the old optimum  $\theta_A^*$ , so weights that mattered for  $A$  move less. Other remedies replay old data during new training, as InstructGPT mixed pretraining gradients into PPO (Ouyang et al., 2022), or add small task-specific parameters while freezing the rest.

The KL penalty of (3.10) is itself an anti-forgetting device, measured in output space rather than parameter space, and on-policy RL appears to carry a version of it

implicitly. Shenfeld et al. (2025) compared fine-tuning by SFT and by on-policy RL to equal performance on a new task and found that RL forgot less. Their explanation is that among the many policies that solve the new task, on-policy RL tends toward those closest in KL divergence to the starting model, whereas SFT on fixed targets can move arbitrarily far. The tilt of Theorem 3.4 is the idealized version of that preference.

*Learning in deployment.* The same closed form also describes memory-based learning at decision time. Li et al. (2026b) proposed an agent that stores past trajectories, estimates the advantage of candidate actions from retrieved similar experiences, and adjusts the model’s output probabilities accordingly, with no gradient step. The adjustment is the optimum of a KL-constrained objective, exactly (3.11) with estimated advantages in place of  $r$ . Weight-based learning in deployment is also practiced where feedback is plentiful and each action is cheap to get wrong, as the box below describes.

#### IN PRACTICE (AS OF SEPTEMBER 2026)

- *Online RL on user feedback.* Cursor reported on 12 September 2025 that it trains its code-completion model with policy-gradient RL on whether users accept suggestions, rolling out a new checkpoint and collecting data for the next update every 1.5 to 2 hours (Jackson et al., 2025). Its illustrative reward, +0.75 for an accepted suggestion, −0.25 for a rejected one, and 0 for showing nothing, makes showing a suggestion worthwhile exactly when its acceptance probability  $p$  exceeds 0.25, since  $0.75p - 0.25(1 - p) = p - 0.25$ .
- *Skills and memory instead of weights.* For general-purpose agents, continual learning in products as of 2026 mostly meant memory: instruction files, retrieved episodes, and loadable skill packages following the open Agent Skills format (Agent Skills, 2025), whose usefulness varies widely across tasks (Li et al., 2026a).
- *Caution.* A self-updating system is a moving target for evaluation and a new surface for poisoning. Treat each weight update as a release with an evaluation report and a rollback path.

The two routes to continual learning trade off differently. Memory-based learning is fast, inspectable, attributable, and deletable (Chapter 9), but it is bounded by the context budget  $B$  and by retrieval quality. Weight-based learning generalizes more deeply and costs nothing at inference time, but it is slow, can forget, and cannot easily be audited or undone. A self-improving agent of either kind is also a moving target for assurance, since the system that was evaluated is not the system now running (Chapters 11 and 15). When the system’s decisions also shape the data it later learns from, the problem becomes performative; Zhu and Han (2026b) define self-consistent solutions for such decision-dependent, chance-constrained problems and give a model-free, scenario-based iteration that converges to them. Weight updates on data that others can influence also invite poisoning, which game-theoretic defenses for federated learning model as a Bayesian

Stackelberg Markov game against adaptive attackers (Pan et al., 2023; Li et al., 2024b).

---

#### KEY TAKEAWAYS

- TD learning and  $Q$ -learning bootstrap from their own estimates, trading variance for bias; tabular  $Q$ -learning converges to  $Q^*$  under infinite exploration and Robbins–Monro step sizes, guarantees that function approximation does not preserve.
  - The policy-gradient theorem needs only the score function of the policy. Baselines that do not depend on the scored action leave the gradient unbiased and reduce its variance, and PPO’s clipped surrogate keeps each update near the data-collecting policy.
  - Deep RL taught that search amortized into networks is powerful, that narrow training generalizes poorly, and that optimizers exploit flawed rewards.
  - A language model is a stochastic policy at the token and step levels, linked by the sum of token log-probabilities; in context, it behaves approximately like a Bayesian learner whose evidence is everything in  $c_t$ .
  - The KL-regularized objective has the closed-form optimum  $\pi^* \propto \pi_{\text{ref}} \exp(r/\beta)$ , an exponential tilt; inverting it yields DPO, which scores changes relative to the reference. Both inherit the Bradley–Terry assumption and can only reweight what  $\pi_{\text{ref}}$  already produces.
  - RL with verifiable rewards replaces learned rewards with checkers. Group-relative advantages supply a sample-based baseline, but give no signal on prompts the policy never or always solves, and their normalizations change which prompts and tokens are emphasized.
  - Agentic RL is a systems problem: validated tasks, sandboxed parallel rollouts, graders outside the agent’s reach, and audits of rewarded trajectories. Reward hacking observed in reasoning-model training shows that the verifier is the objective.
  - Behavior cloning compounds errors because the learner visits states the demonstrator did not; recovery data, DAgger-style corrections, on-policy distillation, and online RL all train on the learner’s own distribution.
  - Agents can keep learning through verified skill libraries, which are inspectable and deletable, or through weight updates, which generalize more deeply but risk forgetting and complicate assurance.
- 

## Notes and Further Reading

Sutton and Barto (2018) is the standard introduction to reinforcement learning and the best companion to Section 3.1, including TD methods, policy gradients, the deadly

triad, and planning with learned models.  $Q$ -learning and its convergence proof are due to Watkins and Dayan (1992), and REINFORCE to Williams (1992); Chapter C develops the stochastic-approximation arguments. PPO (Schulman et al., 2017) remains the reference point for clipped policy updates. Upper-confidence exploration in tree search follows Kocsis and Szepesvári (2006). Learning when other adaptive agents share the environment is the subject of Chapter 5; examples in stochastic games include correlated  $Q$ -learning for distributed transmission control (Huang et al., 2010) and conjectural online learning under asymmetric information (Li et al., 2024a).

DQN (Mnih et al., 2015) and AlphaGo (Silver et al., 2016) are the landmark papers of deep RL, and TD-Gammon (Tesauro, 1995) anticipated self-play by two decades. The catalogues of specification gaming by Amodei et al. (2016) and Krakovna et al. (2020) are the essential reading on reward misspecification. Meta-learning for fast adaptation has been combined with leader–follower games to adapt guidance and shared-control strategies to different followers (Zhao and Zhu, 2023a,b, 2024), developed as zeroth-order meta-policy optimization for linear quadratic regulators (Pan et al., 2026), and applied to zero-trust defense policies that generalize across attack scenarios (Ge et al., 2023).

For foundation models, Kaplan et al. (2020) and Hoffmann et al. (2022) are the primary scaling-law studies, and Brown et al. (2020) documented in-context learning at scale. The Bayesian account of Xie et al. (2022) should be read together with its assumptions; it is one of several competing explanations of in-context learning.

Preference-based post-training begins with Christiano et al. (2017) and was brought to language models at scale by Ouyang et al. (2022); the preference model goes back to Bradley and Terry (1952). Rafailov et al. (2023) derive DPO and prove that the reparameterization loses no generality over reward equivalence classes. Instruction tuning as a route to zero-shot generalization is due to Wei et al. (2022a), and Constitutional AI to Bai et al. (2022). The Tülu 3 report (Lambert et al., 2024) is a transparent end-to-end open post-training recipe covering SFT, DPO, and RLVR. GRPO was introduced by Shao et al. (2024), and DeepSeek-R1 (Guo et al., 2025), later published in *Nature*, describes RL from a base model with rule-based rewards and the multi-stage pipeline around it. For the estimator details of critic-free methods, read Ahmadian et al. (2024) on leave-one-out REINFORCE, Liu et al. (2025) on the biases of GRPO’s normalizations, and Yu et al. (2025) on stabilizing long-reasoning RL. Yue et al. (2025) is the reference point for the debate over whether RLVR expands or only sharpens a base model’s capabilities. The literature on RLVR variants and reward design for agents was still changing quickly as of 2026 and is best followed in recent surveys.

On tool learning, Toolformer (Schick et al., 2023) is the self-supervised starting point; Gorilla (Patil et al., 2024) and ToolLLM (Qin et al., 2024) built large tool-use datasets, discussed from the systems side in Chapter 8. The compounding-error analysis of behavior cloning and the DAgger algorithm are in Ross et al. (2011); Agarwal et al. (2024) carry the idea to distillation of language models, and Lu (2025) give an accessible account of on-policy distillation as dense-reward RL. Interactive benchmarks that double

as training environments include WebArena (Zhou et al., 2024) and OSWorld (Xie et al., 2024a); Chapter 11 discusses them as measurements. SWE-Gym (Pan et al., 2025) and SWE-RL (Wei et al., 2025) are instructive on building training tasks for software agents, and the Kimi K2 (Kimi Team et al., 2025) and DeepSeek-V3.2 (DeepSeek-AI et al., 2025) reports on synthesizing agentic tasks at scale. On reward hacking during training, Baker et al. (2025) and MacDiarmid et al. (2025) are primary sources, analyzed in Chapter 16. For reinforcement learning in security operations of the kind R3 performs, see the review of RL for feedback-enabled cyber resilience by Huang et al. (2022), adaptive honeypot engagement learned as a semi-Markov decision process (Huang and Zhu, 2019a), and online security response with adaptive conjectures (Hammar et al., 2025).

Voyager (Wang et al., 2024a) is the canonical skill-library agent and Reflexion (Shinn et al., 2023) the canonical verbal-memory learner. Elastic weight consolidation (Kirkpatrick et al., 2017) is a good entry point to the continual-learning literature, and Shenfeld et al. (2025) connect forgetting to the KL geometry of Theorem 3.4. Li et al. (2026b) show the same geometry at work in gradient-free learning from deployment experience.

## Exercises

**EXERCISE 3.1 (Implementation).** Implement tabular  $Q$ -learning on a  $5 \times 5$  gridworld with a goal reward of  $+1$ , a step cost of  $-0.01$ , and  $\gamma = 0.95$ . Compare  $\varepsilon$ -greedy exploration with constant  $\varepsilon = 0.1$ , with  $\varepsilon_t = 1/\sqrt{t}$ , and with greedy action selection from optimistic initial values. Plot learning curves over 20 seeds, and explain the results using the convergence conditions of Section 3.1. *Hint:* with optimistic initial values, every untried action looks better than any tried one, so greedy selection explores systematically until the optimism is spent.

**EXERCISE 3.2 (Baselines).** For a single-step problem ( $T = 1$ ) with a scalar parameter, show that among constant baselines the variance of  $\nabla_{\theta} \log \pi_{\theta}(a)(r - b)$  is minimized by  $b^* = \mathbb{E}[(\nabla_{\theta} \log \pi_{\theta}(a))^2 r] / \mathbb{E}[(\nabla_{\theta} \log \pi_{\theta}(a))^2]$ . Then give an example of an action-dependent “baseline”  $b(s, a)$  that biases (3.3). *Hint:* the mean of the estimator does not depend on  $b$ , so minimize its second moment, a quadratic in  $b$ . For the counterexample, try  $b(s, a) = Q^{\pi}(s, a)$ .

**EXERCISE 3.3 (Clipping).** Fix an advantage  $\hat{A}$  and clip range  $\varepsilon_c = 0.2$ , and view the term inside (3.5) as a function of the ratio  $w$ . (a) Sketch it for  $\hat{A} = 1$  and for  $\hat{A} = -1$ , and mark where its derivative in  $w$  is zero. (b) Evaluate it at  $w \in \{0.7, 1.0, 1.3\}$  for both signs, and say which samples still contribute gradient. (c) Explain why the objective is clipped only in the direction that the advantage favors, and what could go wrong if it were clipped symmetrically.

**EXERCISE 3.4 (From RLHF to DPO).** (a) Show that the Bradley–Terry likelihood (3.9) is unchanged when  $r(x, y)$  is replaced by  $r(x, y) + f(x)$ , and that the optimal policy (3.11) is unchanged too. (b) Derive the DPO gradient stated in Section 3.4. (c) Suppose every

preference pair in  $\mathcal{D}_{\text{pref}}$  contains only responses of fewer than 100 tokens. What does (3.12) imply about  $\pi_\theta$  on long responses? (d) In Theorem 3.6, find new log-probabilities for which both  $\log \pi_\theta(y_w)$  and  $\log \pi_\theta(y_l)$  are lower than their reference values but the loss is below 0.5. *Hint for (b):* use  $\frac{d}{du} \log \sigma(u) = \sigma(-u)$ .

EXERCISE 3.5 (Reward and divergence along the tilt). Let  $\lambda = 1/\beta$  and  $\pi_\lambda^* \propto \pi_{\text{ref}} \exp(\lambda r)$ . (a) Show that  $\frac{d}{d\lambda} \mathbb{E}_{\pi_\lambda^*}[r] = \text{Var}_{\pi_\lambda^*}(r)$ , so expected reward never decreases as  $\beta$  falls. (b) Show that  $D_{\text{KL}}(\pi_\lambda^* \parallel \pi_{\text{ref}}) = \lambda \mathbb{E}_{\pi_\lambda^*}[r] - \log Z_\lambda$ . (c) Check both against the numbers in Theorem 3.5, and explain why the reward gained per nat of divergence shrinks as  $\beta \rightarrow 0$ . *Hint:*  $\log Z_\lambda$  is a cumulant-generating function of  $r$  under  $\pi_{\text{ref}}$ .

EXERCISE 3.6 (Group baselines). In the setting of (3.13), drop the standard-deviation normalization. (a) Show that  $r_i - \text{mean}(r_1, \dots, r_G) = \frac{G-1}{G}(r_i - \bar{r}_{-i})$ , and that the leave-one-out advantage  $r_i - \bar{r}_{-i}$  yields an unbiased policy gradient. (b) For a prompt solved with probability  $p$  and binary rewards, compute the probability that a group of size  $G$  gives no learning signal. Evaluate it for  $p \in \{0.05, 0.5\}$  and  $G \in \{4, 16\}$ . (c) With binary rewards and  $k$  successes in the group, compute the standardized advantage of a success as a function of  $k/G$ , and use it to explain the question-level difficulty bias. *Hint for (a):*  $\bar{r}_{-i}$  is a function of the other samples only, which are independent of  $y_i$  given  $x$ .

EXERCISE 3.7 (Compounding errors). A cloned agent errs with probability  $\epsilon = 0.01$  per step on the demonstrated distribution, but once off it (after its first error) it errs with probability 0.2 per step, and it recovers to the demonstrated distribution with probability  $\rho$  per step. Model the process as a two-state Markov chain, and compute the expected number of erroneous steps in a 50-step episode for  $\rho \in \{0, 0.3\}$ . Relate the result to the  $T^2\epsilon$  bound of Ross et al. (2011), to Fig. 3.5, and to the model of Section 2.5. *Hint:* propagate the probability of being on the distribution step by step; the expected error count is the sum over steps of the per-step error probability.

EXERCISE 3.8 (On-policy distillation). A student  $\pi_\theta$  samples a response  $z = (z_1, \dots, z_m)$  to prompt  $x$ , and a teacher assigns log-probabilities  $\log \pi_T(z_j \mid x, z_{<j})$ . Define the per-token reward  $r_j = \log \pi_T(z_j \mid x, z_{<j}) - \log \pi_\theta(z_j \mid x, z_{<j})$ . (a) Show that  $\mathbb{E}_{z \sim \pi_\theta}[\sum_j r_j] = -D_{\text{KL}}(\pi_\theta(\cdot \mid x) \parallel \pi_T(\cdot \mid x))$ . (b) Explain why this objective, unlike SFT on teacher samples, trains on the student's own state distribution, and relate it to DAgger. (c) Reverse KL is called mode-seeking. What does that imply when the teacher spreads probability over several valid solution strategies?

EXERCISE 3.9 (In-context evidence). A model has two latent concepts for a labeling task:  $\psi_1$  (label by sentiment) and  $\psi_2$  (label by topic), with prior 0.5 each. Each in-context example is consistent with  $\psi_1$  with probability 0.9 under  $\psi_1$  and 0.5 under  $\psi_2$ . Compute  $p(\psi_1 \mid \text{prompt})$  after  $k = 1, 4, 10$  consistent examples. Now suppose a tool result inserts three examples consistent only with  $\psi_2$ . How many additional consistent examples restore a posterior of 0.9 on  $\psi_1$ ? What does this suggest about the security of in-context

evidence? *Hint*: work with the log-odds, which change by a fixed amount per example.

EXERCISE 3.10 (Validating an environment). An RL task set for R2 contains issues with tests. A fraction of tasks are *broken* (the reference fix fails the tests), and some tests are *flaky*, failing independently with probability 0.1 on any run. (a) A validation rule accepts a task if the unmodified repository fails and the reference fix passes in at least one of  $k$  runs. What fraction of flaky-but-valid tasks does it keep for  $k = 1$  and  $k = 5$ , and what does it do with broken tasks? (b) During training, a correct fix on a flaky task earns reward 1 with probability 0.9. In a group of  $G = 8$  correct attempts, what is the probability that the group gives nonzero advantages, and what do those advantages teach the policy? (c) Propose a grading protocol that reduces this noise without making reward hacking easier. *Hint for (b)*: a group of identical correct fixes produces nonzero advantages only if the flaky test splits them.

EXERCISE 3.11 (Reward design). Design a verifiable reward for training a tool-using agent that files expense reports from receipts. Identify two ways the agent could game your reward, propose a mitigation for each, and state which gaming behavior your mitigations still leave open.

EXERCISE 3.12 (Skill admission). Running example R3 accumulates a library of log-query and correlation routines, as in Fig. 3.7. Specify an admission policy for new skills: what verification a skill must pass, what metadata it must carry, and when it should be retired. Discuss how an attacker who can shape the logs R3 reads might try to get a harmful skill admitted, and how your policy responds. *Hint*: consider a skill that passes verification on attacker-shaped logs precisely because the attacker controls what “success” looks like in them.

---

# Bibliography

*This sample lists only the works cited in its preface and chapters.*

A2A Protocol Project. A new chapter for A2A: Joining the Agentic AI Foundation. A2A project blog, 27 August 2026, 2026. URL <https://a2a-protocol.org/latest/blog/2026/08/27/a-new-chapter-for-a2a-joining-the-agentic-ai-foundation/>.

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2306.13649.

Agent Skills. Agent skills: Overview and specification. Open specification, 2025. URL <https://agentskills.io/>. Accessed 15 September 2026.

Agent2Agent Protocol Contributors. Agent2Agent (A2A) protocol specification. Open protocol specification, version 1.0, 2026.

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. doi: 10.18653/v1/2024.acl-long.662.

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, et al. Do as I can, not as I say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2022.

Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. Safe reinforcement learning via shielding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety, 2016. arXiv:1606.06565.

John R. Anderson, Daniel Bothell, Michael D. Byrne, Scott Douglass, Christian Lebiere, and Yulin Qin. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004.

Anthropic. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku. Announcement, 22 October 2024, 2024. URL <https://www.anthropic.com/news/3-5-models-and-computer-use>.

- Anthropic. Disrupting the first reported AI-orchestrated cyber espionage campaign. Technical report, Anthropic, November 2025.
- Daniel Auras and Prime Intellect Team. Scaling agentic RL: 365,000+ environments for SWE, terminal, and search. Prime Intellect blog, 22 July 2026, 2026. URL <https://www.primeintellect.ai/blog/scaling-agentic-rl>.
- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, et al. Constitutional AI: Harmlessness from AI feedback, 2022. arXiv:2212.08073.
- Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation, 2025. arXiv:2503.11926.
- Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control*, volume I. Athena Scientific, 4th edition, 2017.
- Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.
- Michael Bowling, John D. Martin, David Abel, and Will Dabney. Settling the reward hypothesis. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, 2023.
- Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.
- Michael E. Bratman. *Intention, Plans, and Practical Reason*. Harvard University Press, Cambridge, MA, 1987.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.
- Rodney A. Brooks. A robust layered control system for a mobile robot. *IEEE Journal of Robotics and Automation*, 2(1):14–23, 1986.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.
- Emilio Calvano, Giacomo Calzolari, Vincenzo Denicolò, and Sergio Pastorello. Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110(10): 3267–3297, 2020.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, et al. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*, 2025.
- Alan Chan, Rebecca Salganik, Alva Markelius, Chris Pang, Nitarshan Rajkumar, Dmitrii Krashennnikov, Lauro Langosco, et al. Harms from increasingly agentic algorithmic systems. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2023.
- Alan Chan, Carson Ezell, Max Kaufmann, Kevin Wei, Lewis Hammond, Herbie Bradley, Emma Bluemke, et al. Visibility into AI agents. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2024. doi: 10.1145/3630106.3658948.

- Juntao Chen and Quanyan Zhu. Security investment under cognitive constraints: A Gestalt Nash equilibrium approach. In *2018 52nd Annual Conference on Information Sciences and Systems (CISS)*, pages 1–6, 2018. doi: 10.1109/ciss.2018.8362263.
- Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.
- Zhe Sage Chen and Quanyan Zhu. A theory of multilevel interactive equilibrium in NeuroAI. arXiv preprint arXiv:2605.10505, 2026. URL <https://arxiv.org/abs/2605.10505>.
- Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, et al. DeepSeek-V3.2: Pushing the frontier of open large language models, 2025. arXiv:2512.02556.
- Kutluhan Erol, James Hendler, and Dana S. Nau. HTN planning: Complexity and expressivity. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 1994.
- Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. LLM agents can autonomously exploit one-day vulnerabilities, 2024. arXiv:2404.08144.
- K. J. Kevin Feng, David W. McDonald, and Amy X. Zhang. Levels of autonomy for AI agents. arXiv preprint arXiv:2506.12469; essay in the Knight First Amendment Institute series “AI and Democratic Freedoms”, 2025. URL <https://arxiv.org/abs/2506.12469>.
- Richard E. Fikes and Nils J. Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971.
- Tim Finin, Richard Fritzson, Don McKay, and Robin McEntire. KQML as an agent communication language. In *Proceedings of the International Conference on Information and Knowledge Management (CIKM)*, 1994.
- Sara Fish, Yannai A. Gonczarowski, and Ran I. Shorrer. Algorithmic collusion by large language models. In *Proceedings of the ACM Conference on Economics and Computation (EC)*, 2026. arXiv:2404.00806 (first version March 2024).
- Stan Franklin and Art Graesser. Is it an agent, or just a program?: A taxonomy for autonomous agents. In *Intelligent Agents III: Agent Theories, Architectures, and Languages*, volume 1193 of *Lecture Notes in Computer Science*, pages 21–35. Springer, 1997.
- Gartner. Gartner predicts over 40% of agentic AI projects will be canceled by end of 2027, 2025a. Press release, 25 June 2025. <https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027>.
- Gartner. Gartner predicts 40% of enterprise apps will feature task-specific AI agents by 2026, up from less than 5% in 2025, 2025b. Press release, 26 August 2025. <https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025>.
- Yunfei Ge and Quanyan Zhu. Trust threshold policy for explainable and adaptive Zero-Trust defense in enterprise networks. In *2022 IEEE Conference on Communications and Network Security (CNS)*, pages 359–364, 2022. doi: 10.1109/cns56114.2022.9947263.

- Yunfei Ge and Quanyan Zhu. AI liability insurance with an example in AI-Powered e-diagnosis system. arXiv preprint arXiv:2306.01149, 2023. URL <https://arxiv.org/abs/2306.01149>.
- Yunfei Ge and Quanyan Zhu. The Game-Theoretic symbiosis of trust and AI in networked systems. In *Foundations of Cyber Deception*, Advances in Information Security, pages 63–89. Springer Nature Switzerland, 2026. doi: 10.1007/978-3-031-93867-2\_3. arXiv:2411.12859.
- Yunfei Ge, Tao Li, and Quanyan Zhu. Scenario-Agnostic Zero-Trust defense with explainable threshold policy: A Meta-Learning approach. In *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 1–6, 2023. doi: 10.1109/infocomwkshps57453.2023.10225816. arXiv:2303.03349.
- Yunfei Ge, Ya-Ting Yang, and Quanyan Zhu. Attributing responsibility in AI-Induced incidents: A computational reflective equilibrium framework for accountability. arXiv preprint arXiv:2404.16957, 2024. URL <https://arxiv.org/abs/2404.16957>.
- Itzhak Gilboa and David Schmeidler. Maxmin expected utility with non-unique prior. *Journal of Mathematical Economics*, 18(2):141–153, 1989.
- GitHub. GitHub Copilot coding agent in public preview, 2025. GitHub Changelog, 19 May 2025. <https://github.blog/changelog/2025-05-19-github-copilot-coding-agent-in-public-preview/>.
- Google DeepMind. Introducing the Gemini 2.5 Computer Use model. Google blog, 7 October 2025, 2025. URL <https://blog.google/innovation-and-ai/models-and-research/google-deepmind/gemini-computer-use-model/>.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645 (8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.
- Jeremy Hadfield, Barry Zhang, Kenneth Lien, Florian Scholz, Jeremy Fox, and Daniel Ford. How we built our multi-agent research system. Anthropic engineering blog, 13 June 2025, 2025. URL <https://www.anthropic.com/engineering/multi-agent-research-system>.
- Kim Hammar, Tao Li, Rolf Stadler, and Quanyan Zhu. Adaptive security response strategies through conjectural online learning. *IEEE Transactions on Information Forensics and Security*, 20:4055–4070, 2025. doi: 10.1109/tifs.2025.3558600.
- Lewis Hammond, Alan Chan, Jesse Clifton, Jason Hoelscher-Obermaier, Akbir Khan, Euan McLean, Chandler Smith, et al. Multi-agent risks from advanced AI, 2025. arXiv:2502.14143.
- Norm Hardy. The confused deputy (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.
- Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4 (2):100–107, 1968.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, et al. An empirical analysis of compute-optimal large

- language model training. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.
- Bengt Holmström. Moral hazard and observability. *The Bell Journal of Economics*, 10(1):74–91, 1979.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- Jane Wei Huang, Quanyan Zhu, Vikram Krishnamurthy, and Tamer Basar. Distributed correlated q-learning for dynamic transmission control of sensor networks. In *2010 IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 1982–1985, 2010. doi: 10.1109/icassp.2010.5495265.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.
- Junjie Huang and Quanyan Zhu. PenHeal: A Two-Stage LLM framework for automated pentesting and optimal remediation. In *Proceedings of the Workshop on Autonomous Cybersecurity*, pages 11–22, 2023. doi: 10.1145/3689933.3690831. arXiv:2407.17788.
- Linan Huang and Quanyan Zhu. Adaptive honeypot engagement through reinforcement learning of Semi-Markov decision processes. In *Decision and Game Theory for Security*, Lecture Notes in Computer Science, pages 196–216. Springer International Publishing, 2019a. doi: 10.1007/978-3-030-32430-8\_13. arXiv:1906.12182.
- Linan Huang and Quanyan Zhu. Strategic learning for active, adaptive, and autonomous cyber defense. In *Adaptive Autonomous Secure Cyber Systems*, pages 205–230. Springer International Publishing, 2020a. doi: 10.1007/978-3-030-33432-1\_10. arXiv:1907.01396.
- Linan Huang, Juntao Chen, and Quanyan Zhu. DISTRIBUTED AND OPTIMAL RESILIENT PLANNING OF LARGE-SCALE INTERDEPENDENT CRITICAL INFRASTRUCTURES. In *2018 Winter Simulation Conference (WSC)*, pages 1096–1107, 2018. doi: 10.1109/wsc.2018.8632399. arXiv:1809.02231.
- Yunhan Huang and Quanyan Zhu. Deceptive reinforcement learning under adversarial manipulations on cost signals. In *Decision and Game Theory for Security*, Lecture Notes in Computer Science, pages 217–237. Springer International Publishing, 2019b. doi: 10.1007/978-3-030-32430-8\_14. arXiv:1906.10571.
- Yunhan Huang and Quanyan Zhu. Infinite-Horizon Linear-Quadratic-Gaussian control with costly measurements. arXiv preprint arXiv:2012.14925, 2020b. URL <https://arxiv.org/abs/2012.14925>.
- Yunhan Huang and Quanyan Zhu. Self-Triggered Markov decision processes. In *2021 60th IEEE Conference on Decision and Control (CDC)*, pages 4507–4514, 2021. doi: 10.1109/cdc45484.2021.9682918. arXiv:2102.08571.
- Yunhan Huang and Quanyan Zhu. Reinforcement learning for linear quadratic control is vulnerable under cost manipulation. arXiv preprint arXiv:2203.05774, 2022. URL <https://arxiv.org/abs/2203.05774>.
- Yunhan Huang, Linan Huang, and Quanyan Zhu. Reinforcement learning for feedback-enabled cyber resilience. *Annual Reviews in Control*, 53:273–295, 2022. doi: 10.1016/j.arcontrol.2022.01.001. arXiv:2107.00783.

- Evan Hubinger, Chris van Merwijk, Vladimir Mikulik, Joar Skalse, and Scott Garrabrant. Risks from learned optimization in advanced machine learning systems, 2019. arXiv:1906.01820.
- Garud N. Iyengar. Robust dynamic programming. *Mathematics of Operations Research*, 30(2): 257–280, 2005.
- Jacob Jackson, Phillip Kravtsov, and Shomil Jain. Improving Cursor Tab with online RL. Cursor blog, 12 September 2025, 2025. URL <https://cursor.com/blog/tab-rl>.
- Pengyi Jiang, Xiaoguang Zhu, and Quanyan Zhu. Semantic cooperative games for contribution attribution in LLM-Based Multi-Agent systems. arXiv preprint arXiv:2607.18255, 2026. URL <https://arxiv.org/abs/2607.18255>.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Jiong Jin, Zhibo Pang, Jonathan Kua, Quanyan Zhu, Karl H. Johansson, Nikolaj Marchenko, and Dave Cavalcanti. Cloud-Fog automation: The new paradigm toward autonomous industrial Cyber-Physical systems. *IEEE Journal on Selected Areas in Communications*, 43(9):2917–2937, 2025. doi: 10.1109/jsac.2025.3574587.
- Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.
- Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Paul Saldyt, and Anil B Murthy. Position: LLMs can’t plan, but can help planning in LLM-modulo frameworks. In *International Conference on Machine Learning (ICML)*, 2024.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. arXiv:2001.08361.
- Atoosa Kasirzadeh and Iason Gabriel. Characterizing AI agents for alignment and governance. arXiv preprint arXiv:2504.21848, 2025. URL <https://arxiv.org/abs/2504.21848>.
- Kimi Team, Yifan Bai, Yiping Bao, Y. Charles, Cheng Chen, Guanduo Chen, Haiting Chen, et al. Kimi K2: Open agentic intelligence, 2025. arXiv:2507.20534.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- Levente Kocsis and Csaba Szepesvári. Bandit based Monte-Carlo planning. In *European Conference on Machine Learning (ECML)*, pages 282–293, 2006.
- Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: The flip side of AI ingenuity. DeepMind blog, 2020.
- Thomas Kwa. Clarifying limitations of time horizon. METR note, 22 January 2026, 2026. <https://metr.org/notes/2026-01-22-time-horizon-limitations/>.
- Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, et al. Measuring AI ability to complete long software tasks. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, pages 92213–92266, 2025.
- John E. Laird. *The Soar Cognitive Architecture*. MIT Press, 2012.

- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, et al. Tülu 3: Pushing frontiers in open language model post-training, 2024. arXiv:2411.15124.
- John D. Lee and Katrina A. See. Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1):50–80, 2004.
- Haozhe Lei and Quanyan Zhu. Cognitive level- $k$  Meta-Learning for safe and Pedestrian-Aware autonomous driving. arXiv preprint arXiv:2212.08800, 2022. URL <https://arxiv.org/abs/2212.08800>.
- Haozhe Lei and Quanyan Zhu. Neurosymbolic Meta-Reinforcement lookahead learning achieves safe Self-Driving in Non-Stationary environments. arXiv preprint arXiv:2309.02328, 2023. URL <https://arxiv.org/abs/2309.02328>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.
- Tao Li and Quanyan Zhu. On convergence rate of adaptive multiscale value function approximation for reinforcement learning. In *2019 IEEE 29th International Workshop on Machine Learning for Signal Processing (MLSP)*, pages 1–6, 2019. doi: 10.1109/mlsp.2019.8918816. arXiv:1908.08578.
- Tao Li and Quanyan Zhu. Agentic AI for cyber resilience: A new security paradigm and its System-Theoretic foundations. arXiv preprint arXiv:2512.22883, 2025. URL <https://arxiv.org/abs/2512.22883>.
- Tao Li, Guanze Peng, and Quanyan Zhu. Blackwell online learning for Markov decision processes. In *2021 55th Annual Conference on Information Sciences and Systems (CISS)*, pages 1–6, 2021. doi: 10.1109/ciss50987.2021.9400319. arXiv:2012.14043.
- Tao Li, Juan Guevara, Xinhong Xie, and Quanyan Zhu. Self-Confirming transformer for Belief-Conditioned adaptation in offline Multi-Agent reinforcement learning. arXiv preprint arXiv:2310.04579, 2023a. URL <https://arxiv.org/abs/2310.04579>.
- Tao Li, Haozhe Lei, and Quanyan Zhu. Self-Adaptive driving in nonstationary environments through conjectural online lookahead adaptation. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 7205–7211, 2023b. doi: 10.1109/icra48891.2023.10161368. arXiv:2210.03209.
- Tao Li, Kim Hammar, Rolf Stadler, and Quanyan Zhu. Conjectural online learning with first-order beliefs in asymmetric information stochastic games. In *2024 IEEE 63rd Conference on Decision and Control (CDC)*, pages 6780–6785, 2024a. doi: 10.1109/cdc56724.2024.10886479. arXiv:2402.18781.
- Tao Li, Henger Li, Yunian Pan, Tianyi Xu, Zizhan Zheng, and Quanyan Zhu. Meta Stackelberg game: Robust federated learning against adaptive and mixed poisoning attacks. arXiv preprint arXiv:2410.17431, 2024b. URL <https://arxiv.org/abs/2410.17431>.
- Tao Li, Ya-Ting Yang, Yunian Pan, and Quanyan Zhu. From texts to shields: Convergence of large language models and cybersecurity. arXiv preprint arXiv:2505.00841, 2025. URL <https://arxiv.org/abs/2505.00841>.
- Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, et al. SkillsBench: Benchmarking how well agent skills work across diverse tasks. arXiv preprint arXiv:2602.12670, 2026a.

- Yibo Li, Zijie Lin, Ailin Deng, Xuan Zhang, Yufei He, Shuo Ji, Tri Cao, and Bryan Hooi. Just-in-time reinforcement learning: Continual learning in LLM agents without gradient updates, 2026b. arXiv:2601.18510.
- Linux Foundation. Linux foundation announces the formation of the Agentic AI Foundation (AAIF), anchored by new project contributions including Model Context Protocol (MCP), goose and AGENTS.md. Press release, 9 December 2025, 2025. URL <https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation>.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024a.
- Shutian Liu and Quanyan Zhu. Dynamic information manipulation game. arXiv preprint arXiv:2312.07862, 2023a. URL <https://arxiv.org/abs/2312.07862>.
- Shutian Liu and Quanyan Zhu. On the impact of gaslighting on partially observed stochastic control. arXiv preprint arXiv:2304.01134, 2023b. URL <https://arxiv.org/abs/2304.01134>.
- Xuan Liu, Jie Zhang, Haoyang Shang, Song Guo, Chengxu Yang, and Quanyan Zhu. Exploring prosocial irrationality for LLM agents: A social cognition view. arXiv preprint arXiv:2405.14744, 2024b. URL <https://arxiv.org/abs/2405.14744>.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective. In *Conference on Language Modeling (COLM)*, 2025. arXiv:2503.20783.
- Kevin Lu. On-policy distillation. Thinking Machines Lab blog, 27 October 2025, 2025. URL <https://thinkingmachines.ai/blog/on-policy-distillation/>.
- Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jon Kutasov, Sara Price, Naia Bouscal, et al. Natural emergent misalignment from reward hacking in production RL, 2025. arXiv:2511.18397.
- David Manheim and Scott Garrabrant. Categorizing variants of Goodhart’s law, 2018. arXiv:1803.04585.
- Andreas Matthias. The responsibility gap: Ascribing responsibility for the actions of learning automata. *Ethics and Information Technology*, 6(3):175–183, 2004.
- Meta Fundamental AI Research Diplomacy Team (FAIR), Anton Bakhtin, Noam Brown, Emily Dinan, Gabriele Farina, Colin Flaherty, Daniel Fried, et al. Human-level play in the game of Diplomacy by combining language models with strategic reasoning. *Science*, 378(6624): 1067–1074, 2022.
- METR. Time horizon 1.1. METR blog, 29 January 2026, 2026a. <https://metr.org/blog/2026-1-29-time-horizon-1-1/>.
- METR. Task-completion time horizons of frontier AI models. Web page, last updated 8 May 2026, 2026b. <https://metr.org/time-horizons/>, accessed 15 September 2026.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- Model Context Protocol Contributors. Model context protocol specification. Open protocol specification, revision 2026-07-28, 2026. URL <https://modelcontextprotocol.io>.

- National Institute of Standards and Technology. Announcing the “AI agent standards initiative” for interoperable and secure innovation, 2026. News release, 17 February 2026. <https://www.nist.gov/news-events/news/2026/02/announcing-ai-agent-standards-initiative-interoperable-and-secure>.
- Allen Newell, J. C. Shaw, and Herbert A. Simon. Report on a general problem-solving program. In *Proceedings of the International Conference on Information Processing*, pages 256–264. UNESCO, Paris; R. Oldenbourg, Munich; Butterworths, London, 1959. Conference held 15–20 June 1959; proceedings published 1960.
- Anh Tung Nguyen and Quanyan Zhu. Detection-Control games under hidden modes: Resilience-Induced blindness phenomenon. arXiv preprint arXiv:2606.29080, 2026a. URL <https://arxiv.org/abs/2606.29080>.
- Anh Tung Nguyen and Quanyan Zhu. Divergence-based safety measure for large language models via rational inattention. arXiv preprint arXiv:2606.29081, 2026b. URL <https://arxiv.org/abs/2606.29081>.
- Arnab Nilim and Laurent El Ghaoui. Robust control of Markov decision processes with uncertain transition matrices. *Operations Research*, 53(5):780–798, 2005.
- Nils J. Nilsson. Shakey the robot. Technical Note 323, SRI International, Artificial Intelligence Center, Menlo Park, CA, 1984.
- OpenAI. Computer-using agent. Announcement, 23 January 2025, 2025a. URL <https://openai.com/index/computer-using-agent/>.
- OpenAI. Introducing Codex. Announcement, 16 May 2025, 2025b. URL <https://openai.com/index/introducing-codex/>.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.
- Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2412.21139.
- Yunian Pan, Tao Li, Henger Li, Tianyi Xu, Zizhan Zheng, and Quanyan Zhu. A first order meta Stackelberg method for robust federated learning. arXiv preprint arXiv:2306.13800, 2023. URL <https://arxiv.org/abs/2306.13800>.
- Yunian Pan, Tao Li, and Quanyan Zhu. Model-Agnostic Hessian-Free meta-policy optimization via Zeroth-Order estimation: A linear quadratic regulator perspective. *Dynamic Games and Applications*, 16(1):386–425, 2026. doi: 10.1007/s13235-025-00693-9.
- Raja Parasuraman, Thomas B. Sheridan, and Christopher A. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, 30(3):286–297, 2000.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- Shishir Patil, Tianjun Zhang, Xin Wang, and Joseph Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.

- Guanze Peng, Veeraruna Kavitha, and Quanyan Zhu. On optimal control of discounted cost Infinite-Horizon Markov decision processes under local state information structures. *IFAC-PapersOnLine*, 53(2):6881–6886, 2020. doi: 10.1016/j.ifacol.2020.12.348.
- Amir Pnueli. The temporal logic of programs. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 46–57, 1977.
- Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley, 1994.
- Zhentian Qian, Jie Fu, and Quanyan Zhu. A Receding-Horizon MDP approach for performance evaluation of moving target defense in networks. In *2020 IEEE Conference on Control Technology and Applications (CCTA)*, pages 1–7, 2020. doi: 10.1109/ccta41146.2020.9206360. arXiv:2002.05146.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2024.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- Prithvi Rajasekaran, Ethan Dixon, Carly Ryan, and Jeremy Hadfield. Effective context engineering for AI agents. Anthropic engineering blog, 29 September 2025, 2025. URL <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- Anand S. Rao and Michael P. Georgeff. BDI agents: From theory to practice. In *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS)*, pages 312–319, 1995.
- Stefan Rass, Stefan Schauer, Sandra König, and Quanyan Zhu. Bounded rationality. In *Advanced Sciences and Technologies for Security Applications, Cyber-Security in Critical Infrastructures*, pages 99–114. Springer International Publishing, 2020. doi: 10.1007/978-3-030-46908-5\_5.
- R. Tyrrell Rockafellar and Stanislav Uryasev. Optimization of conditional value-at-risk. *Journal of Risk*, 2(3):21–41, 2000.
- Arturo Rosenblueth, Norbert Wiener, and Julian Bigelow. Behavior, purpose and teleology. *Philosophy of Science*, 10(1):18–24, 1943.
- Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 15 of *Proceedings of Machine Learning Research*, pages 627–635, 2011.
- Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.
- Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
- Arthur L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959.
- Leonard J. Savage. *The Foundations of Statistics*. Wiley, 1954.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can

- teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- Erik Schluntz and Barry Zhang. Building effective agents. Anthropic engineering blog, 2024. URL <https://www.anthropic.com/engineering/building-effective-agents>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. arXiv:1707.06347.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024. arXiv:2402.03300.
- Yonadav Shavit, Sandhini Agarwal, Miles Brundage, Steven Adler, Cullen O’Keefe, Rosie Campbell, Teddy Lee, et al. Practices for governing agentic AI systems. White paper, OpenAI, 2023.
- Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. RL’s razor: Why online reinforcement learning forgets less, 2025. arXiv:2509.04259.
- Thomas B. Sheridan and William L. Verplank. Human and computer control of undersea teleoperators. Technical report, Man-Machine Systems Laboratory, Massachusetts Institute of Technology, Cambridge, MA, 1978.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529:484–489, 2016.
- Herbert A. Simon. A behavioral model of rational choice. *The Quarterly Journal of Economics*, 69 (1):99–118, 1955.
- Akshit Sinha, Arvindh Arun, Shashwat Goel, Steffen Staab, and Jonas Geiping. The illusion of diminishing returns: Measuring long horizon execution in LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.09677.
- Reid G. Smith. The contract net protocol: High-level communication and control in a distributed problem solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980.
- Leon Staufer, K. J. Kevin Feng, Kevin Wei, Luke Bailey, Yawen Duan, Mick Yang, A. Pinar Ozisik, Stephen Casper, and Noam Kolt. The 2025 AI agent index: Documenting technical and safety features of deployed agentic AI systems. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pages 1536–1576, 2026. doi: 10.1145/3805689.3806728.
- Jayakumar Subramanian, Amit Sinha, Raihan Seraj, and Aditya Mahajan. Approximate information state for approximate planning and reinforcement learning in partially observed systems. *Journal of Machine Learning Research*, 23(12):1–83, 2022.
- Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A

- framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2): 181–211, 1999.
- Hamidou Tembine, Quanyan Zhu, and Tamer Basar. Risk-Sensitive Mean-Field games. *IEEE Transactions on Automatic Control*, 59(4):835–850, 2014. doi: 10.1109/tac.2013.2289711. arXiv:1210.2806.
- Gerald Tesauro. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3):58–68, 1995.
- Endel Tulving. Episodic and semantic memory. In Endel Tulving and Wayne Donaldson, editors, *Organization of Memory*, pages 381–403. Academic Press, 1972.
- Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. On the planning abilities of large language models: A critical investigation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- John von Neumann and Oskar Morgenstern. *Theory of Games and Economic Behavior*. Princeton University Press, 1944.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024a.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024b.
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning (ICML)*, 2024c.
- Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.
- Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations (ICLR)*, 2022a.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022b.
- Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2502.18449.
- Norbert Wiener. *Cybernetics: Or Control and Communication in the Animal and the Machine*. MIT Press, 1948.
- Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- Michael Wooldridge. *An Introduction to MultiAgent Systems*. Wiley, 2nd edition, 2009.

- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023. arXiv:2308.08155.
- Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwon Ding, Boyang Hong, Ming Zhang, et al. The rise and potential of large language model based agents: A survey, 2023. arXiv:2309.07864.
- Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An explanation of in-context learning as implicit Bayesian inference. In *International Conference on Learning Representations (ICLR)*, 2022.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Hua, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volume 37, 2024a.
- Xinhong Xie, Tao Li, and Quanyan Zhu. Learning from response not preference: A Stackelberg approach for LLM detoxification using non-parallel data. arXiv preprint arXiv:2410.20298, 2024b. URL <https://arxiv.org/abs/2410.20298>.
- John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.
- Ya-Ting Yang and Quanyan Zhu. PACT: A Contract-Theoretic framework for pricing agentic AI services powered by large language models. In *GLOBECOM 2025 - 2025 IEEE Global Communications Conference*, pages 5677–5682, 2025. doi: 10.1109/globecom59602.2025.11431824. arXiv:2505.21286.
- Ya-Ting Yang and Quanyan Zhu. Internet of agentic AI: Incentive-Compatible distributed teaming and workflow. arXiv preprint arXiv:2602.03145, 2026a. URL <https://arxiv.org/abs/2602.03145>.
- Ya-Ting Yang and Quanyan Zhu. A multi-resolution dynamic game framework for Cross-Echelon Decision-Making in cyber warfare. In *Game Theory and AI for Security*, Lecture Notes in Computer Science, pages 335–355. Springer Nature Switzerland, 2026b. doi: 10.1007/978-3-032-08064-6\_17. arXiv:2507.03021.
- Ya-Ting Yang and Quanyan Zhu. Toward reliable design of LLM-Enabled agentic workflows: Optimizing Latency-Reliability-Cost tradeoffs. arXiv preprint arXiv:2605.23929, 2026c. URL <https://arxiv.org/abs/2605.23929>.
- Ya-Ting Yang, Yunian Pan, and Quanyan Zhu. Preference-Centric route recommendation: Equilibrium, learning, and provable efficiency. In *2025 IEEE 64th Conference on Decision and Control (CDC)*, pages 5873–5878, 2025. doi: 10.1109/cdc57313.2025.11312511. arXiv:2504.01192.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023a.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023b.
- Mingsheng Yin, Tao Li, Haozhe Lei, Yaqi Hu, Sundeep Rangan, and Quanyan Zhu. Zero-Shot wireless indoor navigation through Physics-Informed reinforcement learning. In *2024 IEEE*

- International Conference on Robotics and Automation (ICRA)*, pages 5111–5118, 2024. doi: 10.1109/icra57147.2024.10611229.
- Justin Young. Effective harnesses for long-running agents. Anthropic engineering blog, 26 November 2025, 2025. URL <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, et al. DAPO: An open-source LLM reinforcement learning system at scale, 2025. arXiv:2503.14476.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2504.13837.
- Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W. Lin, Eliot Jones, et al. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *International Conference on Learning Representations (ICLR)*, 2025a.
- Barry Zhang, Keith Lazuka, and Mahesh Murag. Equipping agents for the real world with Agent Skills. Anthropic engineering blog, 16 October 2025; updated 18 December 2025, 2025b. URL <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-a-gent-skills>.
- Tao Zhang and Quanyan Zhu. Bayesian promised persuasion: Dynamic Forward-Looking multiagent delegation with informational burning. arXiv preprint arXiv:2201.06081, 2022. URL <https://arxiv.org/abs/2201.06081>.
- Zhili Zhang and Quanyan Zhu. Deceptive kernel function on observations of discrete POMDP. arXiv preprint arXiv:2008.05585, 2020. URL <https://arxiv.org/abs/2008.05585>.
- Yuhan Zhao and Quanyan Zhu. Stackelberg Meta-Learning for strategic guidance in Multi-Robot trajectory planning. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 11342–11347, 2023a. doi: 10.1109/iros55552.2023.10342202. arXiv:2211.13336.
- Yuhan Zhao and Quanyan Zhu. Stackelberg Meta-Learning based control for guided cooperative LQG systems. *IFAC-PapersOnLine*, 56(2):10120–10125, 2023b. doi: 10.1016/j.ifacol.2023.10.884. arXiv:2211.06512.
- Yuhan Zhao and Quanyan Zhu. Stackelberg Meta-Learning based shared control for assistive driving. arXiv preprint arXiv:2403.10736, 2024. URL <https://arxiv.org/abs/2403.10736>.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, et al. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024.
- Quanyan Zhu. Foundations of cyber resilience: The confluence of game, control, and learning theories. In *Risk, Systems and Decisions, Cyber Resilience: Applied Perspectives*, pages 27–58. Springer Nature Switzerland, 2025a. doi: 10.1007/978-3-031-90109-6\_2. arXiv:2404.01205.
- Quanyan Zhu. Game theory meets LLM and agentic AI: Reimagining cybersecurity for the age of intelligent threats. arXiv preprint arXiv:2507.10621, 2025b. URL <https://arxiv.org/abs/2507.10621>.
- Quanyan Zhu. LLM-Stackelberg games: Conjectural reasoning equilibria and their applications to spearphishing. arXiv preprint arXiv:2507.09407, 2025c. URL <https://arxiv.org/abs/2507.09407>.

- Quanyan Zhu. Reasoning and behavioral equilibria in LLM-Nash games: From mindsets to actions. arXiv preprint arXiv:2507.08208, 2025d. URL <https://arxiv.org/abs/2507.08208>.
- Quanyan Zhu. Agentomics: Economic foundations for the valuation, attribution, and pricing of AI agents in Human-AI workflows. arXiv preprint arXiv:2606.14769, 2026a. URL <https://arxiv.org/abs/2606.14769>.
- Quanyan Zhu. AI-Native insurance for agentic AI: Pricing, underwriting, and End-to-End automation. arXiv preprint arXiv:2607.13230, 2026b. URL <https://arxiv.org/abs/2607.13230>.
- Quanyan Zhu. AI tokenomics: The economics of tokens, computation, and pricing in foundation models. arXiv preprint arXiv:2606.24616, 2026c. URL <https://arxiv.org/abs/2606.24616>.
- Quanyan Zhu. Insurance of agentic AI. arXiv preprint arXiv:2606.05449, 2026d. URL <https://arxiv.org/abs/2606.05449>.
- Quanyan Zhu. Internet of agentic things: Networked AI agents for Closed-Loop IoT orchestration. arXiv preprint arXiv:2607.12662, 2026e. URL <https://arxiv.org/abs/2607.12662>.
- Quanyan Zhu. The Internet of agentic AI: Communication, coordination, and collective intelligence at scale. arXiv preprint arXiv:2606.12835, 2026f. URL <https://arxiv.org/abs/2606.12835>.
- Quanyan Zhu. A variational framework for LLM Generator-Regulator games. arXiv preprint arXiv:2606.18424, 2026g. URL <https://arxiv.org/abs/2606.18424>.
- Quanyan Zhu and Tamer Başar. Disentangling resilience from robustness: Contextual dualism, interactionism, and Game-Theoretic paradigms. *IEEE Control Systems*, 44(3):95–103, 2024. doi: 10.1109/mcs.2024.3382415. arXiv:2403.06299.
- Quanyan Zhu and Zhengye Han. A mathematical programming approach to computing and learning Berk–Nash equilibria in Infinite-Horizon MDPs. arXiv preprint arXiv:2603.13641, 2026a. URL <https://arxiv.org/abs/2603.13641>.
- Quanyan Zhu and Zhengye Han. Performative scenario optimization. arXiv preprint arXiv:2603.29982, 2026b. URL <https://arxiv.org/abs/2603.29982>.
- Quanyan Zhu and Zhiheng Xu. Introduction to partially observed MDPs. In *Cross-Layer Design for Secure and Resilient Cyber-Physical Systems*, Advances in Information Security, pages 139–145. Springer International Publishing, 2020. doi: 10.1007/978-3-030-60251-2\_10.